

Turning Data into Insight with Machine Learning for IBM z/OS

Julia Bulgannawar

Colton Cox

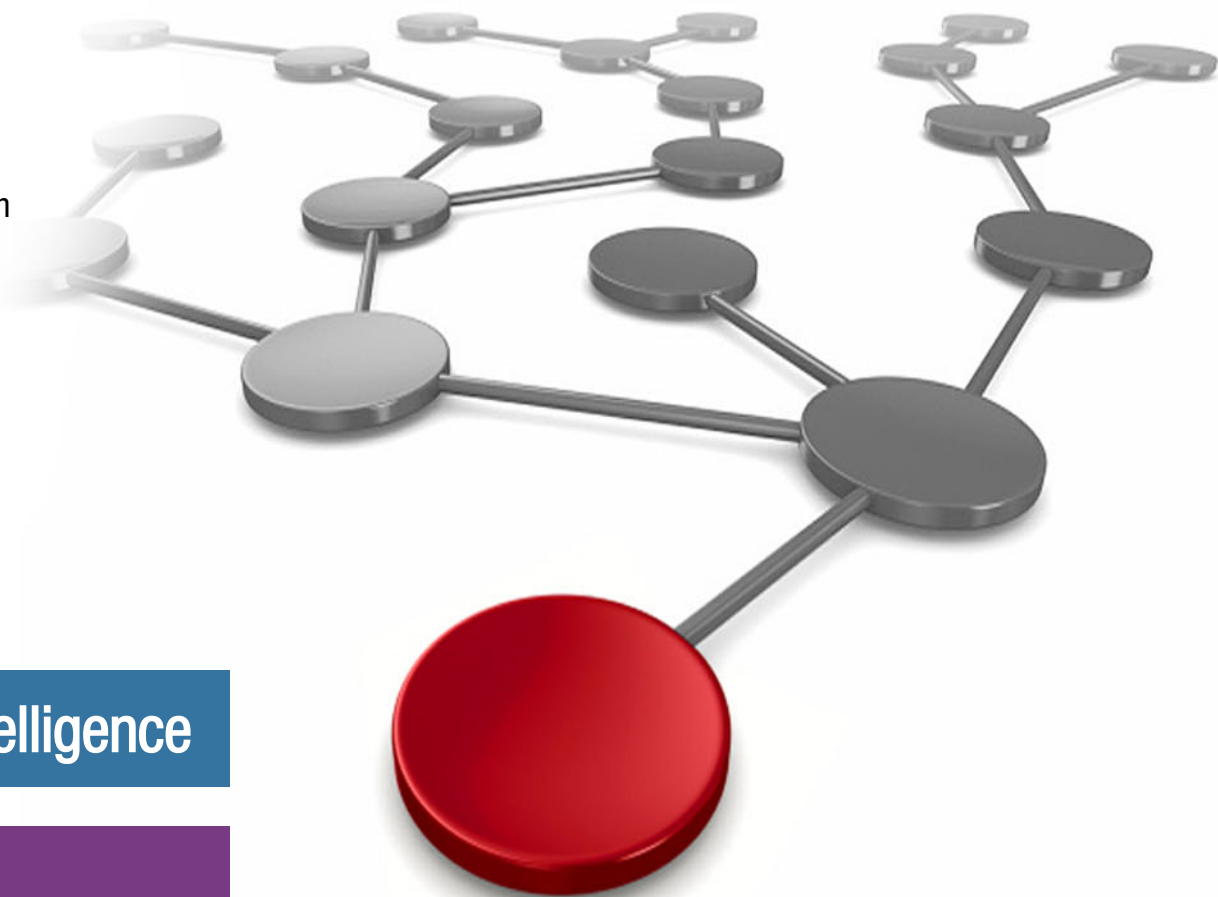
Galina Ford

Jim Owen

Mahalakshmi Vignesh

Markus Wolff

Kelly Xiang



Artificial Intelligence

IBM Z



IBM Redbooks

**Turning Data into Insight with Machine Learning for
IBM z/OS**

March 2026

Note: Before using this information and the product it supports, read the information in “Notices” on page vii.

Second Edition (April 2026)

This edition applies to Machine Learning for IBM z/OS version 3.2.

© Copyright International Business Machines Corporation 2023, 2026. All rights reserved.

Note to U.S. Government Users Restricted Rights -- Use, duplication or disclosure restricted by GSA ADP Schedule Contract with IBM Corp.

Contents

Notices	vii
Trademarks	viii
Preface	ix
Authors	x
Now you can become a published author, too!	xi
Comments welcome	xi
Stay connected to IBM Redbooks	xii
Chapter 1. Unlocking the business value of AI on IBM Z	1
1.1 AI trust and transparency: IBM's commitment	2
1.2 AI methodology - Classifications of AI	2
1.2.1 Reactive machines	2
1.2.2 Machine learning	2
1.2.3 Deep learning	3
1.2.4 Generative AI	3
1.3 AI opportunities in today's enterprises	3
1.3.1 Machine Learning on IBM z/OS and AI acceleration	3
1.3.2 AI operational patterns: sampling vs comprehensive	4
1.4 AI model lifecycle overview	4
1.4.1 Define your project goals	5
1.4.2 Choose a tool	5
1.4.3 Preparing your data	6
1.4.4 Building and training your model	6
1.4.5 Deploying and serving your model	6
1.4.6 Monitoring and governing your model	7
1.5 Using Machine Learning for IBM z/OS in IBM Z application modernization	7
1.5.1 IBM z16 with IBM Telum	7
1.5.2 Deep learning models on IBM z16 with Telum	8
1.5.3 IBM z17 with Telum II	9
1.5.4 Multiple model AI on IBM z17 with Telum II	9
1.6 Using MLz in IBM Z application modernization	9
Chapter 2. Building business-driven and strategic use cases with Machine Learning for IBM z/OS	11
2.1 Planning	12
2.1.1 Data readiness	12
2.1.2 Technical infrastructure	12
2.1.3 Organizational agility	13
2.1.4 Success metrics	13
2.2 Ideation	13
2.2.1 Ideation in business value	14
2.2.2 Feasibility	14
2.2.3 Prioritizing ideas	15
2.3 Assessment and implementation	15
2.3.1 Defining success metrics	15
2.3.2 Evaluating training data	15
2.3.3 Designing for integration and performance	16
2.3.4 Finalizing assessment criteria	16

2.4 Following the AI lifecycle	16
2.4.1 Defining project goals	17
2.4.2 Choosing a tool	17
2.4.3 Preparing the data	17
2.4.4 Training the model	17
2.4.5 Deploying your model	17
2.4.6 Serving your model for inference	17
2.4.7 Governing and monitoring your model	18
2.5 AI applications in various fields	18
2.5.1 Banking and Financial Services	18
Chapter 3. The art of data engineering	21
3.1 Nature of data	22
3.2 Characteristics of data	22
3.2.1 Data classification	22
3.2.2 Forms of data	23
3.3 Data preprocessing	24
3.3.1 Data profiling	26
3.3.2 Data cleansing	30
3.3.3 Data reduction	32
3.3.4 Data transformation	33
3.3.5 Data enrichment	35
3.3.6 Data validation	35
3.4 Data integration patterns	36
3.4.1 Data exposure pattern: Enabling modern access to IBM Z data	37
3.4.2 Data virtualization pattern: Virtualizing IBM Z data	38
3.4.3 Real-time information sharing pattern: cache IBM Z data	41
3.4.4 Data transformation pattern: Transforming IBM Z data	42
3.4.5 Data synchronization pattern: Replicating IBM Z data	44
Chapter 4. Model building for IBM Z	47
4.1 Model training frameworks	48
4.1.1 Model training frameworks and technologies on IBM Z	48
4.1.2 Open-source frameworks	49
4.2 Model formats	50
4.3 Training strategy	50
4.3.1 Train anywhere and deploy on IBM Z	50
4.3.2 Configuring a secure environment in IBM Cloud	51
4.3.3 Model development in hybrid cloud	53
4.3.4 AI reference architecture in hybrid cloud	54
4.3.5 Training on-premises with IBM Z	56
4.4 Best practices for model training	58
4.4.1 Training strategy considerations	58
4.4.2 Considerations for choosing a framework	58
4.4.3 Best practices on saving models for deployment on IBM Z	59
4.4.4 Evaluating model formats	61
4.4.5 Tuning model parameters	61
4.5 Model building use cases	61
Chapter 5. Model deployment and inferencing	65
5.1 Model deployment environments	66
5.1.1 Advantages of mainframe systems	66
5.1.2 On-premises versus cloud-based deployment	66
5.2 Model deployment on mainframe systems	67

5.2.1 Large-scale inferencing.	68
5.2.2 Leveraging data gravity.	68
5.2.3 Using existing mainframe data hubs for AI model deployment.	68
5.2.4 Ensuring data integrity, security, and compliance.	69
5.3 Model development and deployment on Machine Learning for IBM z/OS	69
5.3.1 Model development.	70
5.3.2 Model deployment.	70
5.3.3 Model monitoring	71
5.3.4 Scalability	71
5.3.5 Secure and trusted environment.	71
5.4 Model inferencing	72
5.4.1 Making online predictions	72
5.4.2 Interfaces of MLz online scoring service.	74
5.5 Sample use case for MLz model deployment	77
5.5.1 Batch job elapsed-time prediction.	77
5.5.2 Business questions and personas	77
5.5.3 Batch job elapsed time modeling building.	80
5.5.4 Applying batch job elapsed time model with MLz for application integration	92
5.6 Best practice on model deployment and inference	96
5.6.1 Optimizing ML model integration and deployment for business decisions	97
5.6.2 Choosing the right inferencing interface for your use case.	97
Chapter 6. Streamlining AI from models to applications by using machine learning operations	99
6.1 Understanding MLOps	100
6.1.1 Colocation of applications and data	100
6.1.2 Optimizing MLOps with IBM Machine Learning for z/OS	100
6.2 Model development - model training and tuning	104
6.2.1 Data preprocessing.	104
6.2.2 Hyper parameter tuning	105
6.2.3 Automated quality assurance testing	105
6.2.4 Documentation and reproducibility	106
6.2.5 Continuous improvement and feedback loop	106
6.3 Model review and deployment	106
6.3.1 Model review and validation	107
6.3.2 Model versioning.	108
6.3.3 Integrating with CI/CD pipelines	108
6.3.4 Model deployment.	110
6.3.5 Model inference and scoring.	111
6.4 Model monitoring.	112
6.4.1 Evaluating and reevaluating a model under MLz	113
6.4.2 Monitoring deployed models.	113
6.5 AI Governance and security	120
6.6 How generative AI is evolving MLOps	121
Abbreviations and acronyms	123
Related publications	125
IBM Redbooks	125
Help from IBM	125

Notices

This information was developed for products and services offered in the US. This material might be available from IBM in other languages. However, you may be required to own a copy of the product or product version in that language in order to access it.

IBM may not offer the products, services, or features discussed in this document in other countries. Consult your local IBM representative for information on the products and services currently available in your area. Any reference to an IBM product, program, or service is not intended to state or imply that only that IBM product, program, or service may be used. Any functionally equivalent product, program, or service that does not infringe any IBM intellectual property right may be used instead. However, it is the user's responsibility to evaluate and verify the operation of any non-IBM product, program, or service.

IBM may have patents or pending patent applications covering subject matter described in this document. The furnishing of this document does not grant you any license to these patents. You can send license inquiries, in writing, to:

IBM Director of Licensing, IBM Corporation, North Castle Drive, MD-NC119, Armonk, NY 10504-1785, US

INTERNATIONAL BUSINESS MACHINES CORPORATION PROVIDES THIS PUBLICATION "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Some jurisdictions do not allow disclaimer of express or implied warranties in certain transactions, therefore, this statement may not apply to you.

This information could include technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in new editions of the publication. IBM may make improvements and/or changes in the product(s) and/or the program(s) described in this publication at any time without notice.

Any references in this information to non-IBM websites are provided for convenience only and do not in any manner serve as an endorsement of those websites. The materials at those websites are not part of the materials for this IBM product and use of those websites is at your own risk.

IBM may use or distribute any of the information you provide in any way it believes appropriate without incurring any obligation to you.

The performance data and client examples cited are presented for illustrative purposes only. Actual performance results may vary depending on specific configurations and operating conditions.

Information concerning non-IBM products was obtained from the suppliers of those products, their published announcements or other publicly available sources. IBM has not tested those products and cannot confirm the accuracy of performance, compatibility or any other claims related to non-IBM products. Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products.

Statements regarding IBM's future direction or intent are subject to change or withdrawal without notice, and represent goals and objectives only.

This information contains examples of data and reports used in daily business operations. To illustrate them as completely as possible, the examples include the names of individuals, companies, brands, and products. All of these names are fictitious and any similarity to actual people or business enterprises is entirely coincidental.

COPYRIGHT LICENSE:

This information contains sample application programs in source language, which illustrate programming techniques on various operating platforms. You may copy, modify, and distribute these sample programs in any form without payment to IBM, for the purposes of developing, using, marketing or distributing application programs conforming to the application programming interface for the operating platform for which the sample programs are written. These examples have not been thoroughly tested under all conditions. IBM, therefore, cannot guarantee or imply reliability, serviceability, or function of these programs. The sample programs are provided "AS IS", without warranty of any kind. IBM shall not be liable for any damages arising out of your use of the sample programs.

Trademarks

IBM, the IBM logo, and ibm.com are trademarks or registered trademarks of International Business Machines Corporation, registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the web at “Copyright and trademark information” at <https://www.ibm.com/legal/copytrade.shtml>

The following terms are trademarks or registered trademarks of International Business Machines Corporation, and might also be trademarks or registered trademarks in other countries.

CICS®	IBM Security®	Redbooks (logo)  ®
Cloudant®	IBM Telum®	Spyre™
DataStage®	IBM Watson®	watsonx®
Db2®	IBM watsonx®	watsonx.ai®
GDPS®	IBM Z®	WebSphere®
IBM®	IBM z16®	z/OS®
IBM Cloud®	InfoSphere®	z16®
IBM Cloud for Financial Services®	OpenPages®	z17™
IBM Cloud Pak®	Promontory®	
IBM Research®	Redbooks®	

The registered trademark Linux® is used pursuant to a sublicense from the Linux Foundation, the exclusive licensee of Linus Torvalds, owner of the mark on a worldwide basis.

Windows, and the Windows logo are trademarks of Microsoft Corporation in the United States, other countries, or both.

Java, and all Java-based trademarks and logos are trademarks or registered trademarks of Oracle and/or its affiliates.

Other company, product, or service names may be trademarks or service marks of others.

Preface

The exponential growth in data over the last decade, coupled with the adoption of artificial intelligence, has opened organizations to valuable insights, efficiency improvements, and increased profits. Organizations can deliver more business value to employees, shareholders, clients, and consumers. The data that fuels these insights is the foundation for innovation. To stay ahead of the competition, organizations must capitalize on advanced AI capabilities while operating in a secure environment that protects data throughout its lifecycle and enables real-time access.

IBM® Z® provides that secure environment. Its multi-workload transactional platform powers the core business processes of many Fortune 500 enterprises and delivers security, availability, reliability, and scalability. With core transactions and data originating on IBM Z, AI should exist and run on the same platform.

Some businesses previously moved sensitive data off IBM Z. Many have reconsidered due to the massive growth of digital data, the high cost of security exposures, and the unprecedented demand for real-time actionable intelligence. Instead, businesses now embrace the strategy of data gravity for AI. *Transaction gravity* refers to the high volume of transactions that move through IBM Z. To get real-time business and operational insights at scale, organizations must colocate AI workloads with transactional applications where the data originates.

Machine Learning for IBM z/OS® is an IBM offering that helps organizations deploy machine learning models and infuse them with AI on IBM z/OS environments natively that are close to their most mission-critical workloads. In the secure Z environment, your machine-learning scoring services can coexist with your transactional applications and data. This configuration helps to support high throughput, minimize response time, and deliver consistent service level agreements (SLAs).

This book introduces Machine Learning for IBM z/OS version 3.2.0 (MLz) and describes its unique value proposition in addition to the business value of AI on the IBM Z platform. It provides guidance for you to get started with aligning your business goals and use cases for AI. It outlines the various patterns for accessing and using IBM Z data on-premises, in the cloud, and in hybrid environments. It discusses in detail the steps and best practices for model building, deployment, inferencing, and post deployment monitoring.

The book includes examples of how you can use the MLz web-based user interface to train, evaluate, and deploy a model. The book explores the value of MLz by using a batch job elapsed time prediction example and shows how to apply the batch job elapsed time model with MLz for application integration. The book provides insight into how MLz can be used within machine learning operations to run AI on IBM Z. This book also examines use cases across industries to illustrate how you can turn your data into valuable insights with MLz.

Authors

This book was produced by a team of specialists from around the world.

Julia Bulgannawar is an Experience Architect for IBM Z Simplification, which was previously known as Design Lead for Machine Learning for IBM z/OS. She helps shape the IBM Z user experience strategy for AI capability and supports clients on their AI journey on IBM Z.

Colton Cox is a Product Manager for AI on IBM Z and LinuxONE. He is responsible for planning and delivering product proof points across a portfolio of AI offerings. He formerly worked as a Design Manager and has directly supported Machine Learning for IBM z/OS for several releases.

Galina Ford is the Lead Architect and Technical Lead of Machine Learning for IBM z/OS. She has over 20 years of experience in IBM z/OS development, including development and technical leadership of products such as z/OS BCPii. She has experience in defining product enhancements that benefit IBM Z Systems customers.

Jim Owen is an AI Architect for AI on IBM Z and LinuxONE and Technical Lead for IBM Synthetic Datasets Product. With over 18 years of experience in Artificial Intelligence, Machine Learning, Deep Learning, and Generative AI, he specializes in building Enterprise AI use case architectures, Model development, optimization, and deployment using Machine Learning for z/OS. He leads the design and implementation of Multi-model and Generative AI use cases on IBM Z and LinuxONE.

Mahalakshmi Vignesh is an Engineering Project Manager with AI on Z at India Systems Development Lab (ISDL). At IBM, she has prior development experience in developing a product that enables AI on IBM z/OS. Before joining IBM 3 years ago, she was working in development of applications on IBM z/OS for various banking and insurance companies.

Markus Wolff is a Technical Specialist for IBM Z solutions in the context of Data and AI on IBM Z since 2019. He holds a Bachelor's Degree in Business Administration with focus on Industry 4.0 and Digitization. During his studies, he focused on the digitization of business models and the economical parts of IT Operations Analytics and Machine Learning. In his current role, he supports clients in Proof of Concepts or client workshops for Data Provisioning, Data Science and Artificial Intelligence in regard to IBM Machine Learning for z/OS, IBM Db2® AI for z/OS, Open Source on IBM Z / LinuxONE and the IBM watsonx® portfolio.

Kelly Xiang is a Content Designer for Machine Learning on IBM z/OS®. In her current role, she designs clear, intuitive content experiences that simplify complex AI capabilities and translate technical procedures into accessible, human-centered guidance. She also supports content for the IBM Spyre™ Accelerator for IBM Z® and LinuxONE, IBM Z Crypto Discovery & Inventory, and the AI Control Interface for IBM z/OS.

Thanks to the following people for their contributions to this project:

Lydia Parziale
IBM Redbooks®, Poughkeepsie Center

Andrew Sica, Steve Warren, Kelly Yang, Maggie Lin, Maryela Weihrauch, Sueli Almeida, Paul Cadarette, Greg Vance, Elpida Tzortzatos, Krishna Ratakonda, Kelly Yang
IBM USA

Sharon Yu, Deng Ke Zhao
IBM China

Daniel Martin, Khadija Souissi, Markus Wolff, Jan Hofmeier
IBM Germany

Thanks to the authors of the previous version of this book:

Makenzie Manna, Abid Alam, Roger Bales, Vineet Dumir, Nicholas Kunze, Jia Li, Sunil Mishra, Evan Rivera, Meng Wan, Yichong Yu

Now you can become a published author, too!

Here's an opportunity to spotlight your skills, grow your career, and become a published author—all at the same time! Join an IBM Redbooks residency project and help write a book in your area of expertise, while honing your experience using leading-edge technologies. Your efforts will help to increase product acceptance and customer satisfaction, as you expand your network of technical contacts and relationships. Residencies run from two to six weeks in length, and you can participate either in person or as a remote resident working from your home base.

Find out more about the residency program, browse the residency index, and apply online at:

ibm.com/redbooks/residencies.html

Comments welcome

Your comments are important to us!

We want our books to be as helpful as possible. Send us your comments about this book or other IBM Redbooks publications in one of the following ways:

- Use the online **Contact us** review Redbooks form found at:

ibm.com/redbooks

- Send your comments in an email to:

redbooks@us.ibm.com

- Mail your comments to:

IBM Corporation, IBM Redbooks
Dept. HYTD Mail Station P099
2455 South Road
Poughkeepsie, NY 12601-5400

Stay connected to IBM Redbooks

- ▶ Find us on LinkedIn:
<https://www.linkedin.com/groups/2130806>
- ▶ Explore new Redbooks publications, residencies, and workshops with the IBM Redbooks weekly newsletter:
<https://www.redbooks.ibm.com/subscribe>
- ▶ Stay current on recent Redbooks publications with RSS Feeds:
<https://www.redbooks.ibm.com/rss.html>



Unlocking the business value of AI on IBM Z

Artificial intelligence (AI) adoption has evolved from a nice-to-have enhancement to an essential component of business infrastructure and strategy. Companies use AI to increase efficiency and profits and deliver more business value to clients.

In a 2024 survey, 85% of executives believe AI is reshaping longstanding business models, and 89% see AI as a key driver of product and service innovation.¹ Furthermore, 64% of CEOs report they must rethink their organizational playbook to stay competitive.² AI will continue to shape how organizations plan business and technical operations.

This chapter discusses the business value of AI, particularly the value and opportunities from implementing AI in your IBM Z environment. Machine Learning for IBM z/OS (MLz) is IBM's flagship machine learning platform for developing, deploying, managing, and scaling AI models in z/OS environments, bringing AI capabilities to the secure and powerful IBM Z platform.

This chapter includes the following key topics and subtopics that provide a foundation for understanding the business value of AI on IBM Z:

- ▶ 1.1, “AI trust and transparency: IBM’s commitment” on page 2
- ▶ 1.2, “AI methodology - Classifications of AI” on page 2
- ▶ 1.3, “AI opportunities in today’s enterprises” on page 3
- ▶ 1.4, “AI model lifecycle overview” on page 4
- ▶ 1.5, “Using Machine Learning for IBM z/OS in IBM Z application modernization” on page 7
- ▶ 1.6, “Using MLz in IBM Z application modernization” on page 9

¹ <https://www.ibm.com/thought-leadership/institute-business-value/en-us/report/industries-ai-era>

² <https://www.ibm.com/thought-leadership/institute-business-value/c-suite-study/ceo>

1.1 AI trust and transparency: IBM's commitment

To ensure responsible development and deployment of new technology, IBM follows core principles that guide how it handles client data and insights. As AI becomes more integrated into business, governments are introducing regulations, such as the European Union AI Act and the US Executive Order on the Safe, Secure, and Trustworthy Development and Use of Artificial Intelligence.

IBM's trust and transparency principles are based on the following tenets:

- ▶ The purpose of AI is to augment human intelligence. AI should help you perform your job more effectively. The benefits of AI should reach everyone, not just a select few.
- ▶ Data and insights belong to their creator: the IBM client. Your data is your data, and your insights are your insights.
- ▶ Technology must be transparent and explainable. Technology companies must be clear about who trains their AI systems, what data was used in training, and what went into their algorithms' recommendations.

For more information, see [IBM's Principles for Trust and Transparency](#).

1.2 AI methodology - Classifications of AI

Recent AI innovation is focused on large language models (LLMs) and generative AI (gen AI). Although these algorithms share characteristics with traditional AI models, they offer distinct capabilities and use cases.

1.2.1 Reactive machines

Reactive machines are the most basic type of AI system. They perform specific tasks without memory or the ability to learn from past experiences. Because they cannot adapt, they always respond to identical inputs with identical outputs. This approach is now rare because of its limited memory and lack of context.

For example, IBM Deep Blue was a chess-playing expert system that competed against world champion Garry Kasparov in 1996. Deep Blue won two games and lost four. After refinements, Deep Blue competed again in 1997 and won the match with two wins and three draws. This victory marked a milestone in artificial intelligence history.

1.2.2 Machine learning

Machine learning (ML) focuses on developing algorithms and models that enable systems to learn and improve from experience without explicit programming. ML encompasses several primary techniques:

- ▶ Supervised learning. Models are trained on labeled data to make predictions or classifications.
- ▶ Unsupervised learning. Models identify patterns or structures within unlabeled data.
- ▶ Reinforcement learning. Agents learn to make decisions through reward-based feedback.

Machine Learning for IBM z/OS (MLz) applies these ML capabilities to enterprise use cases. MLz helps financial institutions scale ML models for fraud detection, anti-money laundering, and risk scoring. These models are continuously trained on large volumes of transactional

data. Unlike reactive machines that require predefined rules and logic, ML models use training and retraining methods to quickly identify new and evolving fraud techniques. MLz provides a scalable and flexible approach for organizations that need real-time AI-based scoring.

1.2.3 Deep learning

Deep learning (DL) is a subset of machine learning that uses multi-layered neural networks to mimic the human brain's decision-making capabilities. These networks, inspired by the structure of the human brain, consist of interconnected nodes that process information hierarchically. In facial recognition, for example, a deep learning model first detects basic features such as simple shapes and angles. It then identifies more complex features such as facial characteristics. At the highest level, a DL model recognizes combined features and their unique patterns for each person.

DL models have transformed fields involving complex, unstructured data, including image classification and natural language processing (NLP). Many virtual assistants use deep learning models to interpret voice commands. Autonomous vehicles use deep learning to detect objects and make driving decisions.

1.2.4 Generative AI

Generative AI (gen AI) creates new content that resembles human-created content by learning patterns and structures from existing datasets. This category includes generative adversarial networks (GANs), variational autoencoders (VAEs), and transformers. Unlike other AI systems that focus on classification and identification, gen AI produces unique outputs. In medicine and health research, gen AI assists in molecule generation and drug design by generating novel compounds with useful properties. In art and media, gen AI generates images, music, and text.

For example, OpenAI GPT-5 uses generative AI to produce human-like text from input. It can write essays, design images, and create code. ChatGPT and GPT-5 demonstrate the potential and practical applications of generative AI.

Generative AI has transformed the tech industry. Its ability to generate human-like text, images, and code has accelerated AI adoption across many industries. However, generative AI models are often trained on large amounts of unlabeled data that must be fine-tuned for different use cases. These models also require significant computational power to train and run. Therefore, consider machine learning and deep learning models for IBM Z use cases.

1.3 AI opportunities in today's enterprises

AI can accelerate business growth. Companies use AI in two primary areas: increasing revenue through customer engagement and reducing operational costs. Many companies consider AI essential for competitiveness.

1.3.1 Machine Learning on IBM z/OS and AI acceleration

For Machine Learning for IBM z/OS (MLz), AI acceleration means performing model inferencing faster or with greater accuracy. IBM z17™ introduced the capability to use encoder large language models (LLMs) for inferencing in MLz.

After scoring all transactions, the inference model generates a fraud probability score (confidence score) for each transaction. Transactions that fall below a defined threshold undergo an additional review for potential fraud by using an encoder LLM. This additional assessment increases prediction accuracy and reduces the cost of manual investigations.

For more details on Machine Learning for IBM z/OS and potential AI applications, see 2.5, “AI applications in various fields” on page 18.

1.3.2 AI operational patterns: sampling vs comprehensive

You can choose from several analysis and operational methods based on the AI use case characteristics and functional requirements. In general, the main methods fall into two categories: data sampling and comprehensive data analysis.

Data sampling

Data sampling uses a representative subset of a larger dataset to make predictions. For AI model training, you use sampling to work with large datasets that would otherwise be computationally expensive and resource-intensive. Sampling also helps reduce bias caused by imbalanced or incomplete datasets. For example, down-sampling overrepresented groups and up-sampling underrepresented groups ensures that dominant patterns do not overshadow minority ones.

Consider a fraud detection model where most transactions are legitimate. Without sampling, the model might always predict that transactions are legitimate. By balancing legitimate transactions with underrepresented fraudulent transactions, you help the model learn to detect fraudulent patterns and behaviors.

The following sampling techniques are commonly used:

- ▶ Random sampling: Selects samples at random
- ▶ Stratified sampling: Divides datasets into homogeneous subgroups (strata) and selects samples from each group
- ▶ Systematic sampling: Selects samples at intervals (for example, every 10th transaction)
- ▶ Cluster sampling: Divides datasets into heterogeneous subgroups and selects samples from each group

Comprehensive data analysis

Comprehensive data analysis examines all data in a dataset to extract insights and patterns for decision-making. Comprehensive data analysis is ideal for workloads requiring low-latency operations. For instance, business processes such as credit card approval and fraud detection require all transactions to be processed and verified. Processing all data enables AI models to capture all anomalies and edge cases in new or nuanced fraud techniques.

1.4 AI model lifecycle overview

A typical AI project follows the AI model lifecycle, an example of which is shown in Figure 1-1 on page 5. In the AI model lifecycle, each step uses outputs from preceding steps and provides input to the subsequent step.

This focuses on the AI model lifecycle and section explains the key stages in building and deploying AI projects: defining project goals, choosing tools, preparing data, building and training models, deploying models, and monitoring performance and governance.

01	02	03	04	05	06	07
Define your project goals	Choose a tool	Prepare the data	Train your model	Deploy your model	Serve model for inference	AI Governance monitor your model
What do you want to find out? Do you have the data to analyze?	Pick the tools and entry point that matches your data and desired outcome	Refine the data Add the data as a project asset or in a data repository	Train the model with the data you supply Let a model building tool choose estimators and optimize or choose your own	Move your model from development environment to a production environment	Make a deployed model accessible and usable for real time predictions or inference Exposing a model such as through a REST API	Monitor your model to evaluate the performance of models in production environments for bias and drift Provide explainability of features contributing to a prediction
Key Persona(s): - Line of Business - Data Scientist Stakeholder: - IT Architect - Application Architect Influencer: - Data Engineer	Key Persona(s): - Data Scientist Stakeholder: - IT Architect - Application Architect Influencer: - Line of Business - Data Engineer	Key Persona(s): - Data Engineer Stakeholder: - Data Owners - Data Scientist Influencer: - Line of Business - Data Wrangler	Key Persona(s): - Data scientist Stakeholder: - IT Architect - Application Architect Influencer: - Data science manager	Key Persona(s): - IT Architect - Application Architect Stakeholder: - Data Scientist - ML Engineer Influencer: - Line of Business	Key Persona(s): - ML Engineer - Data Scientist Stakeholder: - Data Scientist - ML Engineer Influencer: - Line of Business	Key Persona(s): - ML Engineer - Data Scientist - Data Engineer Stakeholder: - Data Scientist - ML Engineer Influencer: - Data Engineer

Figure 1-1 AI model lifecycle

1.4.1 Define your project goals

As AI use cases grow, defining clear project goals in the business understanding phase is critical to a successful project. In this phase, you define clear and specific project goals in business terms with measurable outcomes.

You define two elements at a high level:

1. The *what* describes what you want to achieve.

For example: Reduce high-risk credit authorizations by 5% in the next 18 months.

This specific goal aligns subsequent project activities with the business objective and desired value.

2. The *how* provides a high-level view of how you plan to accomplish the goal.

For example: Evaluate each credit request by combining the requestor's payment history and promptness with recent buying history and existing credit scores to predict future propensity to pay on time.

This description provides enough detail to guide the next steps without premature detail.

Chapter 2, "Building business-driven and strategic use cases with Machine Learning for IBM z/OS" on page 11 outlines the process and considerations for defining goals. During this stage, you should brainstorm and critique goals to ensure they are realistic, measurable, and address the right problems.

1.4.2 Choose a tool

Identify the tools and data you need to build and deploy your AI project. Determine the *what* and the *how*, and determine what data is required to achieve these goals.

You might need intermediate data, so repeatedly ask the *what* and the *how* for each data collection you identify. After defining the required data, determine what data already exists and where additional data can be produced or calculated. Creating a data map helps you understand the data requirements and develop effective plans and methods to acquire or generate the data.

Chapter 3, “The art of data engineering” on page 21 recommends various IBM solutions for specific data requirements and stages of data preprocessing. Choosing the right tools simplifies data preparation and model training and prevents costly delays or compliance risks.

1.4.3 Preparing your data

When preparing data, apply the Garbage In, Garbage Out (GIGO) principle. This principle means that poor-quality or incomplete input data produces flawed output, regardless of model sophistication. The quality of your data preparation stage directly determines the quality of your model output.

Data preparation includes several stages: collecting, cleaning, labeling, and splitting. Chapter 3, “The art of data engineering” on page 21 explains these key stages in detail. Following these methods is critical because the quality of your input data can determine the success of your AI project.

1.4.4 Building and training your model

This stage combines data science, programming, and business understanding to build machine learning and AI models. Most models include one or more algorithms that process transactions and interpret data or conditions to produce predictions. Seek insight from subject-matter experts to strengthen the model algorithm.

MLz supports Spark and Jupyter Notebook for developing and running workloads. Jupyter Notebook provides an integrated interface to build, train, and evaluate AI models. You can use the MLz-provided API in the notebook to save your model for deployment and scoring.

The final stage of model building involves designing and testing the model to validate that it produces the expected insights. Chapter 4, “Model building for IBM Z” on page 47 covers key aspects of AI model development, testing, and usage.

1.4.5 Deploying and serving your model

After you build and train the model, it is ready for deployment in test and production environments. Deploy AI models in a format that the AI inference and processing environment supports. In the early stages of deployment, follow standard test-to-production DevOps practices to validate the model on production-like data in a test environment before moving it to production.

MLz supports the following model formats:

- ▶ SparkML
- ▶ MLeap
- ▶ Predictive Model Markup Language (PMML)
- ▶ Scikit-learn
- ▶ XGBoost
- ▶ Snap ML (via PMML)
- ▶ ARIMA/Seasonal ARIMA
- ▶ Open Neural Network Exchange (ONNX)
- ▶ Watson Core Time Series

Chapter 5, “Model deployment and inferencing” on page 65 provides details about model deployment and inferencing. The chapter also describes the benefits of using Machine Learning for IBM z/OS for proper deployment and efficient inferencing to ensure a successful integration of AI and ML into practical solutions.

1.4.6 Monitoring and governing your model

Model monitoring is essential for maintaining AI model health and effectiveness. During this phase, continuously monitor the model as it processes more and varied data.

Two primary areas of focus exist during this phase: explainability and drift detection.

1. Explainability

MLz supports algorithms such as Shapley Additive Explanations (SHAP) and Local Interpretable Model-Agnostic Explanations (LIME). Schedule these explanations at predefined intervals. Each explanation includes visualizations and charts. Scheduled monitoring ensures that results and model performance align with the most recent transaction data.

2. Drift detection

MLz offers drift detection for output drift and feature drift:

- Output drift: Changes in model output from the time you trained the model to its current state
- Feature drift: Changes in categorical and continuous features by analyzing probability distributions and occurrence counts

MLz stores real transaction samples in a reference dataset that grows over time. Use this dataset as a benchmark against near real-time transaction processing. MLz automatically collects data points in the background, enabling scheduled drift detection with minimal manual intervention.

Chapter 6, “Streamlining AI from models to applications by using machine learning operations” on page 99 discusses and demonstrates the power of MLz. The chapter includes a description of the overall IBM Z infrastructure when optimizing and integrating a modern and robust machine learning operations process. A process that is essential for clients to run AI on IBM Z.

1.5 Using Machine Learning for IBM z/OS in IBM Z application modernization

IBM z16® (2022) and IBM z17™ (2025) embedded AI capabilities in the hardware. IBM z16 introduced the on-chip IBM z16 Integrated Accelerator for AI (IBM Telum®). IBM z17 expanded these capabilities to support additional AI use cases, including multiple large language models (LLMs).

MLz uses this hardware to process and analyze data. Deep learning models support in-transaction fraud detection on IBM z17 with Telum. IBM z17 and its Telum II processor support multi-model AI and LLMs, enabling intelligent decision-making in mission-critical operations with speed, security, and scalability.

1.5.1 IBM z16 with IBM Telum

Telum (Figure 1-2 on page 8) accelerates AI inference workloads. It reduces model inference time and supports real-time decision-making.

This flexible on-chip AI accelerator operates with the IBM Telum family of processors used in IBM z16 and IBM z17 systems. Each processor chip includes a centralized on-chip AI accelerator that all eight cores share and access (Figure 1-2). As a result, each core can directly use the full on-chip AI acceleration capabilities for real-time inference, rather than relying solely on isolated per-core processing.

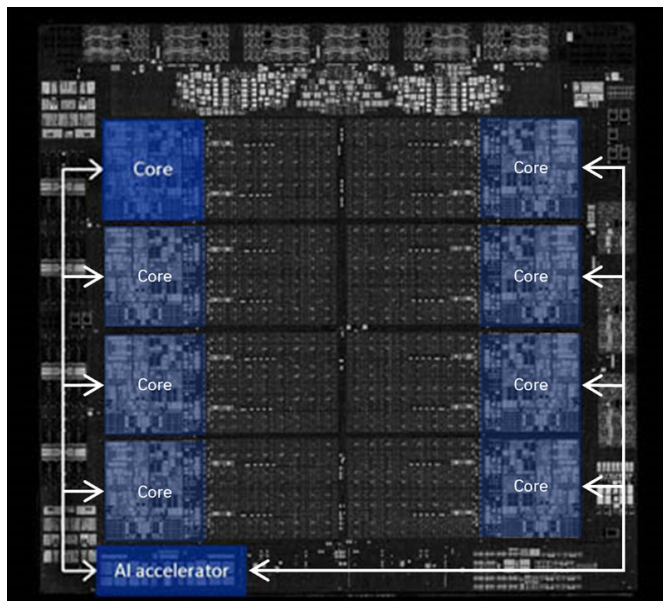


Figure 1-2 IBM Telum

1.5.2 Deep learning models on IBM z16 with Telum

Historically, banks struggled to detect fraud with AI. AI inference for fraud prevention on mainframe systems identified less than 10% of fraudulent transactions due to slow processing speeds, high costs, and customer inconvenience, leaving up to 90% of preventable fraud undetected.

Currently, AI models can run on Telum with high throughput and low latency. You can deploy deep learning models on MLz by using the Open Neural Network Exchange (ONNX) format. ONNX is open source and vendor-neutral, so you can enable model export and deployment across different environments. After you convert models to ONNX format, you can import them into MLz, deploy them, and integrate them into client transactional applications for real-time AI inferencing.

These models use the IBM Z Deep Learning Compiler, which uses ONNX–Multi-Level Intermediate Representation (ONNX–MLIR) to compile deep learning models into shared libraries. You can integrate these libraries into applications written in C, C++, Java, or Python. This approach enables deep learning models on MLz to use IBM Z technologies, including SIMD on IBM z13 and later and Telum on IBM z16, without changing the original model.

Deep learning models based on neural networks are increasingly common. According to a 2024 Celent report, IBM Z processed up to 5 million inference operations per second and up to 450 billion inference operations per day, with 1 millisecond or less response time. Deep learning models improve throughput and response times, enabling virtually all transactions to pass through deep learning-based fraud detection models.

1.5.3 IBM z17 with Telum II

IBM z17 and Telum II (Figure 1-3) expand on IBM z16 and Telum capabilities. Telum II supports multi-model AI, large language models (LLMs), and generative AI, enabling multiple types of models to run simultaneously.

With eight processor cores running at 5.5 GHz, Telum II delivers a 40% increase in on-chip cache capacity compared to Telum. Virtual L3 and virtual L4 caches increase to 2.88 GB (from 360 MB in Telum). Telum II integrates a data processing unit (DPU) for I/O acceleration. Multi-model AI support enables fraud detection with improved accuracy.

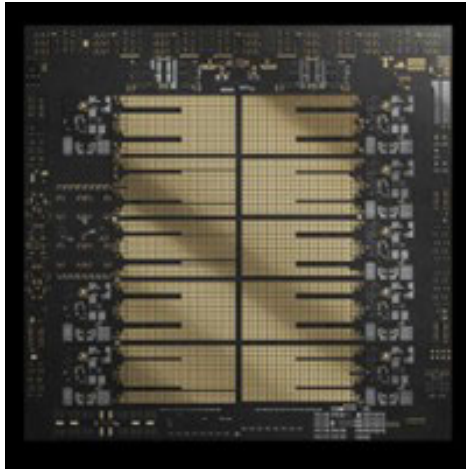


Figure 1-3 IBM Telum II

1.5.4 Multiple model AI on IBM z17 with Telum II

Relying on a single model creates trade-offs between precision and recall. Multi-model AI addresses these challenges by combining models that are optimized for different aspects of fraud detection. For example, you can use a traditional predictive model followed by an encoder-based large language model (LLM) in a fraud prediction workflow.

In this workflow, a traditional machine learning or deep learning model (such as XGBoost) first evaluates all transactions for low-latency scoring. This layer identifies clear fraud cases. Transactions with a fraud probability score below a predefined confidence threshold (for example, less than 80%) escalate to a second layer with an encoder-based LLM (such as BERT). This model refines predictions for borderline cases.

This two-tier inference strategy combines the speed of traditional AI with the contextual analysis of LLMs, balancing accuracy and throughput. It improves decision quality while meeting SLA targets on IBM z17 systems with the Telum II processor and MLz.

1.6 Using MLz in IBM Z application modernization

Most mission-critical applications worldwide run on mainframes. Mainframe operations are as modern and powerful as any other platform, but many applications were designed before AI became widely adopted. Many of these applications were written in languages such as COBOL and PL/I, where major code changes can disrupt workloads.

Clients are modernizing these applications and interfaces to integrate AI. Machine Learning for IBM z/OS (MLz) modernizes applications with minimal code changes, extending the value of existing investments and bringing AI workloads to IBM Z.

IBM Z clients use the following methods to modernize applications with AI and MLz:

- ▶ REST APIs

REST APIs connect systems and services over HTTP requests. With MLz, you can expose machine learning models as services, enabling IBM Z applications to score or infer transactions.

- ▶ IBM CICS® exec link command

For CICS COBOL applications, you can use the CICS exec link command to invoke MLz services directly from the application with minimal changes. This method enables high-throughput, low-latency scoring and inference.

- ▶ WOLA interface

For IMS COBOL applications, WOLA provides a high-performance bridge between IBM WebSphere® for z/OS and COBOL, PL/I, C, and C++ applications in external address spaces. These applications run on the same z/OS system using shared memory between processes. Use this interface to access MLz scoring capabilities for high-volume transactions and low-latency SLA requirements.

- ▶ Java interface

For Liberty or WebSphere Application Server (WAS) applications, configure MLz as a Liberty feature within the Liberty or WAS server that hosts the application. This configuration enables the application to use the MLz-provided native scoring Java interface.

For an overview of IBM's strategy to help you modernize applications faster, see the IBM Redbooks publication [Accelerate Mainframe Application Modernization with Hybrid Cloud](#), REDP-5705.



Building business-driven and strategic use cases with Machine Learning for IBM z/OS

Before developing an AI use case, identify the problems that AI can solve and why AI provides value. By using objective evaluation, you can define the right solution.

It is estimated that only half of AI projects reach production. Organizations struggle to implement AI due to lack of expertise, insufficient data, or limited business value. Structured planning and alignment with business goals are essential. A problem-centric approach that focuses on areas of opportunity and steps toward a solution is more effective than starting with a predefined solution.

This chapter describes proper planning from concept to execution. Review your business plans for market opportunities and existing challenges. Based on objective criteria, identify which opportunities and challenges AI can address then backward to explore existing data sources where AI can unlock business value.

This process includes the following steps:

- ▶ Planning: Includes a self-assessment of organizational readiness for AI adoption.
- ▶ Ideation or Brainstorming: Generate ideas for potential AI use cases.
- ▶ Assessment: Evaluate ideas iteratively for viability and alignment with business goals.
- ▶ Execution: Start with minimal viable products (MVPs) and assess key performance indicators (KPIs) for each use case.

This chapter includes the following topics:

- ▶ 2.1, “Planning” on page 12
- ▶ 2.2, “Ideation” on page 13
- ▶ 2.3, “Assessment and implementation” on page 15
- ▶ 2.4, “Following the AI lifecycle” on page 16
- ▶ 2.5, “AI applications in various fields” on page 18

2.1 Planning

As organizations adopt both traditional and generative AI to improve business outcomes, planning is essential. Without planning, projects often fail to reach production or provide poor ROI due to unclear business objectives, insufficient data readiness, or misaligned stakeholder expectations.

Planning is critical on IBM Z. AI solutions must integrate seamlessly with existing systems without compromising resiliency, security, or performance.

Organizational readiness extends beyond technical capability. In an AI-ready culture, you can establish governance frameworks and ensure teams have the skills to implement and maintain AI solutions.

A key starting point is Identifying the primary business driver: cost reduction, risk mitigation, operational efficiency, customer experience improvement, or a combination. These drivers shape key performance indicators (KPIs) and establish success metrics.

Keep initial use cases limited in scope. Begin with pilots that scale gradually, support iteration, and enable upskilling and cultural change. This approach reduces risk while building confidence and momentum for broader AI adoption.

As part of your planning process, identify your organization's strengths, maturity, and areas for improvement related to AI adoption.

2.1.1 Data readiness

Good data is the foundation of success for an AI model. The IBM Z environment and applications can provide good data resources to train AI models. You must ensure this data can be accessed, used, and protected during training.

The following aspects define data readiness:

- ▶ **Accessibility:** Data can be accessed across systems.
- ▶ **Integration:** Data is integrated across platforms to support AI workflows.
- ▶ **Governance and Compliance:** Frameworks ensure data privacy and regulatory compliance.
- ▶ **Scalability:** Ensure that your data strategy scales to include hybrid frameworks but keeps IBM Z at the center of operations.

2.1.2 Technical infrastructure

Determine whether your systems can support AI workloads. Assess the following areas:

- ▶ **Compute and storage scalability:** Ensure that infrastructure scales to meet the demands of AI training and inferencing
- ▶ **Monitoring and observability:** Verify tools track AI performance and resource utilization
- ▶ **Integration with mission-critical workloads:** Integrate AI inferencing directly into transactional workflows through on-chip processing, such as the IBM Telum II accelerator

2.1.3 Organizational agility

Organizations must be flexible so that their personnel can implement new processes and test the implementation before introducing the processes into production.

Experimentation and Iteration

Planning an AI use case requires experimentation and iteration. Organizations that expect perfection in the first attempt might limit learning and set themselves up for failure.

Successful organizations focus on incremental value that scales and delivers long-term benefits (AI in Action). This approach emphasizes continuous improvement and iterative development, ensuring AI solutions evolve to meet business needs. For more information, see [AI in Action](#).

Governance and Trust

Define expectations for establishing trust in AI use cases and determine the governance structure. Document the following considerations:

- ▶ Essential reviewers and approvers for implementation
- ▶ Stakeholders to consult before moving a use case to production
- ▶ Response procedures when AI model effectiveness drifts over time
- ▶ Documenting these considerations early helps determine how to implement the solution.

Talent and Skills

Assess available talent and skills. Determine whether the team has sufficient data science skills to build a model and deploy it through a machine learning pipeline. If not, identify tools and resources that fit the team's skill set. Consider upskilling the team to handle end-to-end AI use cases. What processes do your teams have in place today for experimentation and iteration? Planning an AI use case requires a willingness to try, fail, and try again to pursue the most optimal solution. An organization that expects perfection in the first attempt might set itself up for failure by limiting this learning process.

2.1.4 Success metrics

Define the specific KPI tied to a core business aspect that indicates success, such as operational costs, customer engagement, or new business. Break down the KPI into smaller, trackable components for iterative implementation. For example, if the ultimate goal is to drive revenue through an AI-enhanced e-commerce offering, an appropriate first KPI might be increasing the average number of items per customer cart.

These considerations provide a framework for adopting and scaling AI. Documenting consensus with stakeholders early positions the organization for success.

2.2 Ideation

After completing a self-assessment of AI readiness, identify actionable opportunities to enhance business through machine learning. Ideation generates and refines ideas that align with business drivers, technical feasibility, and organizational maturity.

Track suggestions against a prioritization matrix. For example, create a grid where the X-axis measures the potential impact of an idea and the Y-axis measures feasibility. This matrix

helps determine which ideas to pursue now, which need further investment, and which to deprioritize.

Use case ideation is the natural next step after your organization has conducted a thorough self-assessment of its readiness for AI. With your priorities now clearly documented and visible, it's time to imagine actionable opportunities to enhance your business through machine learning. Ideation is the process of generating and refining ideas that align with your business drivers, technical feasibility, and organizational maturity.

As you explore possibilities, consider tracking suggestions against a prioritization matrix. or example, a grid where the X-axis measures the potential impact of a given idea, and the Y-axis measures how feasible it would be for your organization to pursue. This can help you visualize which ideas are worth pursuing now, which might need further investment, and which should be deprioritized.

2.2.1 Ideation in business value

Revisit the strategic drivers from the planning phase. Identify which success metrics matter most: cost reduction, risk mitigation, customer experience, or operational efficiency. Determine which processes or pain points most directly affect these outcomes. Identify manual or time-consuming components of business processes that could benefit from automation or predictive insight.

Focus on opportunities that deliver sustained impact rather than short-term quick wins that might not scale. An ideal proof of concept demonstrates value early while remaining adaptable to evolve into a broader solution or support adjacent use cases over time.

IBM Z workloads are mission-critical, and many business processes benefit from machine learning. This chapter details examples later.

2.2.2 Feasibility

Assess the feasibility of promising ideas. Consider the following concerns:

- ▶ Data needed for the use case is accessible, governed, and sufficient for training
- ▶ Data is in a format that can be extracted and used to train a model
- ▶ Technical, ethical, or compliance-related risks exist in using this data

Consider how the model integrates into existing systems. Machine Learning for z/OS offers the following deployment options for IBM Z environments:

- ▶ Host the MLz scoring engine on a WebSphere Liberty server within the CICS runtime so CICS applications can call the scoring service through EXEC CICS LINK
- ▶ Use WebSphere Optimized Local Adapter (WOLA) APIs for IMS COBOL applications
- ▶ Use native APIs for Java applications running in WebSphere
- ▶ Use REST API calls through IBM z/OS Connect or the z/OS client web enablement toolkit

Each of these methods is documented in detail in [Infusing AI into Applications on IBM Z](#).

Feasibility includes organizational readiness. Determine whether the organization has the skills to deploy, maintain, and improve the use case over time. If not, identify paths to close the gap, such as upskilling, partnerships, or tooling.

Consider who will be directly impacted by the use case: who operates it and whose role might change. Gather input from front-line users and decision-makers to reveal potential disruptions or benefits that might not be visible from a strategic perspective.

2.2.3 Prioritizing ideas

Prioritize, document, and communicate ideas and rank ideas one to three idea for further exploration. Document the rationale for each: why it is a good fit, how it aligns with organizational readiness, and what success looks like.

Share these ideas with stakeholders to build alignment and secure sponsorship. Position each use case as iterative: learn from early results, adjust, and test before measuring against final KPIs. This mindset helps build resilience and encourages continuous improvement.

2.3 Assessment and implementation

At this point in the process, business drivers, data posture, and technical constraints are clear. High-priority use cases are identified and aligned. The assessment stage translates these ideas into a testable, actionable plan.

This phase focuses on precision. Define KPIs, set acceptance thresholds, and design a model that upholds IBM Z values of resiliency, security, and performance.

The assessment phase determines whether and how to proceed with each use case, with defined decision gates, timelines, and ownership.

2.3.1 Defining success metrics

Earlier phases defined broad KPIs tied to business outcomes. Refine those goals into specific, measurable targets. For example, a general goal to reduce processing time for loan applications becomes reduce average processing time by 20%. This target represents a meaningful improvement over the current baseline and is achievable within a proof of concept.

Define acceptance thresholds for the use case and include the following considerations:

- ▶ Maximum allowable impact on system performance.
- ▶ Thresholds for model accuracy, latency, or false positives to maintain business continuity.
- ▶ Methods for tracking these metrics. Identify systems or dashboards, assign monitoring responsibilities, and establish how to communicate results to stakeholders.

2.3.2 Evaluating training data

Examine the data required to support the use case. Assess the following aspects of the data:

- ▶ Missing fields or inconsistencies that affect model performance
- ▶ Risks of bias or fairness concerns in the dataset
- ▶ Compliance or privacy issues (such as PII or regulatory constraints) that require attention

If gaps or risks exist, outline mitigation approaches. Mitigation includes data cleansing, enrichment, anonymization, or rebalancing. Scope data engineering work required to prepare the dataset for modeling. This work includes transformation pipelines, feature extraction, or

integration with external sources. Chapter 3, “The art of data engineering” on page 21 explores these approaches in more detail.

2.3.3 Designing for integration and performance

Determine how the AI model is deployed and accessed by business applications. Machine Learning for IBM z/OS provides several deployment options. Select the approach that fits the use case.

Assess the following integration considerations:

- ▶ How the model is invoked (REST API, Liberty server in CICS, WOLA, or another method)
- ▶ Latency, throughput, and security requirements for integration
- ▶ Infrastructure to support the model's runtime demands

Diagram the integration path. This surfaces dependencies, clarifies ownership, and ensures the deployment plan aligns with IBM Z performance and resiliency standards.

2.3.4 Finalizing assessment criteria

Success metrics, data readiness, and integration design determine whether AI and Machine Learning for IBM z/OS is appropriate. For each use case, assess the following criteria:

- ▶ Business value is clearly defined and measurable
- ▶ Data is sufficient, trustworthy, and compliant
- ▶ Model deployment does not disrupt critical systems
- ▶ Skills and resources exist to build, operate, and maintain the model

If any assessment is not yet met, develop a mitigation plan before proceeding.

2.4 Following the AI lifecycle

These following sections define the steps for AI use cases. The steps define goals and expectations to enable teams to choose appropriate tools and prepare data for model training and deployment.

These steps represent early stages of the AI lifecycle, a standard end-to-end process for AI use cases. Machine Learning for IBM z/OS addresses key considerations at each stage, providing a shared platform for managing, deploying, and evaluating AI models on IBM z/OS.

Each stage of the lifecycle involves different key personas, stakeholders, and influencers. A key persona is an essential party who advances the stage. A stakeholder informs the direction of the stage. An influencer is someone whose role or perspective is considered in decision-making.

Determine who to include at each step of the lifecycle.

2.4.1 Defining project goals

This stage involves activities stated earlier in this chapter, including determining problems that AI can solve, assessing data readiness, and setting KPIs for your project. Include the following personnel:

- ▶ Key personas: Line of business leader, data scientist
- ▶ Stakeholders: IT architect, application architect
- ▶ Influencers: Data engineer, machine learning engineer

2.4.2 Choosing a tool

Throughout defining your project goals, consider which tools that you want to use. At this stage, select the tools and entry point that match your data and desired outcomes. Include the following personnel:

- ▶ Key personas: Data scientist
- ▶ Stakeholders: IT architect, application architect
- ▶ Influencers: Line-of-business leader, data engineer, machine learning engineer

2.4.3 Preparing the data

Refine or clean your data to make it usable for your use case. If necessary, add the data as a project asset in a tool that is accessible in your machine learning operations pipeline. Include the following personnel:

- ▶ Key personas: Data engineer
- ▶ Stakeholders: Data owners, data scientist
- ▶ Influencers: Line-of-business leader, data wrangler

2.4.4 Training the model

Supply the model with prepared data to align it with your use case or fine-tune an existing model suited to the use case. Include the following personnel:

- ▶ Key personas: Data scientist
- ▶ Stakeholders: IT Architect, Application Architect
- ▶ Influencers: Data science manager

2.4.5 Deploying your model

After your model is trained, tested, and validated, move it from your development environment to a production environment. Include the following personnel:

- ▶ Key personas: IT Architect, Application Architect
- ▶ Stakeholders: Data Scientist, Machine Learning Engineer
- ▶ Influencers: Line-of-business leader

2.4.6 Serving your model for inference

After deployment, make the model accessible for near real-time predictions or inference, such as exposing the model through a REST API. Include the following personnel:

- ▶ Key personas: Machine Learning Engineer, Data scientist
- ▶ Stakeholders: Machine Learning Engineer, Data scientist

- Influencers: Line-of-business leader, Legal team

2.4.7 Governing and monitoring your model

Continually evaluate the performance of your models in production environments to detect bias, drift in model quality, or other concerns that might signal a need to retrain your model. Include the following personnel:

- Key personas: Legal team, Machine Learning Engineer, Data Scientist, Data Engineer
- Stakeholders: Machine Learning Engineer, Data scientist
- Influencers: Data Engineer

2.5 AI applications in various fields

This section describes AI use cases for IBM Z.

According to the IBM Global AI Adoption Index¹ (2023), 42% of companies reported using AI in their business, , and 40% are exploring AI. Common use cases include IT process automation, IT and software management automation, and customer care automation. IBM Z plays a central role in these domains, enabling users to deploy AI use cases that deliver business value.

The following examples demonstrate how Machine Learning for IBM z/OS delivers value across industries. These examples illustrate possibilities and help you envision how AI can deliver measurable impact while using IBM Z resiliency, security, and performance.

2.5.1 Banking and Financial Services

IBM Z processes the majority of the world's high-value transactions, making it a natural fit for AI-driven insights in real time. Here are some common use cases you can explore

Payments fraud detection

A large European bank needed to identify fraudulent transactions in real time across high payment volumes while reducing manual verification and meeting SLA requirements. Banks historically exported transaction data to external anti-fraud systems, which introduced latency and limited real-time analysis to small samples, creating gaps where fraud occurs.

IBM z17's Telum II on-chip AI accelerator provides eight times the AI processing units of z16. Deep learning models run on IBM Z, scoring every transaction in-flight. This enables up to 5 million inference operations per second (450 billion per day) with sub-millisecond response times, eliminating delays and reducing fraud risk.

According to Celent, if all banking, card, and payment institutions on IBM Z applied advanced AI models for fraud detection, global fraud losses could decrease by \$190 billion annually: \$165 billion in bank fraud and \$25 billion in card and payment fraud.

For more details, see *Solving challenges of instant payments by using AI on IBM zSystems*, REDP-5698.

¹ <https://www.ibm.com/downloads/cas/GVAGA3JP>

Overdraft limit protection

You can use AI to augment rules-based systems for overdraft approvals and dynamically adjust limits based on real-time risk scoring. This approach reduces exposure and maintains customer experience.

Payment system contingency

When your primary balance monitoring system is unavailable, you can use AI models to score payment risk based on historical trends and client behavior. This enables informed decisions without disrupting operations.

Anti-money laundering (AML) in instant payments

Stricter regulations require AML screening during instant payment flows. You can use AI to perform real-time AML checks without breaching SLA requirements, improving compliance and reducing risk.

Loan approval processing

You can use AI-driven scoring to accelerate consumer loan approvals, improve customer satisfaction, reduce latency by over 1,000x, and lower operational risk.

Cross-industry applications

IBM Z security and resiliency support AI workloads, even when core transactions originate elsewhere. AI scoring can be migrated from cloud environments to IBM Z for performance and compliance advantages. The following examples are from production environments:

- ▶ Insurance underwriting
A U.S. insurer uses AI to identify risk factors for home insurance underwriting and set competitive premiums. Underwriting time decreased from hours to seconds, improving customer experience.
- ▶ Claims fraud detection
A U.S. state government automated its manual fraud detection process, reducing case review time from 40 hours to seconds.
- ▶ Geospatial analysis and NLP
A European land registry uses AI to detect property changes for tax purposes and provides chatbot services for residents. Deployment on IBM z17 ensures data security and optimizes IT and operational costs.

The next step is to identify similar high-impact opportunities in your workflows by using the planning, ideation, and assessment frameworks discussed in this chapter.



The art of data engineering

Data is a critical asset. Transactional data provides insights and business opportunities. This chapter describes data characteristics, data analysis, and patterns for accessing mainframe data on-premises or in the cloud.

- ▶ 3.1, “Nature of data” on page 22
- ▶ 3.2, “Characteristics of data” on page 22
- ▶ 3.3, “Data preprocessing” on page 24
- ▶ 3.4, “Data integration patterns” on page 36

3.1 Nature of data

Organizations create, store, and analyze increasing volumes of data. Global data creation is projected to reach 180 zetabytes by 2025.

Organizations receive large volumes of diverse data daily. Curating data and extracting insights provides competitive advantages.

AI systems can learn from historical data to improve decision-making. High-quality data produces efficient, high-performing AI systems.

Raw data from different sources varies widely and requires preprocessing before it is used in AI systems. Raw data might contain missing entries, fake information, hate, abuse, and profanity (HAP), or proprietary and sensitive information prohibited by law. This chapter describes data preprocessing steps and techniques.

3.2 Characteristics of data

Data volumes can grow exponentially. Finding the right dataset is essential for AI projects. The following questions help identify suitable datasets:

- ▶ What is the business problem and preferred outcome?
- ▶ What data is needed and what data sources can be accessed?
- ▶ What is the data volume, velocity, variety, and veracity?
- ▶ What are the constraints and limitations (security, compliance, governance, auditing)?
- ▶ How much data is required to train and evaluate the model?
- ▶ Does the dataset require data labels or annotations?
- ▶ Does the problem benefit from a diverse dataset or specific characteristics?
- ▶ What preprocessing steps are needed?
- ▶ Are resources available for data collection and preparation?

3.2.1 Data classification

Data classification separates and organizes data into categories based on characteristics. Classify data based on sensitivity or confidentiality level. Data sensitivity classification supports risk management and regulatory compliance. Define secure data handling policies for each classification category.

The following are common data categories:

- ▶ **Public data:** Available to the public and not sensitive. Examples include company name and address, and product documentation.
- ▶ **Private data:** Not available to the public. Compromised data poses risk to an individual or organization. Examples include personal phone numbers and employee IDs.
- ▶ **Internal data:** Organization data with access limited to employees. Examples include internal websites and server IPs
- ▶ **Confidential data:** Sensitive data available only to authorized personnel. The loss of confidential data harms an individual or organization. Examples include Social Security numbers and employee records.

- **Restricted data:** Highly sensitive data with strictly controlled access. The loss of restricted data significantly harms an individual or organization. Examples include trade secrets and financial records.

Handle data with different sensitivity levels as needed. For example, Personally Identifiable Information (PII) is private, and it can be used individually or jointly with other information to identify a single person. Because of its sensitive nature, numerous laws worldwide regulate how PII can be stored and processed. Regulations include the General Data Protection Regulation (GDPR) from Europe, Health Insurance Portability and Accountability Act (HIPAA), Gramm-Leach-Bliley Act (GLBA) and others from US, and the Personal Information Protection and Electronic Documents Act (PIPEDA) from Canada.

3.2.2 Forms of data

Data exists in the following forms:

- **Structured data** is usually tabular with fields that contain predefined formats. Some fields have strict formats (phone numbers, addresses), and others have variable-length text strings (names, descriptions). Structured data is normally stored in a relational database and is typically easy to organize, query, and analyze, but it might take more space.
- **Unstructured data** includes various data types that are stored in native formats, such as documents, videos, audio files, and social media posts. Unstructured data is stored in native formats and can be hard to organize and analyze but can take less space.
- **Semi-structured data** is not tabular but has organizational properties, such as XML or JSON files. Semi-structured data can be organized and analyzed with some processing.

Data can be any of the following types:

- **Numerical Data** is data, which consists of numerical values. Numerical data can be of two types:
 - **Discrete Data** is countable numerical data, such as the number of products in inventory and the number of students in a class.
 - **Continuous Data** is data that you continually measure, such as the height or weight of a person.
- **Categorical data** represents distinct categories used to label data into non-numeric classifications. Categorical data can be stored and retrieved based on assigned names and labels. Categorical data includes two types:
 - a. **Ordinal data:** Ordinal data has a natural or meaningful order or ranking as in the following examples:
 - Level of education (high school diploma, college diploma)
 - Economic status (lower income, middle income, higher income)
 - Customer satisfaction ratings (very dissatisfied, neutral, very satisfied)
 - b. **Nominal data:** Nominal data has no inherent order or ranking. It categorizes and labels data without quantitative relationships as in the following examples:
 - Eye color (blue, green, brown, hazel, gray)
 - Marital status (single, married, divorced, widowed)
 - Car makes and models (Ford, Honda, Toyota, Camry, Civic, Focus)

Understanding field data types, value ranges, and distributions is essential for data analysis.

Data is stored in different sources and formats. Relational databases (such as IBM Db2) store tabular data. Non-SQL databases (such as IBM Cloudant®) store JSON format data. Mainframes store sequential datasets (such as VSAM). Cloud Object Storage stores parquet

and open table formats (such as Apache Iceberg). Data resides at different locations: on-premises, in the cloud, or at edge or client locations. AI projects often connect to or extract data from various sources.

Data localization laws enforce data processing requirements within territories. Data sovereignty laws govern data control and storage. Companies that handle data must understand and comply with these laws.

Training AI models requires large volumes of data. Depending on the model, large volumes of annotated and labeled data might be needed, requiring significant human preprocessing. Foundation models use self-supervised learning to create labels from input data. These models can be fine-tuned with less data for domain-specific tasks or use prompt engineering to generate output for various tasks. Foundation models might require more computational power depending on model size. Customer care assistants and code assistants are common foundation model use cases.

Datasets have different characteristics. The goal is to generate business insights and help decision making with the various datasets. Start with the business problem and choose the right datasets to address the business problem. Also consider dataset characteristics, laws, and regulations when handling and storing data.

3.3 Data preprocessing

Raw data must undergo preprocessing before you use it in models. Data preprocessing cleans and transforms raw data into a format suitable for models. Preprocessing maximizes data quality and relevance and enables optimal model performance.

Data preprocessing consists of the following steps (Figure 3-1 on page 25):

1. Data profiling

Data profiling analyzes and assesses the structure, content, relationships, and quality of raw datasets. This process identifies data quality issues, risks, and overall patterns and trends. Data profiling is essential for preparing quality datasets for AI.

2. Data cleansing

Data cleansing identifies and corrects inaccuracies, corruptions, incorrect formatting, duplicate records, missing values, and inconsistencies. This ensures data is accurate, complete, and reliable for AI.

3. Data reduction

Data reduction simplifies datasets and preserves data integrity and quality. This process uses optimization techniques (such as feature selection or sampling) to reduce data volume, improve computational efficiency, and reduce noise.

4. Data transformation

Data transformation changes the format, structure, or values of data to suit specific tasks for ML/AI models.

5. Data enrichment

Data enrichment enhances raw data by adding information and context to improve relevance, value, and usefulness for analysis or ML tasks. Key aspects include integrating data from external sources, combining data from multiple sources for comprehensive datasets, and feature engineering. Enriched data provides added context and insights for decision-making and AI/ML model training.

6. Data validation

Data validation verifies that preprocessed data is accurate, consistent, complete, and suitable for the intended modeling task. This process identifies errors, anomalies, and issues before further analysis, which is essential for accurate and trustworthy results.

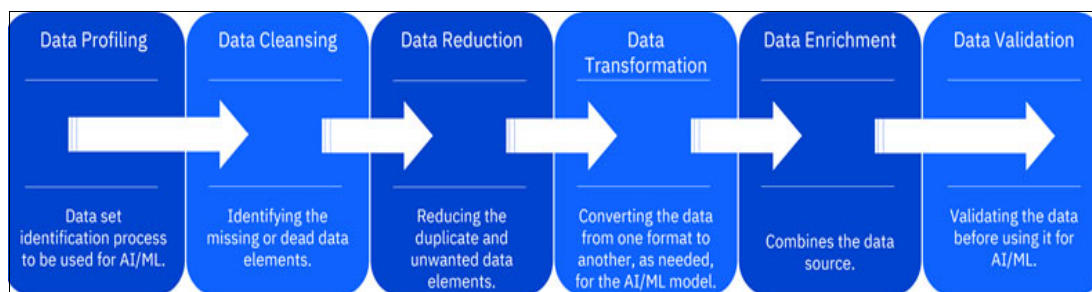


Figure 3-1 Process of understanding the nature of data

Data scientists commonly use Python for AI tasks because of its extensive libraries. Python libraries for data preprocessing include NumPy, Pandas, Scikit-learn, and Matplotlib.

Section 1.3.1, “Machine Learning on IBM z/OS and AI acceleration” on page 3 describes a credit card fraud detection use case and demonstrates how IBM z17 enables in-transaction model inference using the Integrated Accelerator for AI. The synthetic credit card fraud detection model and dataset for training and testing the model are available at the GitHub site [ai-on-z-fraud-detection](#).

This section uses the synthetic credit card fraud detection dataset to demonstrate data preprocessing steps with Python libraries.

IBM provides tools for data preprocessing tasks. The data refinery tool in IBM Watson® Studio and IBM Watson Knowledge Catalog (on-premises and cloud) transforms large amounts of raw data into high-quality information for analytics.

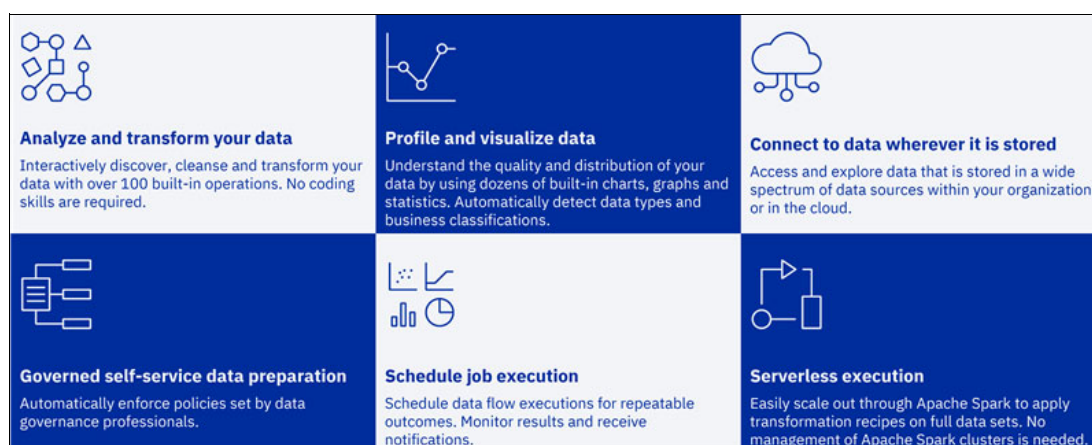


Figure 3-2 IBM data refinery features

For more information on the data refinery tool available in IBM watsonx®.ai® AI Studio and IBM Watson Knowledge Catalog, see [Data refinery](#).

3.3.1 Data profiling

Data profiling analyzes the structure, content, relationships, and quality of raw datasets. This process provides insights into data characteristics and quality through review, summarization, and evaluation.

Data profiling helps organizations understand their data and verify that data consists of standard statistical measures.

Analysts might use the following approaches to profile your data¹:

- ▶ **Structure discovery**
Structure discovery focuses on data format and consistency throughout the database. Pattern matching helps identify format-specific issues. For example, pattern matching can detect missing values in phone number formatting.
- ▶ **Content discovery**
Content discovery analyzes data rows for errors or systemic issues. This process examines individual database elements to find incorrect values.
- ▶ **Relationship discovery**
Relationship discovery identifies data in use and finds connections between datasets. Analysts use metadata analysis to determine relationships between data and narrow connections between specific fields.

Analysts also use the following data profiling methods and techniques:

- ▶ **Column profiling**
Column profiling scans tables and counts the frequency of each value within columns. This method identifies frequency distribution and patterns within a column.
- ▶ **Cross-column profiling**
Cross-column profiling includes key analysis and dependency analysis. Key analysis examines attribute values to identify possible primary keys. Dependency analysis identifies relationships or patterns embedded within the dataset.
- ▶ **Cross-table profiling**
Cross-table profiling uses key analysis to identify stray data. Foreign key analysis identifies orphaned records or differences to examine relationships between column sets in different tables.
- ▶ **Data rule validation**
Data rule validation assesses datasets against established rules and standards to verify compliance with predefined rules.
- ▶ **Key integrity**
Key integrity ensures keys are present in data and identifies orphan keys.
- ▶ **Cardinality**
Cardinality checks relationships (such as one-to-one and one-to-many) between datasets.
- ▶ **Pattern and frequency distribution**
Pattern and frequency distribution ensures data fields are formatted correctly.

¹ <https://www.ibm.com/topics/data-profiling>

For example, you can use Python libraries to analyze the sample dataset for credit card fraud detection. You can load the data into a pandas DataFrame with pandas library (Figure 3-3).

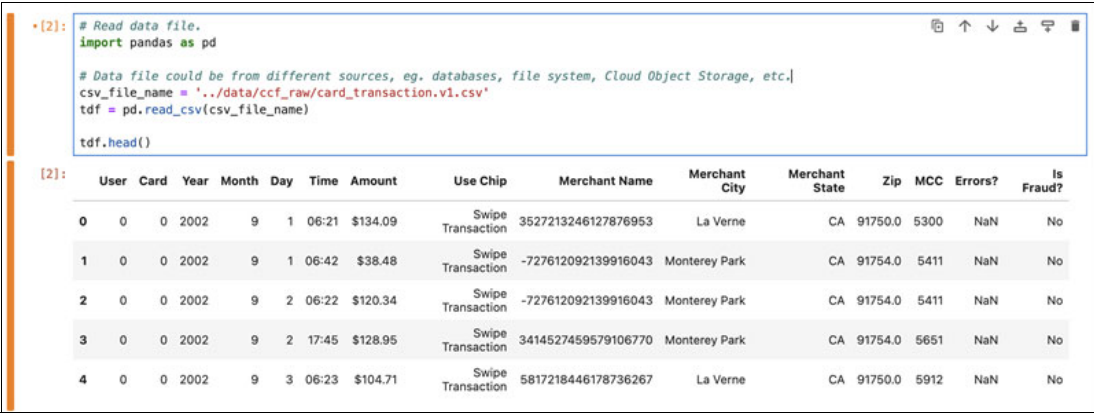


Figure 3-3 Profiling example - load data

You can use Pandas DataFrame to analyze the characteristics of the data, for example, mean, max, count, and standard deviation (Figure 3-4).

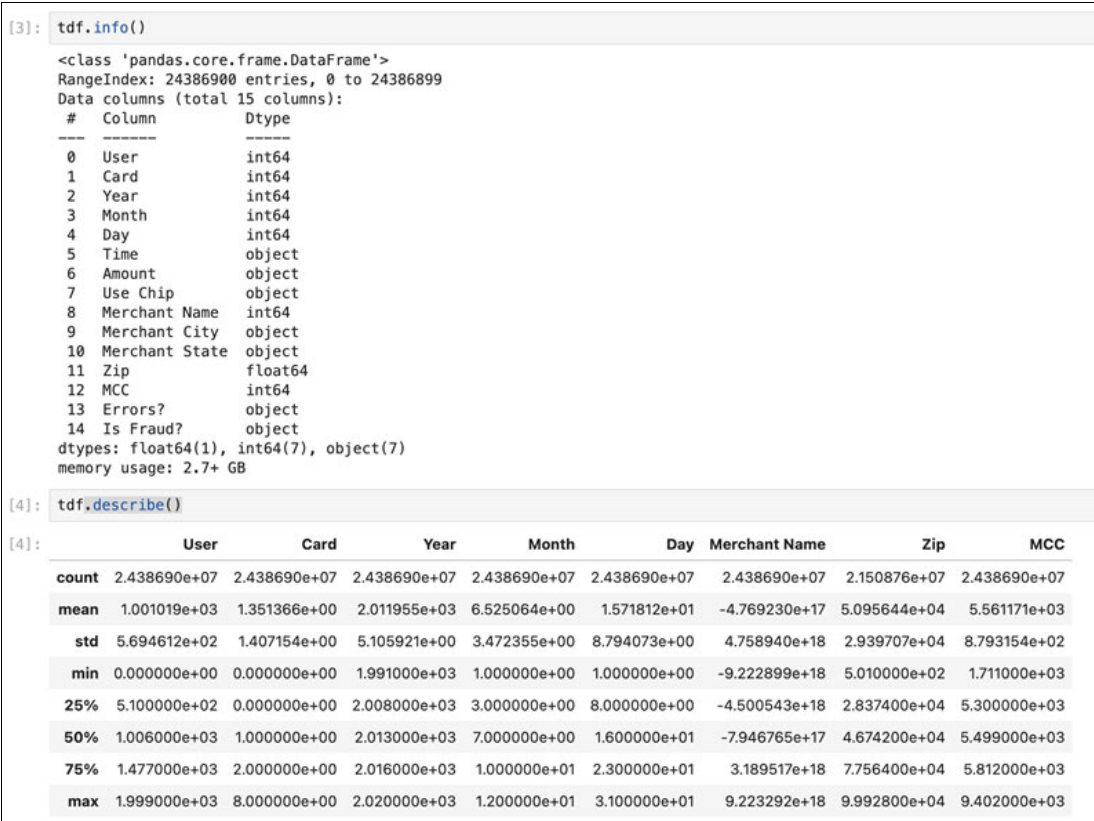


Figure 3-4 Profiling example – Pandas DataFrame

You can use Matplotlib or Seaborn library to visualize the data, or you can use Pandas profiling package, as shown in Example 3-1 (ydata-profiling), to automate and generate detailed reports, statistics, and visualizations.

Example 3-1 Using Pandas profiling package to automate and generate detailed reports, statistics, and visualizations

```
import ydata_profiling as pf
# sample command to generate profiling reports on screen
pf.ProfileReport(tdf)

# You can also save the profiling reports to files
profile = pf.ProfileReport(tdf, title="CCF Profiling Report")
profile.to_file('my_ccf_report.html') # Trigger the computation / alternative you
can use profile.to_json() for no file output
profile_dump('my_ccf_report') # Serialize in pickle to my_report.pp
```

As shown in Figure 3-5 on page 29, Pandas profiling generates various reports on the variables in the dataset and variable interactions and correlations. For the sample fraud dataset, when you load the data as-is, the *Amount* variable has the leading \$ (dollar sign) and is treated as text. Variable 'Use Chip' has 3 distinct categories, Swipe Transaction, Chip Transaction, and Online Transaction and has correlations with other variables. Also, some data is missing in the dataset.



Figure 3-5 Sample output from the Pandas profiling reports

3.3.2 Data cleansing

Data cleansing identifies and corrects inaccuracies, corruptions, misformatting, duplicate records, missing values, and inconsistencies. Data cleansing ensures data is accurate, complete, and reliable for AI.

Data cleansing includes the following tasks:

- ▶ Fix structural errors: Fix inconsistent naming conventions (N/A and Not Applicable), inconsistent capitalization (Male, male, MALE), typing errors, extra spaces, and nonprinting characters.
- ▶ Fix formatting issues: Date and time, phone number, or SSN formats might be inconsistent. Number formats might require corrections.
- ▶ Handle outliers: Datasets might contain outliers. Some outliers might be necessary. Validate outliers to determine whether they are mistakes or irrelevant to the analysis.
- ▶ Remove duplicates: Remove duplicate records from the dataset.
- ▶ Handle missing data: Handle missing data by removing observations with missing data or replacing them with other values.

As shown in the data profiling results in Figure 3-6, the Errors? column in the dataset contains missing values (NaN). These missing values indicate transactions without detected errors. Additional analysis of the missing values can help determine whether more processing is needed.

Figure 3-6 represents the distribution of unique values in the Errors? column of the sample credit-card-fraud dataset after initial profiling and during the cleansing phase.

```
# get unique values
tdf['Errors?'].unique()

array([nan, 'Technical Glitch,', 'Insufficient Balance,', 'Bad PIN,',
       'Bad PIN,Insufficient Balance,', 'Bad Expiration,',
       'Bad PIN,Technical Glitch,', 'Bad Card Number,', 'Bad CVV,',
       'Bad Zipcode,', 'Insufficient Balance,Technical Glitch,',
       'Bad Card Number,Insufficient Balance,',
       'Bad Card Number,Bad CVV,', 'Bad CVV,Insufficient Balance,',
       'Bad Card Number,Bad Expiration,', 'Bad Expiration,Bad CVV,',
       'Bad Expiration,Insufficient Balance,',
       'Bad Expiration,Technical Glitch,',
       'Bad Card Number,Bad Expiration,Technical Glitch,',
       'Bad CVV,Technical Glitch,', 'Bad Card Number,Technical Glitch,',
       'Bad Zipcode,Insufficient Balance,',
       'Bad Zipcode,Technical Glitch,',
       'Bad Card Number,Bad Expiration,Insufficient Balance,'],
      dtype=object)

# Counting NaN values in the 'Errors?' column
nan_count = tdf['Errors?'].isna().sum()

print(nan_count)

23998469
```

Figure 3-6 Data cleansing – unique values

The column captures various error scenarios. The missing transaction values should be without errors. It can be left as-is, or you can use the following sample statements to replace numerical values with median:

```
median_value = mydataset['column_name'].quantile(0.5)
mydataset['column_name'] = mydataset['column_name'].fillna(median_value)
```

Figure 3-7 illustrates median-based imputation of missing numeric values.

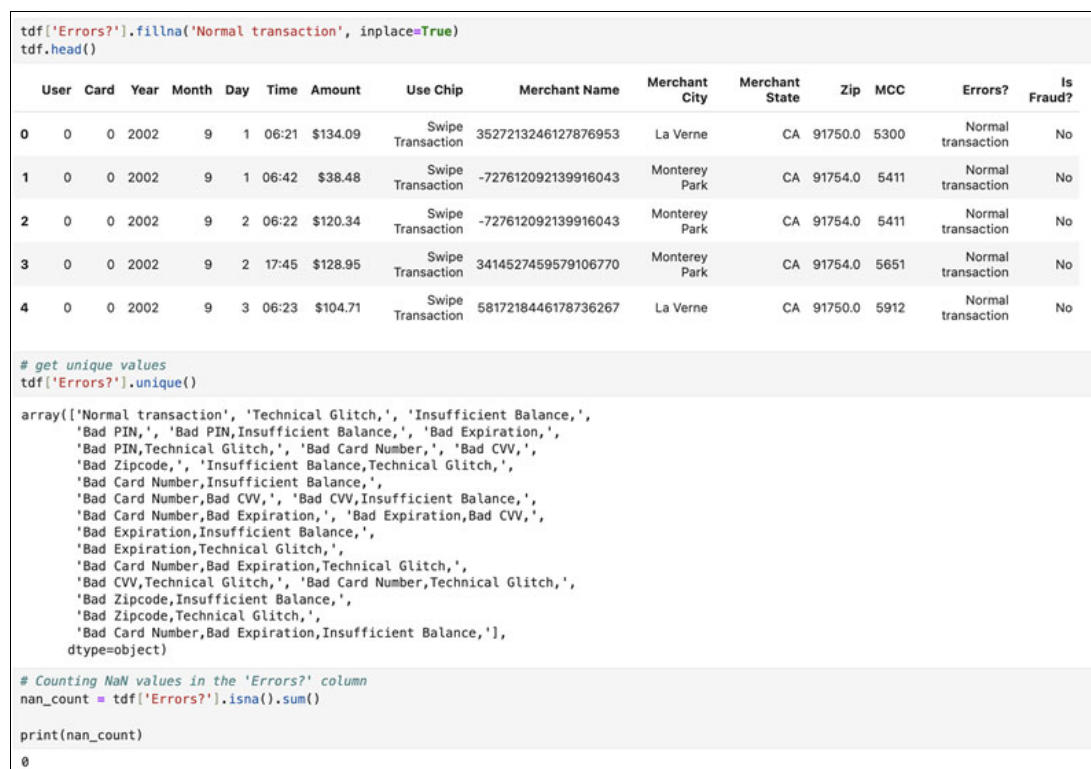


Figure 3-7 Data cleansing – replace text values

Pandas DataFrame also provides a method to drop duplicate records. Figure 3-8 on page 32 is an example to get unique user and card combinations.

You can preprocess the data with various Python libraries as demonstrated previously. Also, you can use DataFrameMapping module from the scikit-learn library to preprocess the data in a more clean and robust way, which is discussed in section 3.3.4, “Data transformation” on page 33.

# Get first of each User-Card combination		
first = tdf[['User','Card']].drop_duplicates()		
first		
	User	Card
0	0	0
5011	0	1
6214	0	2
10546	0	3
19937	0	4
...	...	...
24357917	1997	1
24373125	1997	2
24376357	1998	0
24382139	1999	0
24382226	1999	1
6139 rows x 2 columns		

Figure 3-8 Data cleansing – drop duplicates

3.3.3 Data reduction

Data reduction reduces dataset size, preserves important information, and improves computational efficiency.

Data reduction includes the following techniques:

► Dimensionality reduction

Dimensionality reduction reduces the number of dimensions in a dataset while retaining information. High-dimensional data poses challenges including model performance deterioration, data sparsity, and overfitting. Dimensionality reduction mitigates these problems by reducing model complexity and improving performance.

The following techniques reduce dimensionality:

- Feature selection: Selects a subset of the original features most relevant to the problem to maintain important features and reduce dimensions. Filtering and wrapping are two techniques for feature selection. The `sklearn.feature_selection` module classes perform feature selection and dimensionality reduction. For more information and examples, see [Feature selection](#).
- Feature extraction: Creates new features by combining or transforming original features to capture the essence of the original dataset in a lower-dimensional space. Principal component analysis (PCA) is a common technique for dimensionality reduction. Other methods include linear discriminant analysis (LDA), t-distributed stochastic neighbor embedding (t-SNE), and autoencoders. The `sklearn.feature_extraction` module extracts features. For more information and examples, see [Feature extraction](#).

- Data sampling

Data sampling selects a subset of the dataset instead of using the entire dataset. Data sampling techniques include simple random sampling, systematic sampling, stratified sampling, and cluster sampling.

- Data discretization

Data discretization converts continuous data into discrete data by partitioning the range of possible values into contiguous intervals. Methods include binning, histogram analysis, cluster analysis, decision tree analysis, and correlation analysis. Data reduction reduces the size of a dataset and preserves the most important information. It reduces the volume of data to improve computational efficiency.

Figure 3-9 is an example of random data sampling using `pandas.DataFrame.sample`.

```
tdf2 = tdf.sample(n=10, random_state=1)
tdf2
```

	User	Card	Year	Month	Day	Time	Amount	Use Chip	Merchant Name	Merchant City	Merchant State	Zip	MCC	Errors?	Is Fraud?
4866429	413	1	2011	3	15	07:50	\$50.43	Swipe Transaction	-3220758452254689706	Spokane	WA	99208.0	5311	Normal transaction	No
5124282	433	1	2015	9	20	05:30	\$53.78	Online Transaction	-2088492411650162548	ONLINE	NaN	NaN	4784	Normal transaction	No
17887935	1442	3	2020	2	10	11:41	\$30.98	Chip Transaction	-4113349227963201766	Lakeland	FL	33813.0	5411	Normal transaction	No
7224963	610	2	2017	10	13	18:32	\$10.32	Swipe Transaction	6913268435708117971	Albuquerque	NM	87121.0	5912	Normal transaction	No
19812059	1606	1	2009	5	18	10:44	\$130.45	Swipe Transaction	4937803722023861373	Stockton	CA	95210.0	5310	Normal transaction	No
11960960	987	0	2019	1	6	06:08	\$65.42	Chip Transaction	6181510752141427450	Avoca	MI	48006.0	7538	Normal transaction	No
17818383	1437	1	2013	12	27	14:45	\$104.29	Swipe Transaction	-5467922351692495955	Brooklyn	NY	11222.0	5912	Normal transaction	No
12724868	1043	1	2018	8	18	12:09	\$44.24	Online Transaction	7035602569409149834	ONLINE	NaN	NaN	5311	Normal transaction	No
5643279	484	1	2014	4	10	08:14	\$23.36	Online Transaction	-6160036380778658394	ONLINE	NaN	NaN	4121	Normal transaction	No
13196474	1081	4	2017	9	16	11:57	\$17.17	Chip Transaction	4751695835751691036	Nashville	TN	37211.0	5814	Normal transaction	No

Figure 3-9 Data reduction – random sampling

3.3.4 Data transformation

Data transformation converts the format, structure, or values of data to support analysis and decision-making.

Different data types might require different transformations. The following are data transformation techniques:

- Format conversion
- Data normalization and standardization
- Feature construction and value mapping
- Data aggregation
- Data splitting
- Data integration

Data transformation can be combined with data cleansing and data reduction techniques. For example, a data transformation script might handle missing data to construct new features.

The credit card fraud detection model uses `sklearn_pandas.DataFrameMapper` to transform column values. In Figure 3-10 on page 34, the Amount column is transformed as follows:

FunctionTransformer applies the amtEncoder method. The amtEncoder method removes the first character \$, converts the type to float, ensures all values are at least 1, takes the natural logarithm of the values, and returns the result as a DataFrame. The MinMaxScaler class scales each returning value to the default range of 0–1.

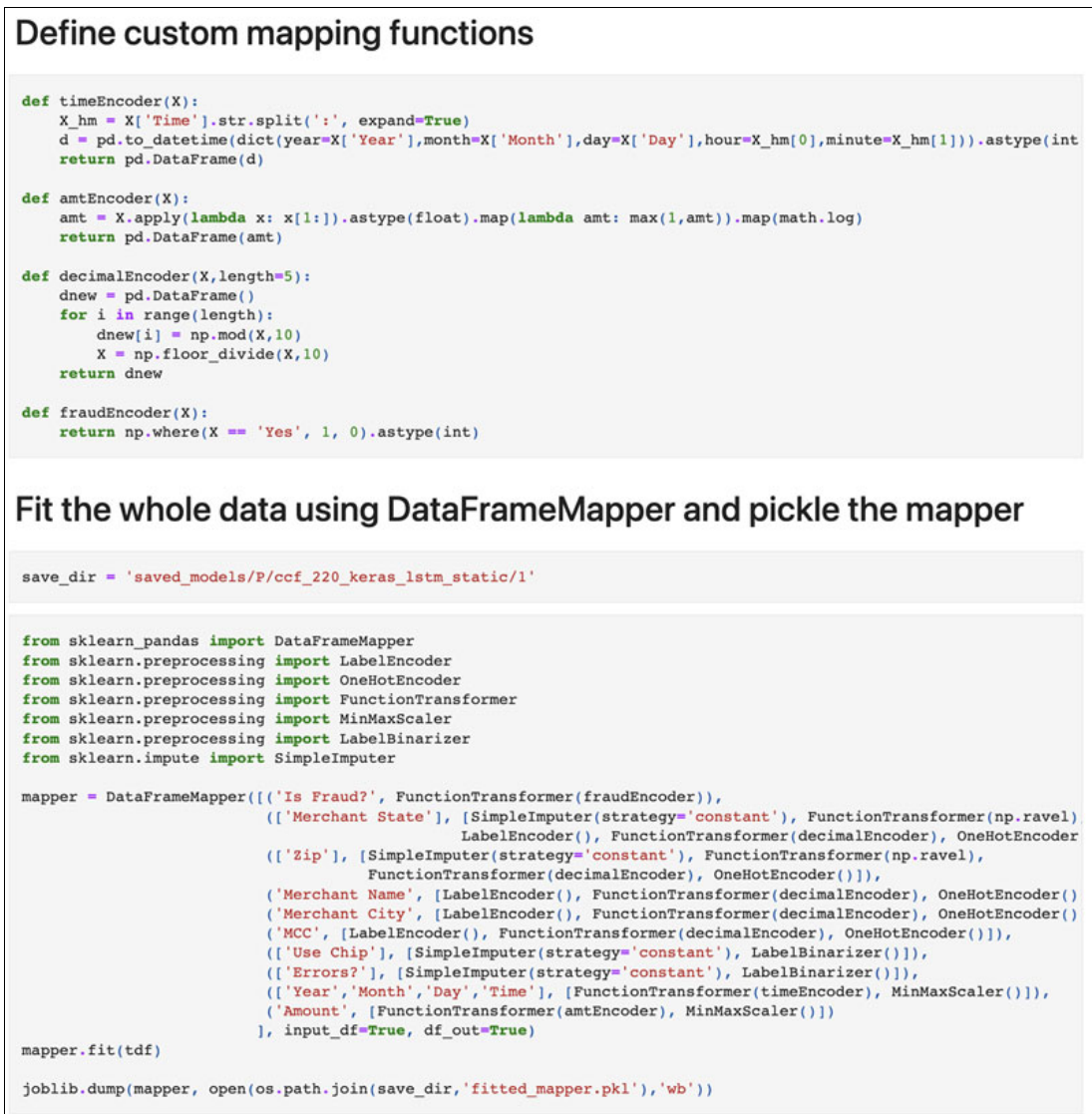


Figure 3-10 Data transformation – custom mapping

In Figure 3-11, each unique value of the column ‘Error?’ is converted to a new binary column, with 0 or 1 as the value.

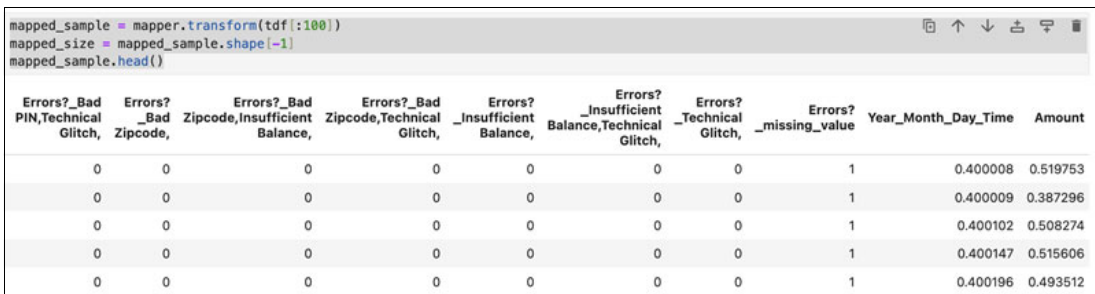


Figure 3-11 Data transformation – feature construction

3.3.5 Data enrichment

Data enrichment is the process of enhancing raw data by adding information and context to improve the relevance, value, and informativeness of the data for analysis or ML tasks.

Data can reside in different data sources and in different formats. For example, transaction data can be stored in Db2 on z/OS, whereas the information about each client can be stored on VSAM. Also, some other information might come from 3rd-party or public sources. Integrate relevant information from multiple sources to generate a more comprehensive dataset, which might improve the value and quality of the data and provide added context and drive more insights. Section 3.4 describes how data from different sources can be integrated efficiently and securely.

Data enrichment includes different ways to enrich and augment the data. Data from different sources can be integrated together to generate a more comprehensive dataset. New data features can be created based on the information from other sources. External information might suggest new ways to process missing values. The previously mentioned data profiling step might help you analyze the data from different sources and decide on the data enrichment strategy.

IBM provides tools for data enrichment. Watson Knowledge Catalog and Watson Discovery can help users with data enrichment tasks.

3.3.6 Data validation

Data validation verifies that preprocessed data is accurate, consistent, complete, and suitable for the intended modeling task before analysis or model building. Data quality directly impacts AI model quality and efficiency.

Data validation addresses the following dimensions:

- **Completeness**

Completeness represents the amount of usable or complete data. A high percentage of missing values might lead to biased or misleading analysis if the data is not representative of a typical data sample.

- **Uniqueness**

Uniqueness accounts for the amount of duplicate data in a dataset. For example, each customer should have a unique customer ID.

- **Validity**

Validity measures how much data matches the required format and business rules. Formatting includes metadata such as valid data types, ranges, and patterns.

- **Timeliness**

Timeliness refers to data readiness within an expected time frame. For example, customers expect to receive an order number immediately after purchase, requiring real-time data generation.

- **Accuracy**

Accuracy refers to the correctness of data values based on the agreed-upon primary data source. Because multiple sources can report on the same metric, designate a primary data source. Other data sources can confirm the accuracy of the primary source. For example, tools can verify that each data source trends in the same direction to increase confidence in data accuracy.

- Consistency

Consistency evaluates data records from different datasets. Multiple sources can report on a single metric. Using different sources to check for consistent data trends and behavior enables organizations to trust actionable insights. This logic also applies to relationships between data. For example, the number of employees in a department should not exceed the total number of employees in a company.

- Fitness for purpose

Fitness for purpose ensures that the data asset meets a business need. This dimension can be difficult to evaluate, particularly with new or emerging datasets.

3.4 Data integration patterns

Data is a critical asset. The IBM Z platform conducts core business transactions across industries, storing large amounts of organizational data. Using transactional data to generate insights and support business opportunities provides competitive advantages.

Multiple factors affect data integration pattern selection. Identify business problems, understand use case requirements, and evaluate different options.

Consider the following factors before choosing a data integration pattern:

- What data is needed?

- What are the data sources (databases or file systems)?
- What are the data schemas and formats?
- What integration is needed among data sources?
- What are the purposes of data processing (feeding applications, training AI models, generating data products)?

- Where is the data and where does it need to go?

Does the data need to be processed and consumed on-premises, in the cloud, or in a hybrid configuration?

- When is the data needed?

- Are there real-time requirements?
- Is the data needed for in-transaction inference in real-time or for batch processing with delay?
- Is near real-time acceptable?
If yes, with how much delay (seconds, minutes, 1 hour)?
- How often does the data need to be refreshed?

- How is the data made available?

- Can the data be accessed in place using virtualization, or does a data copy need to be maintained in cache or off-premises?
- What is the granularity and volume (transaction level or large volume data dump)?

- Why does data need special handling?

- Is there PII data?
- Are there regulations to meet?
- Can the data leave the premises or a certain region?

- Are there other requirements for latency, performance, security, compliance, or auditing?

Evaluate different options and adopt the appropriate data integration pattern or combination of patterns based on business requirements.

3.4.1 Data exposure pattern: Enabling modern access to IBM Z data

Mainframes process 12.6 billion transactions per day and 29 billion ATM transactions annually and are used for \$7.7 trillion in annual credit card payments.² Mainframes often handle mission-critical business operations and require the highest level of security. Mission-critical System of Record (SOR) data is stored on mainframes and is difficult to access. Companies copy data from various sources into central data stores for business cases such as operational transactions and analytics. Establishing and maintaining data replication pipelines is expensive and time-consuming and creates data quality and data latency challenges for consuming applications. Accessing data in place can accelerate transformation.

Modern access to IBM Z data through SQL-based query and REST APIs enables enterprises to use data more effectively for on-premises applications or cloud-native applications. This modern data access simplifies application development without disrupting data management and recovery processes on IBM Z.

IBM Z supports modern access to real-time transactional data in IBM Db2, IMS, and other data sources. Db2 and IMS can be accessed through SQL and REST API by using [IBM z/OS Connect EE](#). You can access other data sources through SQL or REST API by using IBM Data Virtualization Manager for z/OS and z/OS Connect EE. Data Virtualization Manager for z/OS extends SQL access to data sources other than relational databases (Figure 3-12).

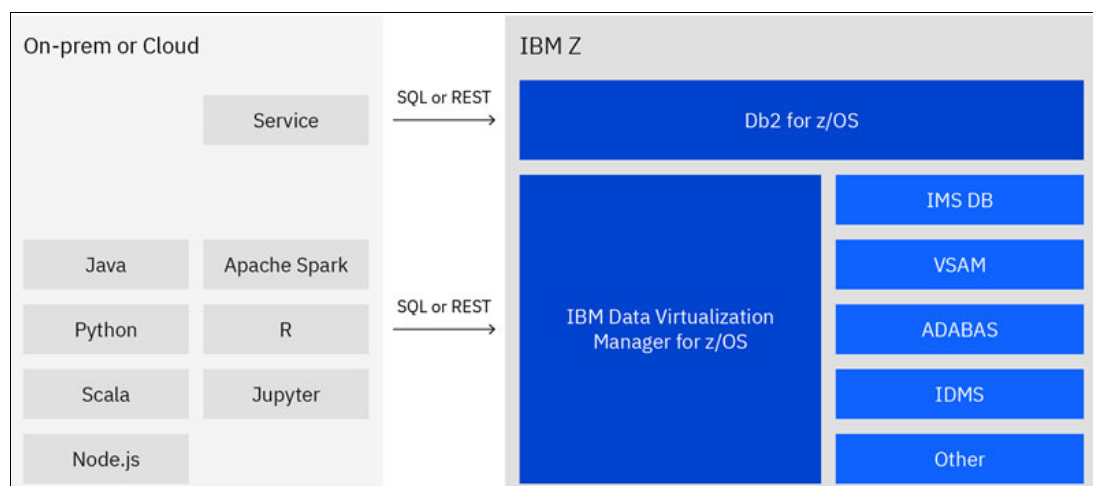


Figure 3-12 Enable modern access to IBM Z data

Java Database Connectivity (JDBC) support enables cloud applications that consume information from Db2 and IMS systems to use SQL to detect the underlying data format and contexts from systems of record.

SQL can access data stored in Db2. Db2 for z/OS supports native RESTful services to expose SQL and stored procedures as REST APIs when combined with z/OS Connect EE. Start Db2 native RESTful services from z/OS Connect EE using the z/OS Connect EE REST Client Service Provider.

² The modern mainframe: A banking platform from the future - <https://www.ibm.com/downloads/cas/8MG0LOB7>

z/OS Connect EE provides access to IBM Z data stored in Db2, IMS, or other data sources through REST API. Applications can use data stored on z/OS. z/OS Connect EE is a common interface for cloud-native applications (Figure 3-13).

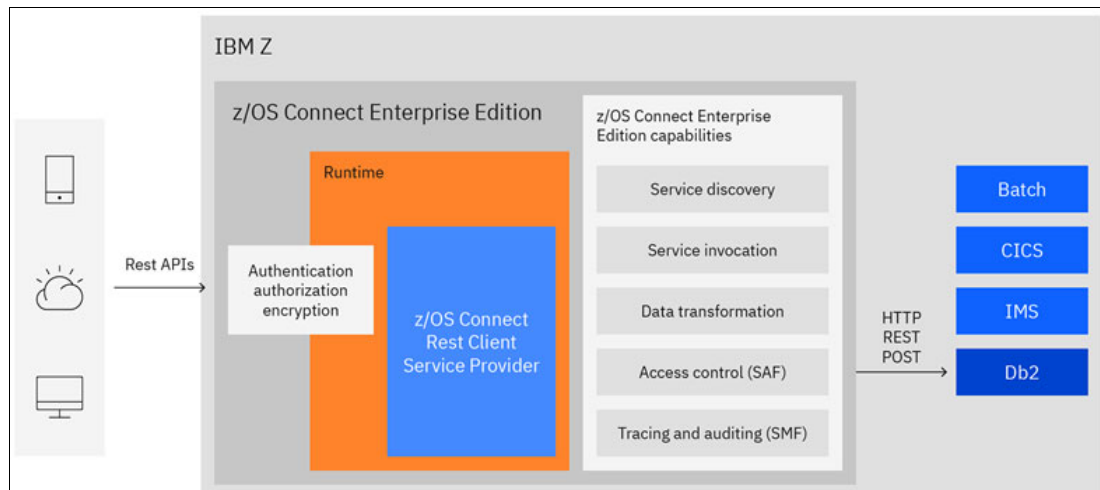


Figure 3-13 Components to support modern access to IBM Z data

Accessing IBM Z data in-place provides several critical business benefits:

- ▶ Reduces the risk of data integrity
- ▶ Reduces the cost that is involved in data movement
- ▶ Increases data quality
- ▶ Preserves the existing data management and recovery processes
- ▶ Allows cloud applications to access the data at its underlying format and context
- ▶ Supports popular access methods, such as SQL and REST APIs
- ▶ Benefits from higher performance by accessing data that is on IBM Z
- ▶ Avoids the introduction of security intrusion points related to moving the data outside the platform

A modern approach to accessing IBM Z data simplifies application delivery. Non-relational IBM Z transactional data and off-platform data can be accessed through traditional APIs and, when combined with IBM z/OS Connect EE, through RESTful APIs. This approach reduces the cost and delay of moving data to non-IBM Z platforms.

3.4.2 Data virtualization pattern: Virtualizing IBM Z data

Data is essential for enterprise digital transformation. New services require simplified access to IBM Z data for business operations that involve read and update operations through APIs. Frequently, IBM Z data must be combined with other data sources.

However, organizations often face challenges when using their data because it originates from diverse sources and varies in type, structure, environment, and platform. These challenges also apply to data stored on IBM Z, which typically hosts most of the operational data in large organizations. A common perception is that IBM Z data is difficult to access and transform.

Moving all data into a single data store, such as an operational data store or data lake, might create additional challenges. Data copy processes can result in data latency, poor data quality, increased cost, risks, and security challenges.

Data virtualization is a data integration method that combines disparate data sources to create virtualized, integrated views without copying and replicating data. Data virtualization provides a data services layer that abstracts the underlying physical implementation of individual data sources. Data virtualization supports compliance requirements because fewer copies of data exist, and data is fresh and accurate from the source.

IBM Z data virtualization enables modern access to IBM Z data across data sources. IBM Z data patterns refer to the configurations that are used to define how data is retrieved and displayed. This architectural design pattern supports access to near real-time transactional data in IBM Db2, IMS, and other data sources. Db2 and IMS can be accessed through SQL, Java Database Connectivity (JDBC), and REST API by using IBM z/OS Connect EE. Data Virtualization Manager for IBM z/OS provides SQL access to all IBM Z data sources. By using z/OS Connect EE, you can expose IBM Z data and services through REST APIs.

The primary use case for Data Virtualization Manager for IBM z/OS is mapping traditional IBM Z data sources (such as VSAM, IMS, or Adabas) into relational views for access through SQL or API. The main use case for data virtualization in IBM Cloud® Pak for Data (CP4D) is to gain a single view of disparate data without data movement and to manage data with less complexity and risk of error. For more information, see [IBM Data Virtualization](#).

The foundation for accessing data across disparate data sources is the IBM Watson Knowledge Catalog in IBM Cloud Pak® for Data. Maintaining metadata across different data sources is more feasible and less costly than moving terabytes of changing data. Watson Knowledge Catalog is a data catalog tool that enables self-service discovery of data structures and models. You can access, curate, categorize, and share data, knowledge assets, and their relationships, backed by active metadata and policy management. The cloud-based enterprise metadata repository also activates information for AI, ML, and DL. In Watson Knowledge Catalog, you can discover, govern, and catalog the metadata of IBM Z data that is stored in Data Virtualization Manager for IBM z/OS and Db2 for IBM z/OS and other data sources (Figure 3-14 on page 40).

IBM data virtualization uses a peer-to-peer computational mesh, which offers advantages over traditional federation architecture. The data virtualization engine delivers query results from many data sources through advanced parallel processing and optimization. Parallel compute models provide query performance up to 430% faster than federation against 100 TB datasets.³ IBM data virtualization scales complex queries with joins and aggregates across dozens of live systems. IBM Z data can be accessed through SQL.

Data virtualization can simplify the development of consuming applications, including AI-infused business applications, and applications can access current and accurate data at its source.

³ Simplify data access and reduce operational costs while advancing your business insights - <https://www.ibm.com/downloads/cas/M76BGEXW>

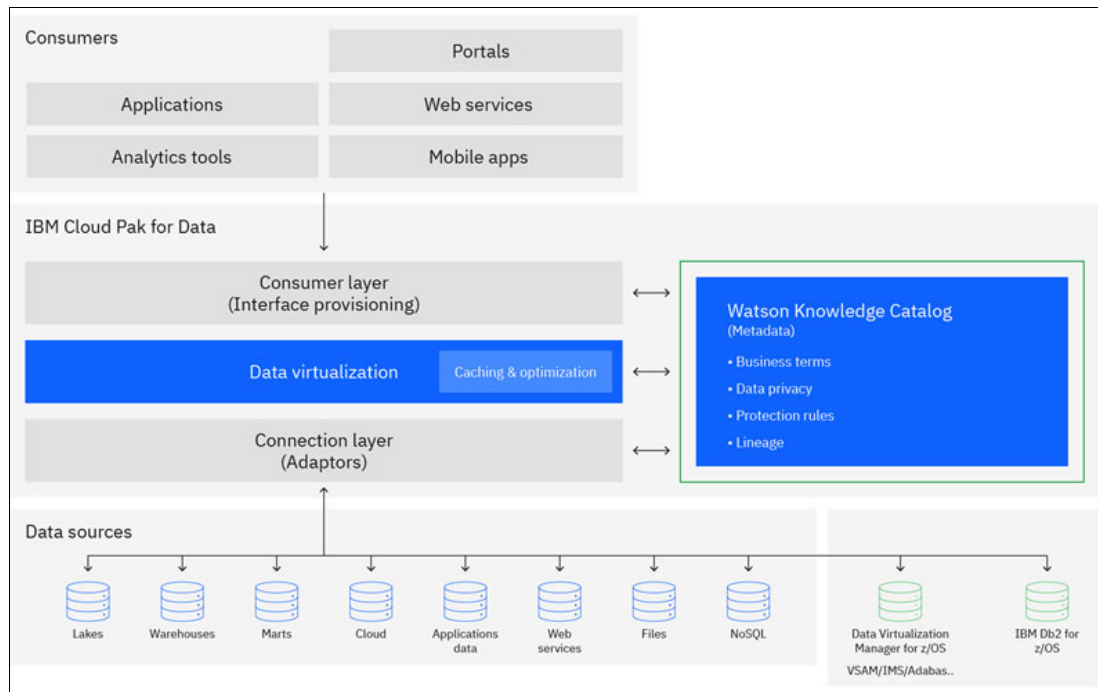


Figure 3-14 Virtualize IBM Z Data

Accessing IBM Z data in place provides the following business benefits:

- ▶ Reduces the risk of data integrity impact
- ▶ Reduces the cost of data movement
- ▶ Increases data quality
- ▶ Preserves existing data management and recovery processes
- ▶ Enables cloud applications to access data in its underlying format and context

Data virtualization in IBM Cloud Pak for Data supports rapid ML model development and deployment and AI integration into business applications.

With a centralized view of data, including IBM Z data, within Watson Knowledge Catalog, you can build, test, and train ML models on the platform of choice. Deploy AI models to Machine Learning for IBM z/OS to address complex information needs within business services that run on z/OS (Figure 3-15 on page 41).

By using the Virtualize IBM Z data pattern, you can access data across IBM Z and other data sources, including joining data, without copying or replicating it. This approach delivers more current and accurate data through in-place access for consuming applications, including analytics.

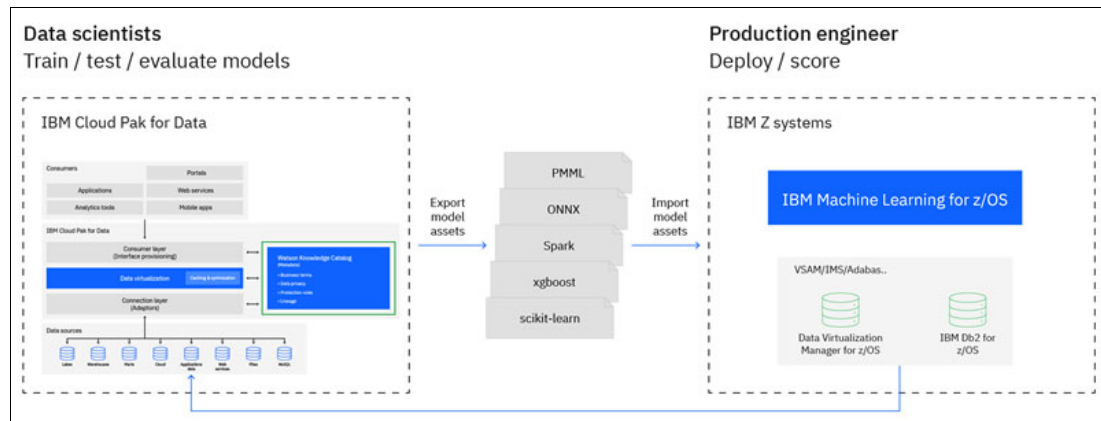


Figure 3-15 AI model building with data virtualization

For more information, see the Redbooks publication, *Mainframe Application Modernization Patterns for Hybrid Cloud*, SG24-8532.

3.4.3 Real-time information sharing pattern: cache IBM Z data

New digital applications drive growth in data access. Usually, these applications run read-only queries of current data but do not generate revenue. Examples include mobile banking, retail online browsing, and insurance open enrollment.

These applications have the following characteristics:

- ▶ High, unpredictable volume
- ▶ Massive, sharp spikes in activity
- ▶ Updates possible but not propagated back to the source (read-only)
- ▶ Expensive to maintain
- ▶ Complex to implement
- ▶ Compromised data currency
- ▶ Prone to instability

Organizations used several methods to extract data for use on other platforms to address these challenges. Often, organizations extracted data off-platform for analytic applications and query-type access because website developers accessing the data were sometimes unfamiliar with the mainframe. Organizations also had concerns that read-only activity might conflict with operational applications.

Traditional incremental copy and ETL approaches are unpredictable and can result in data latency. General-purpose incremental copy and ETL technologies limit efficiency and performance improvement opportunities. Data extraction and incremental copy from IBM Db2 for z/OS can use resources, increase software-related costs, and compete with operational processing resources.

Often, end users use these applications, so data must be current and moments behind a transactional system. This requirement drives complex application logic to ensure data currency. Organizations use customized code to ensure consistency in addition to processes that refresh the read-only data store. This adds complexity, instability, and cost.

Caching on IBM Z improves application response time by storing copies of IBM Z data. IBM Z offers several products that implement variations of this pattern:

- ▶ IBM Db2 Analytics Accelerator

- ▶ IBM Z Digital Integration Hub
- ▶ IBM Db2 for z/OS Data Gate
- ▶ IBM Z Table Accelerator
- ▶ IBM Data Virtualization for z/OS with Cache Option

Some products include a synchronization component to maintain the cache.

The cache has two main differentiators:

1. Pull versus push maintenance

In push maintenance, the cache is kept in sync with the source regardless of actual use. Pull maintenance uses lazy update and invalidation of cached values based on access.

2. Cache data structure

Cache data structures are optimized for the consumption pattern (such as columnar or in memory). The cache might contain derived or precomputed results.

Optimizing application performance by accessing cached IBM Z data can provide several critical business benefits:

- ▶ Achieve service level agreements (SLAs)
- ▶ Improve performance in scenarios where data is read repetitively and at high frequencies
- ▶ Provide a good compromise between cost and complexity
- ▶ Improve efficiency by avoiding accessing the database or other data sources every time for the same request
- ▶ Achieve integrated, lightweight data synchronization
- ▶ Provide excellent performance and resiliency
- ▶ Reduce latency with better data currency

Caching improves application latency. A highly available and resilient cache enables applications to scale. The IBM Z data caching pattern offloads processing from the application's main logic to the caching layer, which releases compute resources to process additional work. Read-intensive applications can benefit by implementing a caching approach (Figure 3-16).

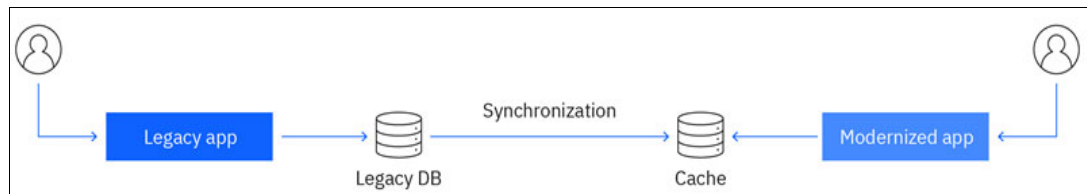


Figure 3-16 Cache IBM Z data

For more information, see *Mainframe Application Modernization Patterns for Hybrid Cloud*, SG24-8532.

3.4.4 Data transformation pattern: Transforming IBM Z data

Software processes transform System of Record (SOR) data to create new datasets. This is a specialization of copy-and-access use cases, which include extract, transform, and load (ETL) processes, software replication processes, and virtualization and federation processes. These processes include transformation capabilities, and light transformations are common during copy-and-access use cases.

This pattern involves heavy, set-based transformations that create datasets from external feeds, derived transformations, and source SOR datasets (such as aggregation, consolidation, and summation). Currency requirements determine whether to use real-time mechanisms (such as virtualization), near-real-time mechanisms (such as software replication), or periodic batch mechanisms, such as ETL or extract, load, and transform (ELT).

A combination of mechanisms might be needed if set-based transformations are required with real-time or near-real-time currency. This pattern applies to SOR data on any platform. Its use with SOR data on IBM z/OS is valuable because many large enterprises use z/OS as an SOR.

In Figure 3-17, the Data Adapter box indicates transformation processes. These transformations can span processes and time.

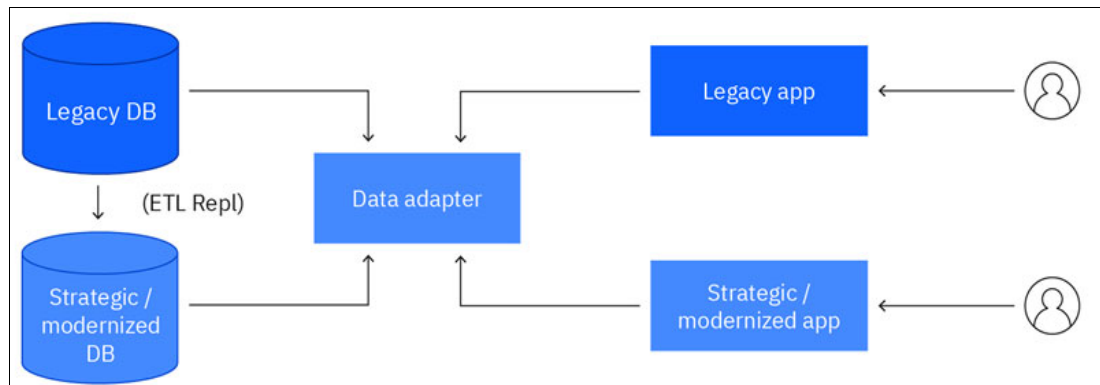


Figure 3-17 Transform IBM Z data

On z/OS, it is typical to have solutions that maintain logically related data across different stacks, such as IBM Db2, IMS, VSAM, or sequential files. For example, transaction data can be stored in Db2, and client information can be stored on VSAM. Viewing this data through federated queries or from an aggregated copy provides value. Extending the aggregation to derive data by using summations and transformations and adding sources through distributed databases and external feeds increases the value of the original z/OS SOR data without disrupting original workloads.

This pattern creates schemas over data. One common use case converts normalized data models from the SOR to a de-normalized data model. De-normalized data models are used in data warehouse solutions and dimensional data marts optimized for statistical and analytical processing. Another common use case creates user-oriented schemas from product-oriented SOR data. This enables consumption by people less familiar with the internal view of the technology or products that underpin the data.

IBM DataStage® provides data transformation capabilities. IBM DataStage is available on-premises, on IBM Cloud Pak for Data, and in cloud implementations.

Creating datasets with additional attributes derived from production data enables extensibility with minimal disruption. It also reduces the need to maintain multiple synchronized copies of the same data. The transformed-data approach provides the following advantages:

- ▶ No changes are required to the original SOR datasets. All these methods apply transformations downstream from the original data.
- ▶ These methods all provide remote access to SOR data in addition to the transformation of that SOR data.

- ▶ Downstream datasets can be created or modified without impacting the source SOR data that they are derived from.
- ▶ New applications can be created outside of the context of the original SOR data.
- ▶ The impact to core SORs from downstream workload requirements is mitigated.

By using transform IBM Z data pattern, you can incrementally build new modernized SOR data stores by accessing IBM Z data traffic through a data adapter to transform to the modernized data format. For more information about the transform IBM Z data pattern, see *Mainframe Application Modernization Patterns for Hybrid Cloud*, SG24-8532.

3.4.5 Data synchronization pattern: Replicating IBM Z data

Software processes replicate System of Record (SOR) data to create alternative, typically remote, copies. This is a specialization of copy use cases, which include extract, transform, and load (ETL) processes, unload and load utilities, and hardware (disk) replication. Software replication provides both near real-time replication with recovery point objectives (RPOs) near zero and continuous availability with recovery time objectives (RTOs) near zero. This pattern applies to SOR data on any platform. Its use with SOR data on IBM z/OS is valuable because many large enterprises use z/OS as an SOR.

Business drivers for real-time SOR data replication include larger solutions. Most customers have internally defined infrastructure support and a subset of deployed packaged solutions.

The following list includes common uses cases:

- ▶ Mission-critical, workload-level continuous availability at a distance to support planned and unplanned outages
- ▶ Real-time data warehouse for accelerated inquiry processing, offloading the SOR
- ▶ Advanced analytics processing on real-time data by using modern analytics platforms (such as Hadoop and Apache Spark)
- ▶ Application modernization use cases in the cloud and on-premises
- ▶ Distributed query use cases (such as CQRS), offloading inquiry from the SOR
- ▶ Aggregation of data from multiple sources into an operational data store (ODS) or data lake
- ▶ Offload of the SOR from directly supporting multiple consumers, evolving consumers, or both by replicating into a data lake

You can implement these drivers by using either homogeneous replication (like models) or heterogeneous replication (unlike models). The diagram in Figure 3-18 on page 45 describes a simplified depiction of a replication setup from a source DBMS to either a target DBMS, a publish/subscribe such as Kafka, or a file system.

Continuous availability is a homogeneous replication use case. When the SOR is IBM Db2® for z/OS, IMS, or VSAM, the target system is Db2 for z/OS, IMS, or VSAM. IBM Z GDPS® Continuous Availability augments replication with a centralized control plane, intelligent routing, automation, and monitoring. These higher-level constructs can be developed over replication using alternative methods.

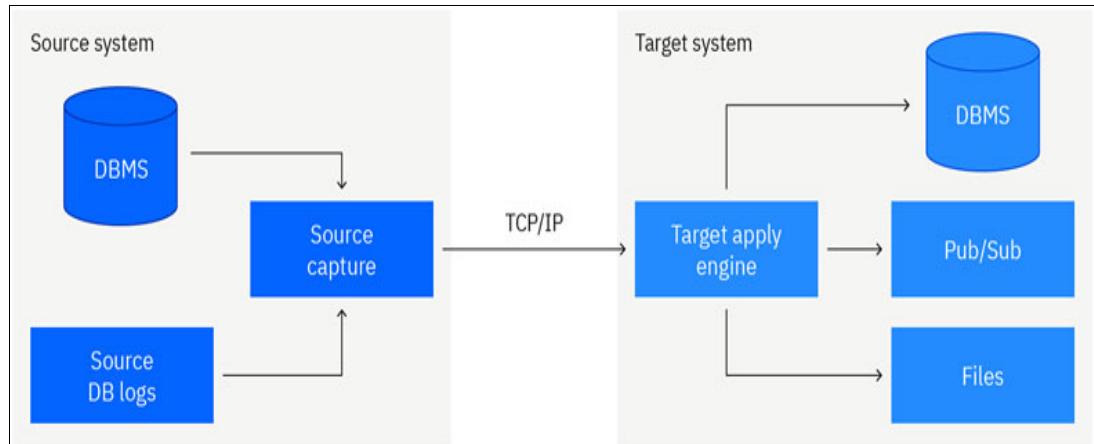


Figure 3-18 Replicate IBM Z data

The other business drivers are heterogeneous. Target systems and DBMS or file systems for these drivers can be deployed in Linux on IBM Z, z/OS Container Extensions, or on IBM Z as an alternative to running components on distributed systems. Running on IBM Z provides high availability characteristics, reliability, security, and management. Performance is better because fewer availability zones and network devices need to be traversed.

IBM InfoSphere® Data Replication and IBM Data Replication products support replication. This family includes QRep, CDC, and Classic products.

For more information, see [InfoSphere Data Replication 11.4.0](#).

The use of software replication to create one or more copies of SOR data provides several key advantages:

- ▶ Near real-time synchronization: Average latency for online (transactional) processes is measured in seconds or subseconds. Average latencies for batch processes are higher but measured in minutes. These latencies meet many companies' SLAs for RPO. Other techniques use periodic pulls of all data or replication to auxiliary storage devices that must be varied online for use.
- ▶ Low RTO: Software replication captures changes to source data as they occur and replicates those changes to target agents that replay those changes through the DBMS or file system stack. This enables the RPO to become the RTO, achieving RTOs of seconds or subseconds from any change.
- ▶ Workload-level granularity: Software replication runs at any granularity and focuses on the sets of data for an application or workload. Custom configurations and processing can be implemented at the workload level. Other solutions (such as disk replication) are site-level and constrained by proper usage at the disk volume level.
- ▶ Transactional consistency: Software replication processes data at transaction boundaries and maintains target copies that are transactionally consistent with the source. This is not possible with alternatives such as ETL or disk replication.
- ▶ Cost optimization: Offloading queries and analytics processing to target systems reduces strain on the SOR and achieves CPU-intensive goals.
- ▶ High-volume processing: Large volumes can be handled in terms of processing throughput with low latency maintained.
- ▶ Spiky workload mitigation: Impact of spiky and unpredictable workloads to core SORs is reduced.

When you implement software replication from SOR data, consider the following factors:

- Supplemental logging

Real-time replication requires supplemental logging. Replication products read change records from DBMS recovery logs. Recovery logs are designed for DBMS recovery processes and contain content tailored for those purposes. Replication requires additional attributes and transactional semantics including commits, rollbacks, operations identified by transaction ID, before images for updates, open and close events, and load utility events.

Augmenting recovery logs to add these attributes increases the volume of changed data to the logs, affects log switch procedures, and impacts source application response time. Some impact to source application response time can be mitigated by using buffering and by offloading designs. Log retention periods are increased for replication use cases to allow for earlier restarts. Adding supplemental logging might require more capacity and affects log switch times.

- CPU intensity

Real-time replication can be CPU intensive. Reading logs, merging change records, transactional buffering, refreshes (select *), transmission, and parallel replay affect CPU. Software replication is compared unfavorably to disk replication because disk replication is done at the control unit and off-platform. Adding replication might require more capacity to achieve performance that is measured by latency.

- Tight synchronization

Real-time replication provides tight synchronization between source and target data. The target copy is an extension of the source copy and must be considered for any source-side change management, batch processing, and utility processing (such as reorgs). Source procedures must be extended to consider the effect on the copy.

By using replicate IBM Z data pattern, you can replicate data in real time by capturing change log activity to drive changes in the target. Enable newer applications to access a broader range of data stores and access methods through replication. For more information, see:

[Data replication on z/OS.](#)



Model building for IBM Z

Model building is a critical step within the AI lifecycle. The decisions that you make for model training can impact the model's performance when it is put into production. The previously discussed art of data engineering integrates closely with model training. The model is only as good as the quality of the data that is used to train it.

- ▶ 4.1, “Model training frameworks” on page 48
- ▶ 4.2, “Model formats” on page 50
- ▶ 4.3, “Training strategy” on page 50
- ▶ 4.4, “Best practices for model training” on page 58
- ▶ 4.5, “Model building use cases” on page 61

4.1 Model training frameworks

When building an AI model, it is helpful to use a training framework. A training framework provides the tools necessary to build, train, and validate models in an efficient way. These efficiencies give you the ability to build models in an agile fashion.

4.1.1 Model training frameworks and technologies on IBM Z

Various common model training frameworks are available today. Many open source frameworks and IBM-provided solutions are available that can reduce the complexity for a developer or data scientist when navigating through the full AI lifecycle.

Several IBM provided solutions reduce complexity and enable the full AI lifecycle.

Machine Learning for IBM z/OS

Machine Learning for IBM z/OS (MLz) provides the Jupyter Notebook functionality to train models directly within its user interface and has the service capability to run, manage, and automate the model training process on z/OS Spark. You can build out custom training pipelines that interface with your data and save them directly within your environment.

For more information, see [Developing a model in the integrated Notebook Editor](#) and [Submitting application to Spark for model training](#).

IBM Cloud Pak for Data

IBM Cloud Pak for Data (CP4D) is a modular set of integrated software components for data analysis, organization, and management. The feature set of CP4D includes the ability to train AI models. You can use Watson Studio and Machine Learning to build custom training pipelines with tools like Jupyter Notebook within CP4D. Also, you can use the AutoAI feature to rapidly train and tune potential models. CP4D, including the AutoAI functionality, is available as a software license, as a service, and on-premises with IBM Z.

IBM Cloud Pak for Data on IBM Z uses hardware acceleration. It uses the IBM Z Integrated Accelerator for AI for Snap ML and TensorFlow models.

Python AI Toolkit for IBM z/OS

The majority of common AI, machine learning, and analytics interfaces are open source, and package security requires currency. Python AI Toolkit for IBM z/OS consists of up-to-date validated and secure open source packages optimized for z/OS that is delivered through a secure channel in a self-service method. Python AI Toolkit for IBM z/OS provides several different training frameworks and technologies. Some available packages include scikit-learn, XGBoost, Jupyter Notebook, numpy, pandas, matplotlib, and many more.

For more information, see [Python AI Toolkit for IBM z/OS 1.1.0](#).

IBM z/OS Platform for Apache Spark

Apache Spark is an open source, distributed processing system used for big data workloads. Spark is a part of the Apache suite, which is one of the largest open source foundations. Spark supports ML workloads by automating data processing for real-time and batch use cases.

For more information, see [IBM Z Platform for Apache Spark 1.1](#).

SnapML

Classical ML model types, such as gradient boosting machine, random forest, and extra trees, use the IBM Z Integrated Accelerator for AI with the Snap ML format. Snap ML is an ML framework developed by IBM Research® Zürich to accelerate the training and inferencing of traditional ML models through parallelism. For more information, see Snap ML.

Snap ML addresses enterprise challenges, such as long training times and resource usage, when training or retraining models on large quantities of data. Snap ML is accessible for data scientists who are familiar with the scikit-learn library, which can be installed by using a **pip** command. After conversion to PMML format, scikit-learn models can be converted to Snap ML format to reduce inferencing latency.

4.1.2 Open-source frameworks

Several open source frameworks are available.

TensorFlow

TensorFlow, originated by the Google Brain team, is a commonly used model training framework. TensorFlow is an open source framework that supports deep learning.

TensorFlow on IBM Z uses the IBM Z Integrated Accelerator for AI. The TensorFlow container in the IBM Z and LinuxONE Container Registry (ICR) supports accelerated inferencing.

After training, TensorFlow models can be converted to ONNX format, compiled with the IBM Z Deep Learning Compiler (DLC), and deployed with MLz. MLz compiles the deep learning model in the background.

Scikit-learn

Scikit-learn is a commonly used model training framework. Scikit-learn is an open source framework that supports machine learning. Scikit-learn provides functionality for classification, regression, clustering, dimensionality reduction, model selection, and preprocessing techniques. It uses and builds on open source Python packages such as NumPy, SciPy, and matplotlib.

Scikit-learn can train models on or off IBM Z. For training on z/OS, acquire this framework through the Python AI Toolkit for IBM z/OS as a stand-alone kit or as part of the MLz training platform.

After the machine learning model is built, convert it to PMML format. Depending on the model type, it can use the SnapML runtime and the IBM Z Integrated Accelerator for AI.

PyTorch

PyTorch is a machine learning library originally developed by Meta. PyTorch is an open source framework that supports deep learning for use cases such as natural language processing and image processing.

The IBM Z and LinuxONE Container Registry (ICR) provides containers with PyTorch support. For more information, see [Container Images for IBM Z and LinuxONE](#).

Jupyter Notebooks

Jupyter Notebooks are a foundational tool for running software languages such as Python, Scala, Java, and R. Jupyter Notebooks are open source software that provides a web-based application for creating and sharing documents that contain live code, equations, visualizations, and narrative text.

Jupyter Notebook runs the open source training frameworks discussed. Jupyter Notebook integrates into MLz for model training. The stand-alone Python package can be acquired from the Python AI Toolkit for IBM z/OS, and containers with Jupyter are available on ICR.

4.2 Model formats

After training is complete and the model was built successfully, it must be converted to a usable format that can be used for deployment. You can use model file formats to perform inferencing with a developed model.

The following list describes some of the model formats applicable to MLz:

- ▶ **MLz model**

MLz supports a diverse list of models such as Spark, scikit-learn, XGBoost, PMML, ARIMA, Seasonal ARIMA, ONNX, and Watson Core Time Series. Pre-trained Spark, Scikit-learn, or XGBoost models must be converted into an MLz model format prior to importing into MLz. This can be accomplished by using the provided model utility API.

For more information, see [Importing a model from file](#).

- ▶ **ONNX**

Open Neural Network Exchange, or ONNX, is an open format that is built to represent machine learning and deep learning models. It provides a common file format that developers and data scientists use to train models by using different frameworks then convert the model into the ONNX format.

- ▶ **PMML**

Predictive Model Markup Language (PMML) is a standard and open model format. It is XML-based and similar to ONNX and provides a way to represent models that were trained by using different frameworks.

4.3 Training strategy

A sound training strategy is important because there's a wide range of available architectures that can be implemented on-premises, in a public cloud, in a private cloud, or some variation of each. Although AI projects can be solved in many ways, there are some solutions that are more optimal based on the use case.

4.3.1 Train anywhere and deploy on IBM Z

Businesses increasingly rely on cloud computing to modernize operations. Industries such as banking, insurance, health care, retail, and government run core transactional and batch workloads on IBM Z for security, reliability, and availability. A hybrid cloud approach retains IBM Z platform strengths and uses IBM Cloud services, security, and regulatory compliance programs.

Cloud computing offers flexibility and scalability, enables collaboration, and supports pay-as-you-go pricing. Data scientists can access tools and training platforms, use DevOps pipelines, and access data and applications in the cloud to develop, train, and test AI models.

Companies require intelligence built into transaction processing logic to handle instant payments, real-time Anti-Money Laundering (AML) regulations, increased online transactions,

and fraud. Implementing real-time AI in enterprise workloads without impacting service-level agreements (SLAs) is critical. For clearing and settlement use cases processed in batch workloads, transaction volume is increasing and time windows are shrinking as the industry moves to same-day settlements. Throughput and latency are critical for real-time and batch processing.

The IBM Telum and Telum II processor chips with the IBM Z Integrated Accelerator for AI help make IBM an important provider for inference workloads. Colocating data preprocessing logic and model inference with core transaction processing logic on IBM Z enables real-time inference integration into core business transactions. Clients can score all transactions in real time without impacting mission-critical SLAs.

In Figure 4-1, enterprises organize data from different applications and sources and build and train a model on public cloud or on-premises infrastructure by using a model framework. The model can be deployed on IBM Z to use hardware optimizations for AI in transactions.



Figure 4-1 Train anywhere, deploy on IBM Z

4.3.2 Configuring a secure environment in IBM Cloud

Although cloud provides many benefits, security is the main concern when using cloud computing services. According to Cost of a Data Breach Report 2025¹, the cost of a data breach in the financial industry might reach \$5.56 million in 2025, so defining a secure environment in the cloud is critical.

Financial institutions must comply with cybersecurity standards, laws, and regulations to maintain security posture and prevent data breaches. The global regulatory standards and compliance landscape (Figure 4-2) is complex and evolving. Security regulations and international, federal, and regional laws vary. Financial institutions must comply with more than one set of requirements. Implementing relevant IT standards, regulations, and local laws can be difficult. Financial institutions must adjust their security controls and processes because of frequent changes in the cybersecurity landscape. Meeting compliance requirements is time-consuming and challenging.

Regulatory Authorities	North America	United Kingdom	Europe	Asia-Pacific	Australia
Industry Standards					
Global Standards					

Figure 4-2 Major global regulatory bodies

¹ https://www.ibm.com/security/data-breach?_ga=2.268429834.560404878.1653059563-954325317.1653059563

Designing a secure cloud environment requires consideration of network security, identity and access control, application security, and data security. Designing a secure cloud environment requires deep knowledge in each area to design an infrastructure without security gaps. After implementation and deployment, continuous monitoring helps ensure security.

IBM Cloud is designed for regulatory workloads and works with an ecosystem of regulated clients and ISVs to improve compliance. IBM Cloud supports Hybrid Cloud deployments and enterprise workloads, including regulated workloads, with heterogeneous compute architectures (x86, Z, and Power). IBM Cloud also benefits from the IBM Z platform-centric enterprise cloud services that are offered in its catalog.

IBM Cloud for Financial Services® is part of IBM Cloud (Figure 4-3). It includes IBM Cloud services and ISV applications that comply with the IBM Cloud Framework for Financial Services. IBM Cloud Framework for Financial Services is designed to build trust and enable a public cloud ecosystem of ISVs and IBM Cloud services with security, compliance, and resiliency features that financial institutions require. The Framework uses IBM Cloud services to create a secure and compliant environment. These services include IBM Hyper Protect Crypto Services (HPCS), IBM DevOps toolchain, and IBM Security® and Compliance Center (SCC). The Framework includes reference architectures and best practices. Security and Compliance Center includes profiles with predefined controls and policies that run, monitor, and audit services. The IBM Cloud Regulatory Council determines policies and controls. The IBM Cloud Regulatory Council includes members who are from risk and compliance functions of leading financial industry companies and IBM Promontory® Financial Group to ensure the Framework is current with regulations. ISVs and IBM Cloud services are validated by controls defined by the regulatory council. After validation, they become Financial Services validated.

The IBM Cloud Framework for Financial Services defines reference architectures, which can be deployed by using the infrastructure-as-code DevOps toolchain and used as a basis for meeting the security and regulatory requirements. Sensitive workloads, such as AI model training jobs can then be deployed into the environment. The IBM Cloud Framework for Financial Services includes the following defined reference architectures:

- ▶ [VPC reference architecture for IBM Cloud for Financial Services](#)
- ▶ [Satellite reference architecture for IBM Cloud for Financial Services](#)
- ▶ [IBM Cloud for VMware Regulated Workloads architecture](#)

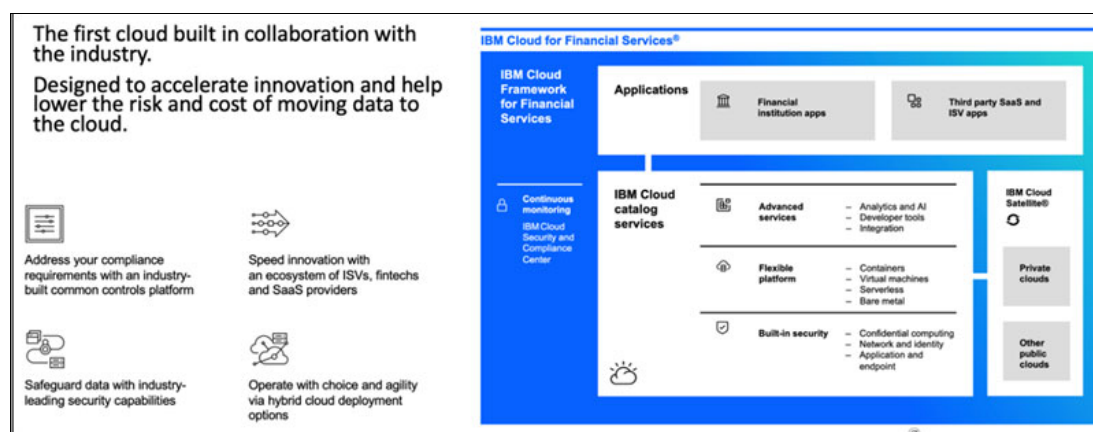


Figure 4-3 IBM Cloud for Financial Services

For more information, see [IBM Cloud for Financial Services](#).

4.3.3 Model development in hybrid cloud

AI model development requires data. Depending on the business problem, data from various sources must be organized for data scientists. Section 3.4, “Data integration patterns” on page 36 describes data integration patterns that enterprises can select based on use case requirements.

Data scientists can develop, train, and test AI models and use their preferred framework on IBM Z or in private or public clouds. Because data can be sensitive and models are valuable enterprise assets, the training environment must be secure and compliant.

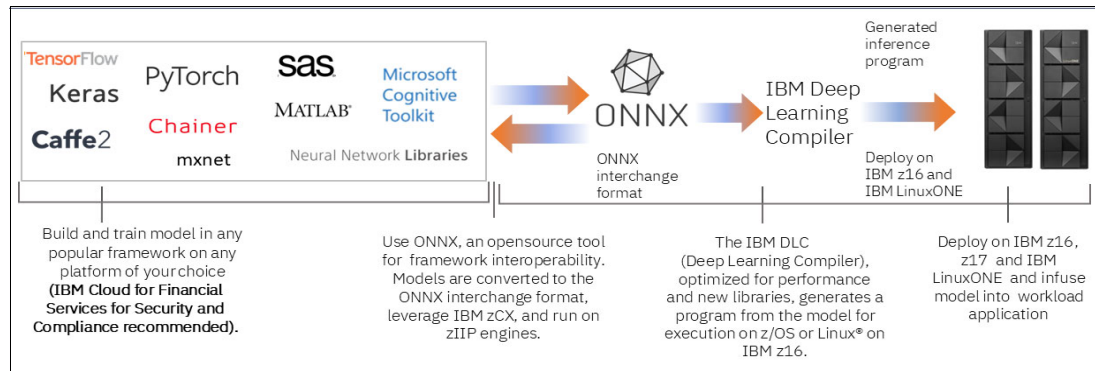


Figure 4-4 Deep learning model development and deployment process

Data scientists can build ML models or DL models to solve business problems. Figure 4-4 shows typical DL model development and deployment and that the model development environment is independent of the model deployment environment. Data scientists can build and train AI models in any framework on any platform that meets security and compliance requirements. After the model is trained and tested, data scientists can convert the model to a format for framework interoperability, such as Open Neural Network Exchange (ONNX) format. An ONNX model can be managed in a secure model repository and securely transferred to IBM Z for testing and deployment through the DevOps pipeline over Direct Link connecting IBM Z with IBM Cloud. The ONNX model can be imported into MLz and compiled by Z Deep Learning Compiler (zDLC). Alternatively, the ONNX model can be compiled manually within a container image of the zDLC to use the Integrated Accelerator for AI on IBM Z.

ONNX is not the only format that can be deployed on IBM Z. Figure 4-5 on page 54 shows supported AI frameworks and libraries for importing developed models into runtime environments on IBM Z. Figure 4-5 on page 54 also shows how the use of the hardware can benefit the model during inferencing. Model types from frameworks can be imported into the IBM Z platform into their native runtime environment, which refers to an Anaconda, Python, or Spark environment in Linux on IBM Z or zCX.

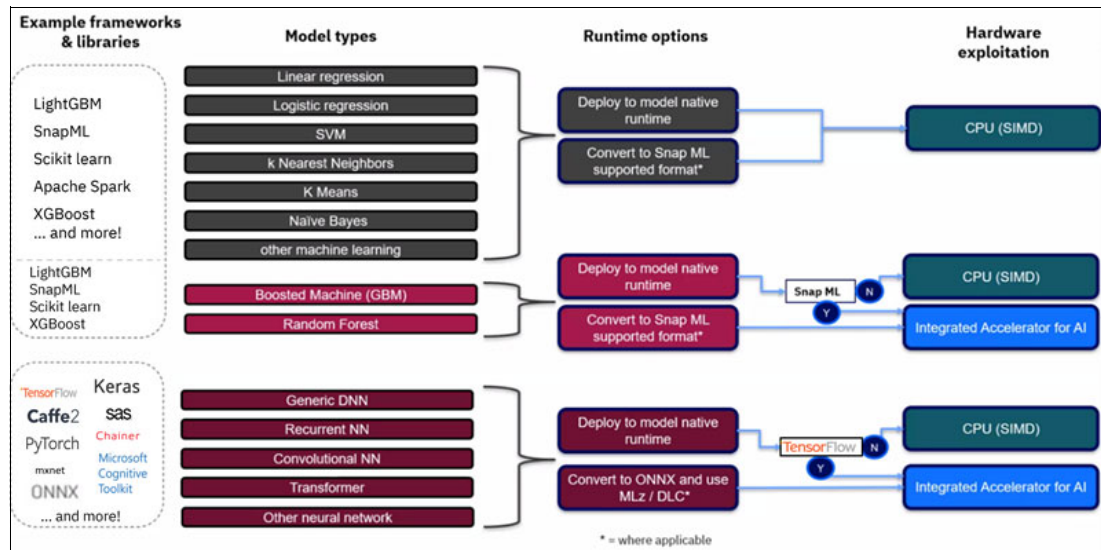


Figure 4-5 Model optimizations on IBM Z

4.3.4 AI reference architecture in hybrid cloud

Enterprise data is sensitive and must be protected. AI workloads must be deployed in a cloud environment that meets regulatory compliance, security, and resiliency requirements. IBM Cloud for Financial Services helps clients mitigate risk and accelerate cloud adoption for sensitive workloads.

AI model building is an iterative process that starts with understanding the business problem and iterates through data understanding and preparation, model training and testing, model deployment, model inference, and model monitoring.

IBM Cloud Framework for Financial Services provides three reference architectures as a basis for meeting security and regulatory requirements defined in the framework. Figure 4-6 on page 55 shows a sample AI reference architecture in VPC for a hybrid cloud environment.

For more information, see [Reference architecture overview](#).

The following steps outline how to set up an AI reference architecture in VPC for a hybrid cloud environment across IBM Cloud and on-premises:

1. Set up the infrastructure by creating a DevOps toolchain for “Deploy infrastructure as Code for the IBM Cloud for Financial Services”, which provides a secure environment for model training jobs. For more information, see the reference architecture for IBM Cloud for Financial Services.
2. Make the enterprise data that is on-premises accessible in the cloud. See the data integration patterns discussed in “Data integration patterns” on page 36.
3. Understand, prepare, and manage the governance of the data with tools in Cloud Pak for Data, for example, organizing and managing data with Watson Knowledge Catalog.

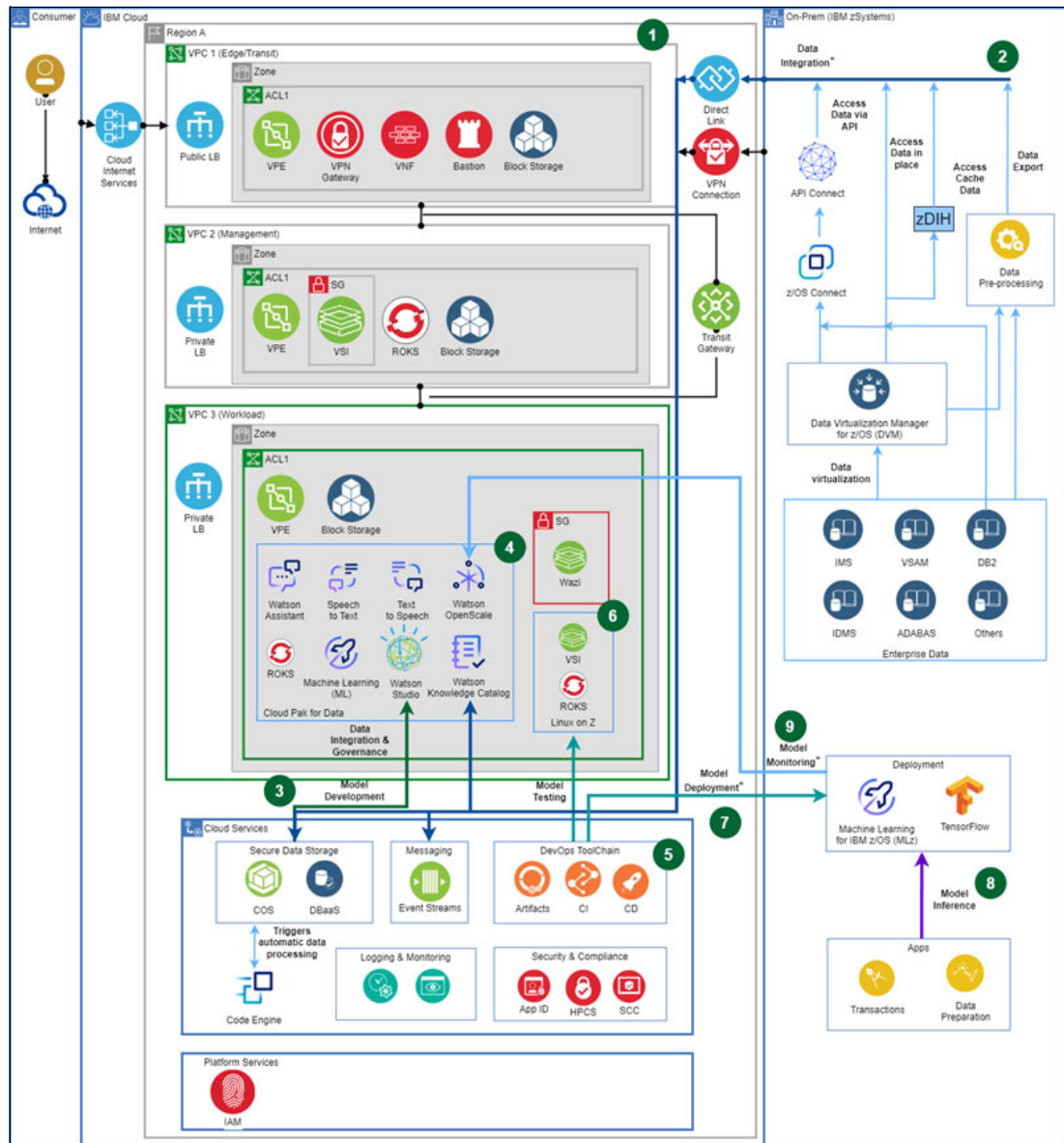


Figure 4-6 Sample AI reference architecture in VPC for a hybrid cloud environment

4. Training jobs can be run in the workload VPC. Various tools in IBM Cloud Pak for Data can be used. Watson Studio can be used as the model development environment. Different machine learning platforms and tools can be used to train the model, such as TensorFlow, PyTorch, and SnapML. The model can be tested in Machine Learning for IBM z/OS. For conversational AI, tools such as Watson Assistant, Speech-to-Text, and Text-to-Speech can be used.
5. IBM DevOps Toolchains can be used to manage the model source codes and tested models.
6. Linux on IBM Z mainframe or IBM Wazi as a Service can be used to test the AI models and AI applications before pushing them to the on-premises mainframe.
7. Tested AI models are deployed on Machine Learning for IBM z/OS.
8. AI model inference is started to gain insights from the data.

9. A watsonx.governance installation provides the capabilities of Watson OpenScale and AI Factsheets, and includes the Model Risk Governance features of OpenPages® in a single service.

4.3.5 Training on-premises with IBM Z

A significant amount of enterprise data is stored directly on IBM Z. Because of the sensitivity of the data, it might necessary to keep data on-premises during training. Figure 4-7 shows the MLz architecture, which supports both on-premises model training and models that are trained on other platforms.

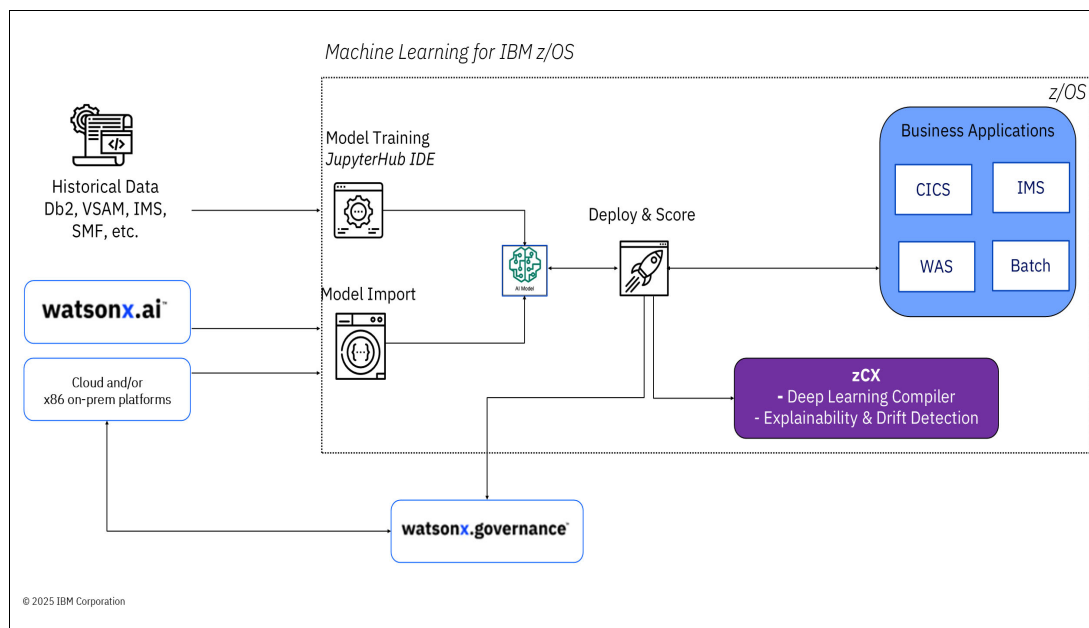


Figure 4-7 MLz architecture

Training with MLz

Model training frameworks run in various environments. On IBM z/OS, models can be built by using MLz UI with the Jupyter Notebook interface. MLz provides an enterprise-ready platform that simplifies model deployment after training.

Figure 4-7 shows the architecture of MLz with an example of how to use a custom ML solution on z/OS to train a model and import it into MLz.

Training with Python AI Toolkit for IBM z/OS

On IBM z/OS, the Python AI Toolkit for IBM z/OS provides Python packages for model training. Available packages include scikit-learn, XGBoost, Jupyter Notebook, NumPy, pandas, and matplotlib.

The latest packages in the Python AI Toolkit for IBM z/OS are available on the product page. For more information, see the IBM Redbooks Solution Guide, *Securely Leverage Open-Source Software with Python AI Toolkit for IBM z/OS*, REDP-5709.

Figure 4-8 on page 57 shows how to use Jupyter within the AI development space to build models with Python AI Toolkit for IBM z/OS. For a more complete environment for AI and open data analytics applications, use IBM z/OS Platform for Apache Spark and IBM Data Virtualization Manager for z/OS.

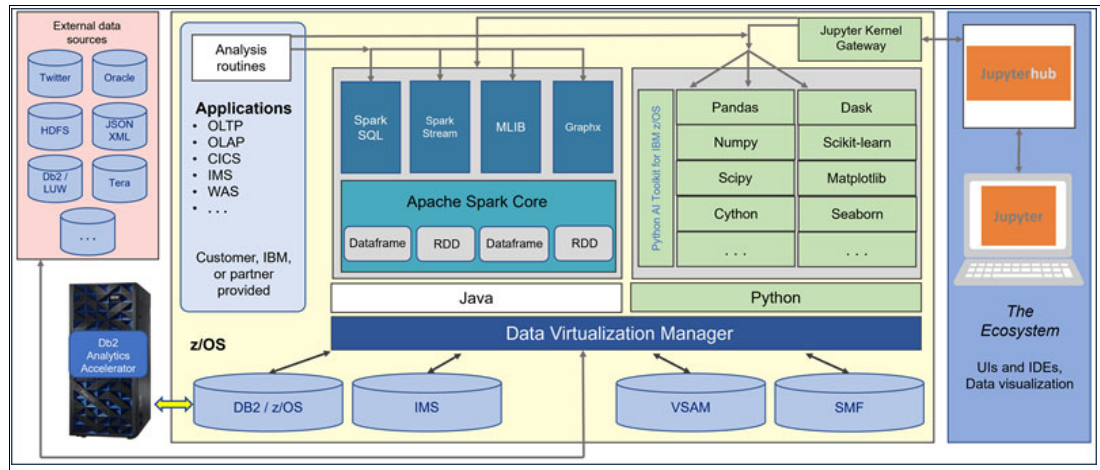


Figure 4-8 Journey to Open Data Analytics on IBM Z

You can use IBM Z Platform for Apache Spark to virtually integrate multiple data sources in a single application. Apache Spark processes unified data and integrates with business applications.

IBM Data Virtualization Manager for z/OS provides virtual, integrated views of data on IBM Z. It enables users and applications to access IBM Z data in place for read and write operations without moving, replicating, or transforming the data.²

Training with containers on Linux on IBM Z

You can run model training frameworks on IBM zCX or Linux on IBM Z environments to use containers from the IBM Z and LinuxONE Container Registry. These training frameworks can also run within x86 environments, such as Windows, Mac, and Linux.

Available containers include TensorFlow, PyTorch, and many more. For a complete list of the latest containers, see [Container Images for IBM Z and LinuxONE](#).

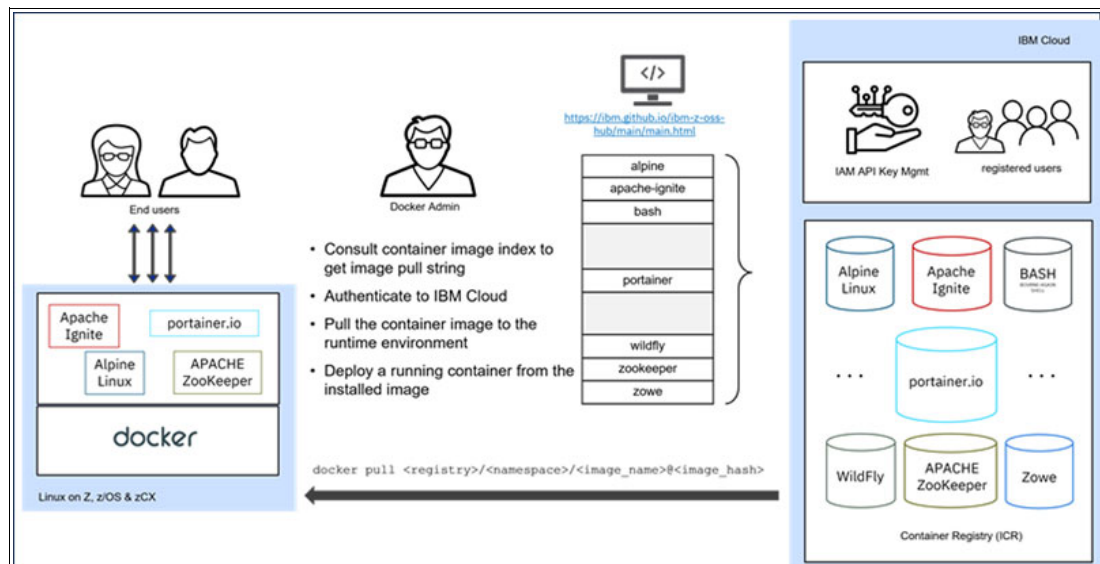


Figure 4-9 IBM Z and LinuxONE Container Repository

² <https://www.ibm.com/support/z-content-solutions/journey-to-open-data-analytics/>

Figure 4-9 on page 57 shows the high-level architecture of a development environment that uses ICR. To perform model training within a Linux on Z, z/OS, or zCX environment, you can pull the docker files directly from ICR.

4.4 Best practices for model training

During model building, although each problem is different, there is a set of best practices that can be applied.

4.4.1 Training strategy considerations

When building a training strategy, identify where data resides. Data accessibility is critical to model building.

Factors that affect training times

Model training is resource-intensive. Dataset size and other variables affect training time. Data quality is more important than data quantity. Model type and hyperparameters affect training times. Additionally, the underlying hardware infrastructure and available resources determine training speed.

Model update frequency

Use cases differ in model retraining frequency. For example, if the model makes decisions based on data generated hourly or daily, build and deploy a new model rapidly. The training pipeline considerations for this use case differ from use cases that only require historical data from the previous year. A subsequent section describes how MLOps integrates with the model training process.

4.4.2 Considerations for choosing a framework

When you choose a framework to train AI models, no single framework provides all answers. General techniques can be applied when experimenting.

Deep learning versus machine learning

Different model types have strengths and weaknesses depending on the use case. Consider the strengths of each model to find the best fit and maximize performance.

Determine whether the use case requires traditional machine learning or deep learning. If the use case involves image processing or natural language processing, a deep learning model is likely needed. For deep learning, use TensorFlow and PyTorch.

If the use case requires traditional machine learning models, use frameworks such as scikit-learn, Snap ML, and XGBoost.

Open source versus enterprise platform

Different training frameworks are available. You can use stand-alone open source frameworks, such as TensorFlow and scikit-learn, to build a customized training infrastructure. MLz supports numerous open source frameworks and provides an end-to-end enterprise platform that enables seamless integration for deployment.

Cloud versus on-premises

Frameworks and technologies are available for on-premises and cloud training. Most technologies available on IBM Cloud and other cloud providers are native on IBM Z. For training natively on z/OS, MLz is the full end-to-end enterprise platform. To develop a custom model training infrastructure, use the open source Python packages in the Python AI Toolkit for IBM z/OS.

4.4.3 Best practices on saving models for deployment on IBM Z

After training is complete, save the model so that it can be used for deployment. When training with MLz, you can save your model directly within your environment. As previously mentioned, you can reference how to save the model using the integrated Notebook Editor in MLz. You can use the MLz UI to import a previously exported MLz model, as shown in Figure 4-10.

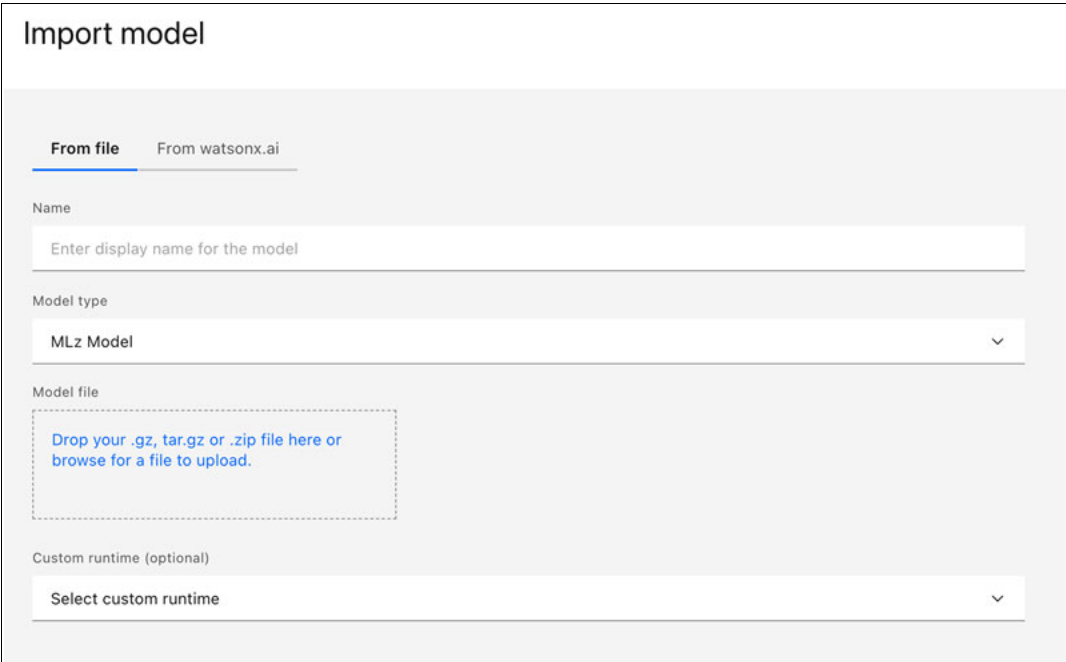


Figure 4-10 Import MLz model to MLz

By using the model utility provided by MLz, you can save Python models, such as scikit-learn and XGBoost, that were trained on any platform. You can save the model into a format that can be imported into MLz for scoring on IBM z/OS. No matter where you decide to train your models, you can deploy them on IBM Z by converting them to ONNX or PMML formats.

For more information, see [Importing a model from file](#).

Example 4-1 shows how to convert a trained TensorFlow model to ONNX. This conversion requires running Python and uses the `tf2onnx` tool to perform the conversion of the deep learning model.

Example 4-1

```
# Run python command with tf2onnx to convert deep learning model to ONNX, and save it to the local file system
```

```
!python -m tf2onnx.convert --saved-model $work_dir"fraud_cnn_model" --opset 10
--output $work_dir"model.onnx"
```

After you convert the model to ONNX, you can import it using the MLz UI (Figure 4-11).

Converting a trained scikit-learn model to PMML requires a Python environment to run the **sklearn2pmm1** library. The **sklearn2pmm1** package performs the conversion with the given machine learning pipeline. The following line of code can be used to convert a trained scikit-learn model to PMML:

```
sklearn2pmm1(pipeline, 'random_forest.pmm1', with_repr=True)
```

After you convert the model to PMML, you can import it using the MLz UI, as shown in Figure 4-12.

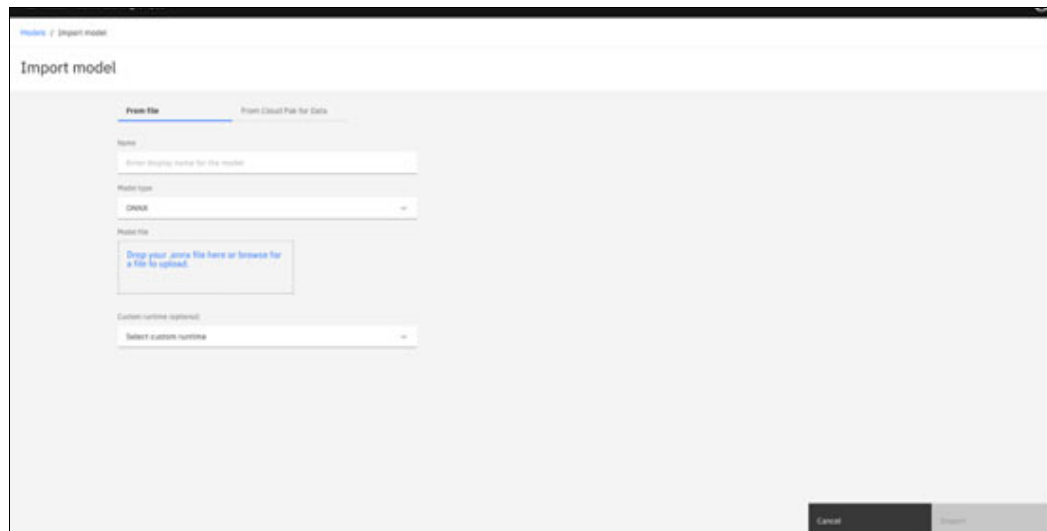


Figure 4-11 Import ONNX model to MLz

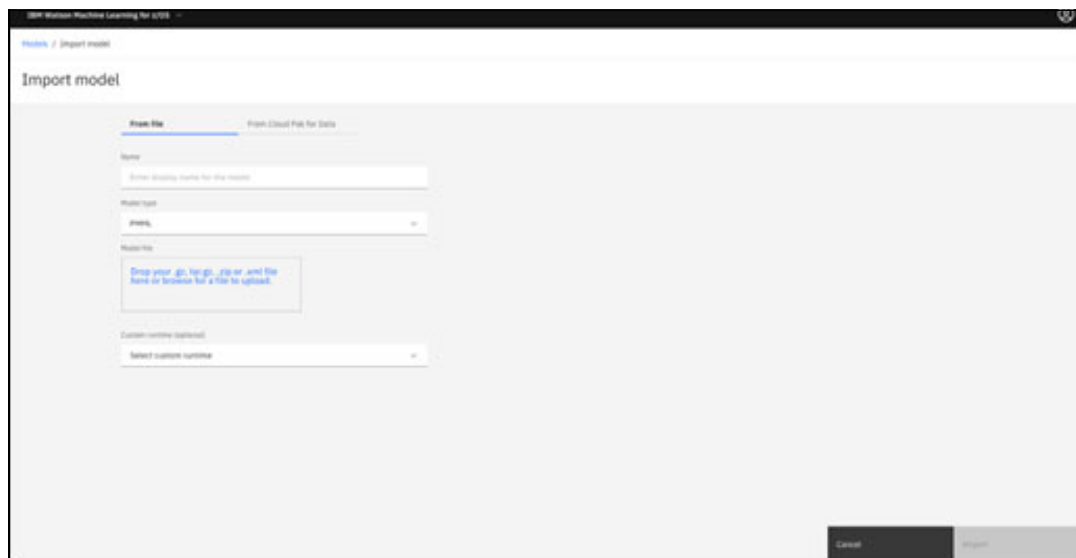


Figure 4-12 Import PMML model to MLz

Example of saving CP4D model

If you trained your model using CP4D, you can also import it into MLz using the UI, as shown in Figure 4-13. Unlike the other model formats, CP4D is not imported from file. Instead, you provide various details that are specific to the CP4D instance and model, such as hostname, port, user name, password or API key, deployment space, mode, and model name.

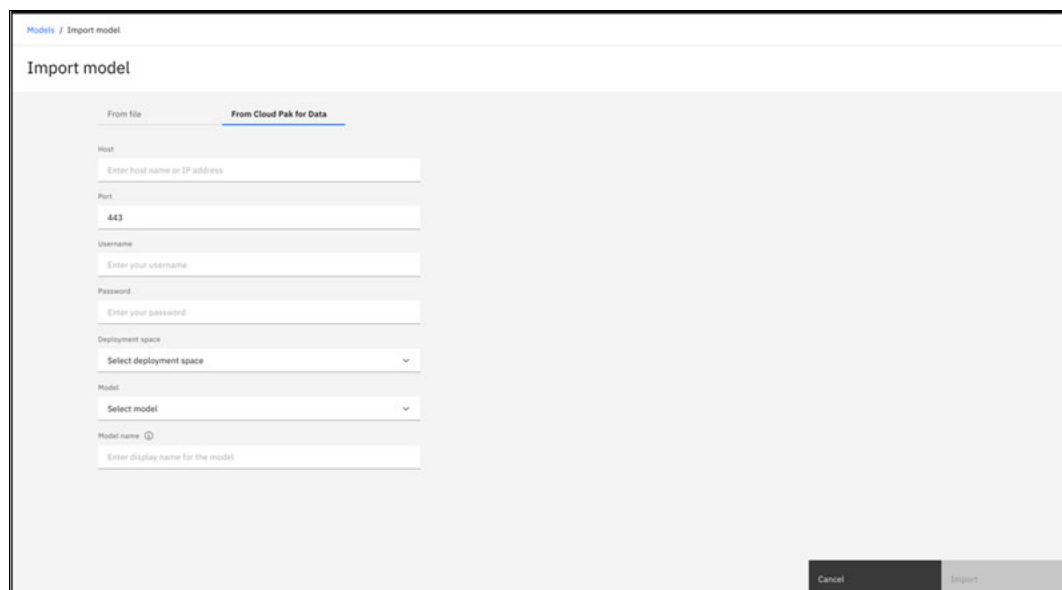


Figure 4-13 Import CP4D model to MLz

4.4.4 Evaluating model formats

MLz supports various model types. You can train your models off-platform either by using ONNX or PMML.

By using ONNX, you can deploy your models to IBM Z with MLz. ONNX does not include preprocessing support, so you might need additional steps at inference time to handle unprocessed data. ONNX uses a standard format, which provides an advantage because many platforms support it.

By using PMML, you have some advantages. One advantage of the PMML format is that you can embed preprocessing directly in the generated PMML pipeline. This approach handles preprocessing at inference time without requiring additional code to process incoming data.

4.4.5 Tuning model parameters

To improve the performance of the model, training requires hyper parameter tuning. This tuning is an experimental process. Each of the training frameworks discussed support to efficiently perform model tuning. As the number of hyper parameter variations increases, the computation and time that is needed to build the model increases.

4.5 Model building use cases

After the credit card transaction dataset has been cleaned, you can perform model training. You can use several different model frameworks and formats. Two use cases are presented in this section.

Use case 1: Training a TensorFlow model and saving to ONNX format

The following steps are provided as an example workflow to build a fraud detection model by using TensorFlow:

1. Build a sequential neural network model with a couple of layers as outlined in Example 4-2.

Example 4-2 Sequential neural network model with a couple of layers

```
# Build a Sequential model
model = Sequential()

# Add a 1D convolution layer
model.add(Conv1D(32, 2, activation='relu', input_shape = X_train[0].shape))
model.add(BatchNormalization())
model.add(MaxPool1D(2))
model.add(Dropout(0.2))

model.add(Conv1D(64, 2, activation='relu'))
model.add(BatchNormalization())
model.add(MaxPool1D(2))
model.add(Dropout(0.5))

# Convert 2D data to vector
model.add(Flatten())
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.5))

# Define output data shape and activation as sigmoid
model.add(Dense(1, activation='sigmoid'))
```

2. Compile the TensorFlow model with Adam optimizer:

```
# Compile the model with Adam optimizer
model.compile(optimizer=Adam(learning_rate=0.0001), loss =
'binary_crossentropy', metrics=['accuracy'])
```

3. Train the TensorFlow model:

```
# Fit the model
model.fit(X_train, y_train, epochs=epochs, validation_data=(X_test, y_test))
```

4. Save the model:

```
# Save model to local file system
model.save(work_dir + "model")
```

5. Convert the model to ONNX:

```
# Run python command with tf2onnx to convert deep learning model to ONNX, and
save it to the local file system
!python -m tf2onnx.convert --saved-model $work_dir"fraud_cnn_model" --opset 10
--output $work_dir"model.onnx"
```

Use case 2: Training a scikit-learn model and saving to PMML

In this example, use scikit-learn to build a fraud detection model by following these steps:

1. Split the training and testing data:

- ```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=1)
```
2. Define the preprocessing pipeline steps:

```
feature_transformers = [
 ('num', Pipeline(steps=[('normalizer', StandardScaler())])), num_feats),
 ('cat', Pipeline(steps=[('normalizer', OrdinalEncoder())])), cat_feats)
]
preprocessor = ColumnTransformer(transformers=feature_transformers)
```
  3. Build a pipeline with random forest model:

```
pipeline = PMMLPipeline([
 ("preprocessor", preprocessor),
 ("classifier", RandomForestClassifier())
])
```
  4. Train model pipeline:

```
pipeline.fit(X=X_train, y=y_train)
```
  5. Convert model pipeline to PMML:

```
Sklearn2pmml(pipeline, 'random_forest.pmml', with_repr=True)
```





## Model deployment and inferencing

Model deployment and model inferencing are two key stages in the AI model lifecycle that work together to enable machine learning solutions.

Model deployment involves taking a trained AI model and making it accessible for real-world applications. Machine Learning for IBM z/OS (MLz) provides a flexible platform that supports various model types and deployments into z/OS environments. With version control, automated deployment features, and additional services, MLz enables you to deploy models with ease and efficiency.

Model inferencing uses the trained, deployed model to make predictions on real-world data. At this stage, a model applies the knowledge that it learns during training to make near real-time predictions, such as image recognition, natural language processing, or anomaly detection. In MLz, models make predictions based on transactional data, providing valuable insights in various industries and applications.

This chapter covers the following topics:

- ▶ 5.1, “Model deployment environments” on page 66
- ▶ 5.2, “Model deployment on mainframe systems” on page 67
- ▶ 5.3, “Model development and deployment on Machine Learning for IBM z/OS” on page 69
- ▶ 5.4, “Model inferencing” on page 72
- ▶ 5.5, “Sample use case for MLz model deployment” on page 77
- ▶ 5.5, “Sample use case for MLz model deployment” on page 77
- ▶ 5.6, “Best practice on model deployment and inference” on page 96

## 5.1 Model deployment environments

In today's data-driven world, machine learning models and algorithms are central to many decision-making processes. Effectively using these models and harnessing their predictive capabilities is now crucial for organizations. Model deployment can take place in various environments, and mainframe systems continue to be a key option.

### 5.1.1 Advantages of mainframe systems

When you work with artificial intelligence (AI) and machine learning (ML), you must deploy models effectively to transform insights into tangible results. The choice of deployment environment plays a pivotal role in determining your success. You can choose from many options. However, mainframe systems offer several key advantages.

- Infrastructure and resources

Model deployment environments provide the infrastructure and resources that support ML models. When you choose a deployment environment, you must ensure that it can handle expected workloads, process complex computations efficiently, and deliver accurate results on time. Mainframe systems allow you to curate your environment with flexibility and variety. You should consider factors such as scalability, performance, data storage, and computational power when you select an environment.

- Scalability and flexibility

Whether you face an increase in data volume, a growing user base, or expanding use cases, your deployment environment must accommodate these changes as your business grows. Starting with IBM z16, mainframe systems prioritize scalability for model deployments by using built-in AI acceleration with Telum and Telum II processors.

- Time sensitive

Latency is critical in model deployments for time-sensitive, high-throughput applications. For instance, use cases such as fraud detection and autonomous vehicles require enough computational power to process data quickly and deliver timely predictions or actions. With mainframe systems, model deployment and inference run on the same system where the data resides. This colocation significantly reduces response time. Mainframe systems also offer clustering and high availability to minimize service interruptions during critical processes.

- Data security and compliance

Data security and compliance are critical in AI implementation, especially when you handle sensitive information. You must ensure that your deployment environment enforces strong safeguards to protect data throughout the AI lifecycle. Mainframe systems secure data by using strong storage protection, encryption mechanisms, access controls, and compliance features. As a result, mainframe systems provide a deployment environment that prioritizes data security, reduces the risk of data breaches, and maintains compliance with privacy regulations.

### 5.1.2 On-premises versus cloud-based deployment

You must decide whether to use on-premises or cloud-based deployments. Each approach offers specific advantages and considerations. Your decision depends on factors such as data privacy requirements, existing infrastructure, budget constraints, and your overall AI strategy. You might also choose a hybrid approach that uses both on-premises and cloud resources to balance control, security, and scalability.



### **Consideration for on-premises deployment**

On-premises deployment prioritizes control, security, and compliance. This approach is ideal when you handle sensitive data and operate under strict data privacy guidelines. By keeping models close to the data, on-premises deployments also support low-latency performance and give you greater oversight of your deployment environment.

- ▶ **Enhanced data control**

On-premises deployment gives you complete control over your data. This is crucial for industries with strict data privacy and compliance requirements because it allows you to maintain sensitive data within your secure infrastructure.

- ▶ **Low latency and increased performance**

When you host AI models on-premises, you reduce latency and achieve faster response times. This is especially important for near real-time applications that require immediate decision-making.

- ▶ **Increased infrastructure costs**

On-premises deployment requires initial investments in hardware, networking, and ongoing maintenance.

- ▶ **Scalability challenges**

Scaling on-premises infrastructure can be resource intensive and time-consuming. You must plan for future growth and ensure that your infrastructure can handle increasing workloads and data volumes.

### **Consideration for cloud-based deployment**

Cloud-based deployment prioritizes scalability, cost efficiency, and accessibility. You should carefully weigh the advantages and considerations of cloud-based deployments by reviewing your specific requirements, data sensitivity, performance needs, and cost constraints.

- ▶ **Data security and compliance**

You must evaluate the security measures that cloud service providers offer and ensure that they meet your data security and compliance requirements. This includes assessing data encryption, access controls, and regulatory compliance.

- ▶ **Latency and network dependency**

Cloud-based deployments depend on network connectivity, which can introduce latency. For certain near real-time applications with strict latency requirements, an on-premises deployment might be more suitable.

By selecting the optimal deployment option, you can deploy AI models effectively and unlock the transformative potential of AI in your business.

## **5.2 Model deployment on mainframe systems**

Model deployment on Machine Learning on IBM z/OS (MLz) provides significant advantages, including scalability, performance, reliability, security, and integration capabilities. With MLz, you can achieve optimal performance, ensure high availability, maintain data security and compliance, integrate with your existing infrastructure, and achieve cost efficiencies.

## 5.2.1 Large-scale inferencing

One of the key advantages of deploying machine learning models on MLz is the ability to achieve large-scale, low-latency in-transaction inference. MLz is compatible with the specialized AI accelerators Telum and Telum II. These AI accelerators are purpose-built to run AI computations efficiently, enabling you to perform high-volume transaction processing and large-scale model inferencing. This capability is valuable in scenarios that require near real-time decision-making, such as fraud detection, risk scoring, and dynamic pricing.

By using the computing power and AI accelerators available on MLz, you can ensure that your models deliver fast, accurate, near real-time insights. This enables you to respond to rapidly changing conditions and make informed decisions immediately.

One of MLz's primary applications is detecting and preventing fraudulent activities across industries such as financial services, insurance, and retail. By analyzing transactional data, customer behaviors, and historical patterns, machine learning models can identify anomalies and detect potential fraud. With Telum II, you can also use multiple models by adding a secondary round of inferencing. This additional inferencing stage can leverage a deep learning model that recognizes and learns emerging fraud patterns, increasing your model's effectiveness for future inference requests.

## 5.2.2 Leveraging data gravity

Data gravity is a metaphor that describes the gravitational pull that large data sets exert on applications, services, and processing. If z/OS contains most of your transactional data, it is advantageous to deploy applications and machine learning models within the mainframe environment. This approach reduces data movement, enhances data access and performance, simplifies integration with existing infrastructure, strengthens data security and compliance, reduces latency, and supports effective data governance.

By leveraging z/OS data gravity, you can harness the full potential of your transactional data and derive valuable insights within a trusted and centralized environment, right where the data resides.

## 5.2.3 Using existing mainframe data hubs for AI model deployment

Using existing mainframe data hubs for AI model deployment is a strategic approach that harnesses the power of machine learning within the mainframe environment. With increased interest in AI and machine learning, you can take advantage of your mainframe data and infrastructure to derive valuable insights.

MLz provides integration with both `watsonx.ai` and `watsonx.governance`. Use `watsonx.ai` to build and train models, and then easily import them into MLz to make them available for real time transactional scoring. When a model is deployed in MLz, you can also connect it with `watsonx.governance` to ensure it meets regulatory and privacy requirements. These trusted governance practices give you confidence that your AI deployments remain secure, compliant, and aligned with organizational policies.

### Eliminating the need for extensive data transfers

When you deploy AI models in the mainframe environment, you eliminate the need for extensive data transfers between systems or platforms. Data transfers are resource-intensive and time-consuming, especially with large volumes of data. Moving data between systems also adds costs and introduces potential security risks and data integrity issues.

## 5.2.4 Ensuring data integrity, security, and compliance

Achieving and maintaining data integrity, security, and compliance is critical in any AI deployment, especially when you work with sensitive and mission-critical data. IBM Z is known for robust security features that are designed to maintain the confidentiality and availability of your data.

By keeping your data in the trusted mainframe environment, you reduce the risk of data leakage or attacks during data transit. IBM z16 and later systems include a Trusted Execution Environment (TEE) that lets only authorized software run on IBM Z. This prevents injection-style attacks from unauthorized technologies. As a result, IBM Z protects your workloads and data inside and outside the system.

### Data integrity

Mainframe data hubs store critical transactional data that is essential for AI model training and inference. The data is protected by mechanisms such as data redundancy, checksums, and error-detection and correction codes. By deploying AI models within the mainframe environment, you can use these built-in data integrity measures to ensure that the data you use for training and inference remains accurate and consistent.

### Data security

Mainframe systems include built-in security features that safeguard sensitive data. Tamper-resistant Hardware Security Modules (HSMs) store and manage cryptographic keys, actively detecting and responding to tampering attempts. HSMs meet FIPS 140-2 Level certification and comply with PCI-DSS, HIPAA, GDPR, and eIDAS requirements. Additionally, quantum-safe encryption introduced with IBM z16 and Telum is designed to withstand future quantum threats. As a result, mainframe systems offer security that protects against traditional data breaches and remains resilient against emerging threats.

### Data compliance

Mainframe systems are designed to comply with various regulatory standards and industry-specific requirements. The IBM Z Security and Compliance Center (zSCC) automates compliance management. For example, zSCC ensures that only compliance-relevant data is collected from mainframe systems. The collected data is time-stamped and transformed into immutable formats to ensure traceability and trust.

zSCC also offers a compliance dashboard that shows your compliance status across all systems. You can use this centralized, interactive view to monitor your systems and identify at-risk environments over time.

## 5.3 Model development and deployment on Machine Learning for IBM z/OS

To streamline model deployment and management, MLz offers a suite of tools that simplify machine learning operations for various key roles. For instance, system administrators can manage and monitor the services through the administration dashboard to ensure their smooth operation. Machine learning administrators or data scientists can deploy machine learning models as services to applications like CICS, Java, and others on the Z platform. Then, application developers can use these services in their own applications to make near real-time predictions.

These tools are accessible with REST APIs or a user-friendly web interface (UI). MLz's UI is easy to use and minimizes the machine learning curve for beginners, thereby reducing the need for extensive training. This not only saves time and resources but also increases user adoption rates. Clear instructions, visual cues, and context-specific help also make it easy for expert users to familiarize themselves with the application's navigation.

### 5.3.1 Model development

You can import machine learning models from your local system into the MLz UI or build models directly on the platform by using the built-in JupyterHub. Building models directly in JupyterHub further maintains data integrity and minimizes security risks. By keeping data in place, you avoid the risks associated with moving large volumes of sensitive, transactional data.

MLz also provides embedded runtimes such as Spark on z/OS and the Python AI Toolkit and uses mainframe AI accelerators such as Telum and Telum II. With these offerings, you can train models faster and at scale.

### 5.3.2 Model deployment

The MLz UI also offers a streamlined model deployment process.

When you deploy models, you can choose from a variety of deployment targets that are tailored to your application requirements. Whether you deploy to CICS, Java, or other z/OS applications, MLz accommodates diverse environments, making it a versatile solution for infrastructures with different needs.

The deployment process involves packaging trained machine learning models into a format that seamlessly integrates with the target application. MLz streamlines this process to save time and resources while ensuring compatibility. The deployment experience is designed to be plug-and-play so that you can quickly activate the model as a service within your applications.

After deployment, models are readily accessible and provide near real-time predictive capabilities to target applications. As a result, your z/OS applications can quickly use insights from machine learning models without requiring complex data transfers or manual interactions.

You can also create a multiple-model AI architecture with MLz that improves fraud detection in the IBM Z environment and enables you to make data-driven decisions with greater confidence.

Multiple-model AI improves fraud detection because modern fraud detection systems must reason across several dimensions, including transactional, temporal, and contextual factors. Relying on a single model often forces trade-offs between precision and recall. Multiple-model AI addresses these issues by combining models that are optimized for different aspects of fraud detection.

For example, you can use a traditional predictive model followed by an encoder-based large language model (LLM) in a fraud prediction workflow. In this workflow, 100% of transactions are first evaluated by a traditional machine learning or deep learning model (such as an XGBoost model) to ensure low-latency scoring for all inputs. This initial layer identifies clear-cut cases of fraud. For transactions that receive a fraud probability score below a predefined confidence threshold (for example, less than 80%), the workflow escalates them to a second layer of fraud detection powered by an encoder-based LLM, such as a BERT model.

This additional model provides enhanced contextual understanding to refine predictions for borderline cases.

By combining the speed of traditional AI with the depth of LLMs, this two-tiered inferencing strategy balances accuracy and throughput. It improves decision quality without compromising SLA targets on IBM z17 leveraging Telum II with MLz.

The ability to deploy machine learning models as services within z/OS applications also opens new opportunities for automation, optimization, and enhanced customer experiences.

### 5.3.3 Model monitoring

The MLz Trustworthy AI feature provides an interface that enables you to monitor model performance. After you deploy a model, MLz monitors the deployment for Drift, which is the degradation of a model's inferencing caused by factors such as newly ingested data. MLz also monitors explainability, which evaluates the explanation behind an AI prediction.

### 5.3.4 Scalability

IBM Z's design principles of reliability, availability, and security are integrated into the MLz platform. IBM Z provides the scalability that helps organizations handle increasing workloads and large-scale deployments.

MLz seamlessly integrates with a wide range of tools, libraries, and frameworks, enabling you to use your existing investments. It supports popular machine learning frameworks such as scikit-learn, XGBoost, PMML, Open Neural Network Exchange (ONNX), and Apache Spark. This support enables data scientists to use familiar tools for model development and training. This integration simplifies your transition to MLz and accelerates time-to-value for machine learning projects.

MLz takes advantage of the Telum processor on IBM Z to accelerate deep learning ONNX model inference. This improvement is achieved by combining the advanced model conversion tools of the IBM Deep Learning Compiler (DLC) with the on-chip AI accelerator available with Telum on IBM z16 and Telum II on IBM z17. Additionally, MLz simplifies the deployment of machine learning models at scale. It provides a platform for deploying models as services to support near real-time predictions and inference. The mainframe's robust processing capabilities help organizations handle high volumes of requests, ensuring smooth and uninterrupted model deployments. MLz also integrates with existing enterprise systems, enabling organizations to infuse machine learning capabilities into critical business processes.

### 5.3.5 Secure and trusted environment

To establish a secure and trusted environment for MLz, consider the following guidelines:

- Authentication and authorization

MLz implements an authentication and authorization mechanism that aligns with the existing z/OS platform. It uses the System Authorization Facility (SAF) and JSON Web Token (JWT) technology to validate the user ID and ensure that you are authorized to perform the requested action.

- Secure communication

Protect the communication channels used by MLz with Transport Layer Security (TLS) or Secure Sockets Layer (SSL) encryption protocols. This protection ensures that data

transmitted between users and the application remains confidential and secure. You can strengthen your network security by using the Application Transparent Transport Layer Security (AT-TLS) capability on z/OS. With AT-TLS support, your MLz services communicate securely with client applications by using policy-based TLS/SSL protection on z/OS.

## 5.4 Model inferencing

Machine learning inferencing, also referred to as scoring, is the phase in the machine learning life cycle in which a trained model makes predictions or classifications on new, unseen data. This stage represents the practical application of the knowledge and patterns that the model learns during training. When you submit new data to the model, it uses the insights gained during training to generate meaningful outputs. This process involves passing input data through the model's algorithms, which produce predictions, classifications, or scores based on the relationships the model identifies. Machine learning inferencing plays a pivotal role in enabling automated decision-making, near real-time recommendations, and the extraction of insights from raw data. It transforms a trained model from a learning system into a practical tool that supports a wide range of applications, including fraud detection, image recognition, natural language processing, and autonomous driving.

In-transaction analytics transforms how organizations use their existing data and introduces a new way to make decisions. With this approach, you can uncover new insights, make predictions, and respond quickly. Instead of relying solely on historical reports, you analyze data as it is generated, which helps you anticipate future outcomes. This capability benefits many industries by enabling timely customer guidance, fraud prevention, increased sales opportunities, improved customer insights, better product management, and stronger competitive positioning. MLz supports these goals by enabling you to use near real-time information to make informed predictions and gain immediate business value.

### 5.4.1 Making online predictions

The significance of machine learning scoring becomes most evident when AI models predict future outcomes, particularly for near real-time business insights. Industries such as payment fraud detection, loan approval, and client onboarding benefit greatly from this capability. By implementing machine learning scoring, you can evaluate the significance of transactions and assign scores directly on the platform where the transactions occur. This approach provides important advantages for IBM Z customers. It strengthens security, improves overall performance, and enables you to maximize the value of your existing IBM Z infrastructure by optimizing resources and enhancing operational efficiency. Machine learning scoring is essential for generating accurate predictions and supporting informed decision-making across critical domains, contributing to a more secure, efficient, and insight-driven business environment.

MLz outlines a typical machine learning life cycle that includes several stages, beginning with data preparation and ingestion. Data—whether sourced from historical systems such as IBM Db2 for z/OS or imported from open-source platforms—is prepared for analysis. The model is then trained to improve its predictive capabilities. After training is complete, the model enters the deployment and scoring phase. MLz provides a range of interfaces to support different application environments, including WebSphere Application Server, CICS, IMS, and IBM Z batch workloads, all of which facilitate business-logic processing.

MLz accelerates AI model development by enabling training anywhere and ensuring that scoring occurs on z/OS. It also leverages Telum II to provide ultra-fast execution, enabling

end-to-end fraud mitigation to run entirely within the mainframe stack. This hybrid architecture shows how traditional ML models and encoder-based LLMs work together to provide secure and scalable fraud detection at industry-leading speed and precision. MLz supports a wide range of model types, including SparkML models; Python-based models such as scikit-learn, XGBoost, and ARIMA; PMML models; and deep learning and encoder based models such as BERT and RoBERTa in ONNX format. This flexibility enables you to use diverse model types to meet your inferencing needs.

The ONNX open standard plays a pivotal role in enabling seamless framework interoperability. It allows you to convert models from frameworks such as PyTorch, TensorFlow, Caffe, and Keras to ONNX format by using each framework’s APIs. MLz can import deep learning and encoder based models, such as BERT and RoBERTa, in ONNX format and prepare them for near real-time inferencing.

To compile an ONNX model for deployment and inference, MLz uses the IBM zDLC. This compilation process requires either zCX or Linux on Z. When the ONNX model is prepared, its scoring runs natively on the z/OS platform within the Liberty server environment. Because the scoring engine is Java-based, it qualifies for the IBM Z Integrated Information Processor (zIIP), which is cost-effective for you. In addition, ONNX scoring gains a substantial performance boost from the Telum II on-chip AI accelerator featured in the z17 architecture.

ONNX model inferencing also supports micro-batch scoring, which helps you handle higher-throughput transactions. This approach ensures that the platform can manage high transaction volumes and respond effectively to diverse business demands.

Overall, ONNX integration within the MLz framework provides a robust architecture that supports model deployment, inferencing, and optimization on the z/OS platform. See Figure 5-1 for an overview of ONNX model inferencing.

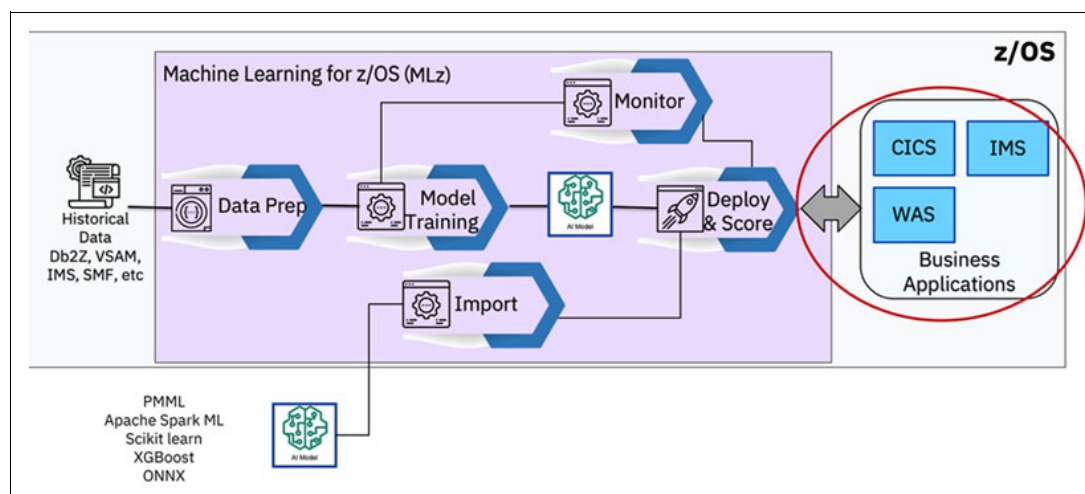


Figure 5-1 ONNX model inferencing

The MLz scoring server offers inferencing interfaces for z/OS developers, including REST, Java, CICS, and WOLA. It enables you to score within ongoing transactions while maintaining z/OS performance and resiliency standards. It achieves single-digit millisecond scoring, enabling rapid decision-making and seamless integration into operations. See section 5.6, “Best practice on model deployment and inference” on page 96 for guidance on choosing the right inferencing interfaces for your use case.

## 5.4.2 Interfaces of MLz online scoring service

Figure 5-2 illustrates two types of interfaces that the MLz online scoring service offers. One common interface type is RESTful, supporting a variety of model types, including SparkML, PMML, ONNX, and Python models. This interface uses JWT token authentication for security. When an application interacts with this interface, it issues a REST API that includes the user provided Serving ID or auto generated Deployment ID. You submit input data, such as the record to be scored, in JSON format, and the scoring output is also returned in JSON. The actual scoring process occurs within the MLz scoring server (Liberty server), where predictive models compute the result. For Python models, MLz uses the Python scoring server (Gunicorn server), tailoring the process to the needs of Python-based workloads.

The second option is the Java interface, which supports SparkML, PMML, and ONNX models. This interface was designed to integrate with rule-based engines such as IBM Operational Decision Manager (ODM). It is particularly useful for transaction-processing scenarios on Liberty or WebSphere Application Server because it enables the MLz scoring service to run on the same Liberty or WebSphere instance. This proximity enables your applications to invoke the MLz scoring service through a native Java API, eliminating the need for RESTful interactions. This design fosters closer component integration and enhances performance, resulting in a more efficient and streamlined scoring process.

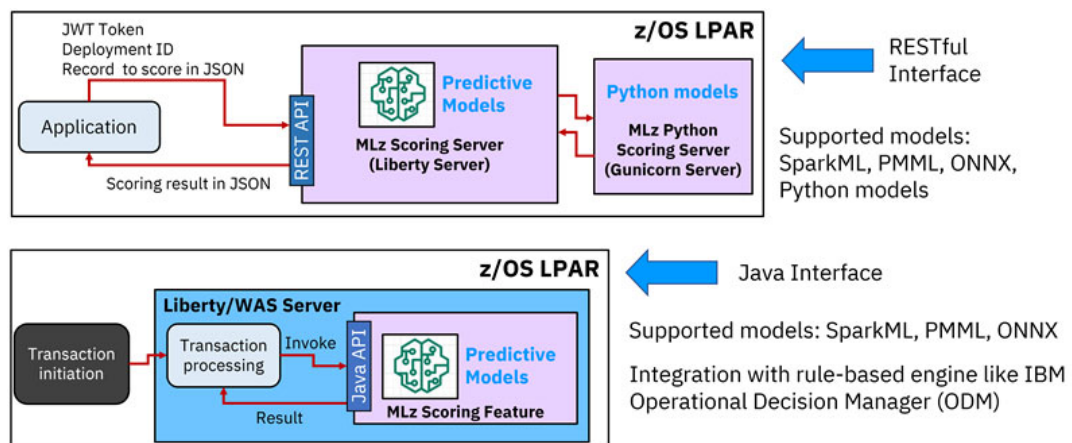


Figure 5-2 Scoring interfaces on z/OS (REST and Java)

WebSphere Optimized Local Adapters (WOLA) is a feature of IBM WebSphere Application Server for z/OS Liberty. It plays a crucial role in managing communication between the Liberty for z/OS server and external address spaces such as CICS, batch, or IMS within the same logical partition. By enabling WOLA in the scoring service, you can perform online scoring for SparkML, PMML, and ONNX models. You can accomplish this by using the native WOLA APIs in your COBOL programs, which can run in CICS, IMS, or batch jobs.

The WOLA scoring interface (Figure 5-3 on page 75) is designed for IBM Z native applications. A key advantage of this interface is its memory-to-memory data exchange between COBOL applications—running in batch jobs, IMS, or CICS—and the MLz scoring server. Unlike traditional network-layer exchanges, this method improves efficiency and enhances security. By using the native callable interfaces provided by the WOLA APIs, you minimize the impact on existing COBOL applications and reduce the effort required from IBM Z application programmers. This streamlined approach aligns with the system architecture, ensuring secure and efficient data exchange while maintaining compatibility with established processes.



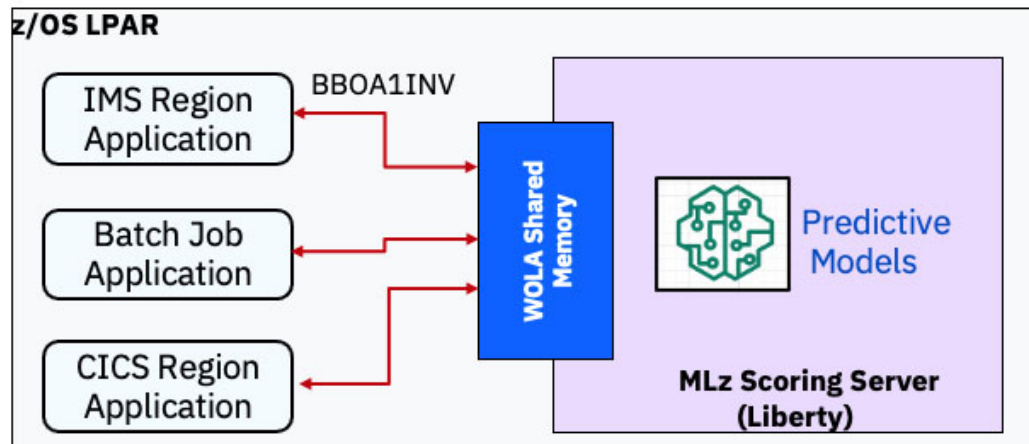


Figure 5-3 Example of using WOLA interface

Figure 5-4 on page 76 shows an example of scoring by using the WOLA interface with a COBOL application. It uses BBOA1INV, a native callable inference routine provided by the WOLA APIs. Input parameters include `rqst-area-addr` and `service-name`. The `rqst-area-addr` parameter serves as a pointer to the data sent to the scoring server. It includes information such as the model-serving ID and the model input record. The `service-name` parameter uses the following value in the example shown in Figure 5-4 on page 76:

```
java:global/scoring-wola/WOLASrvHandler!com.ibm.ml.scoring.online.service.WOLASrvHandler
```

When you call the MLz WOLA scoring service, an output parameter named `resp-area-addr` is used. This parameter serves as a pointer to the data area that carries the scoring output, which shows the model's probability prediction. This COBOL example clarifies how scoring works by using WOLA APIs and illustrates the input and output parameters and their interaction with the MLz scoring service. As a result, native IBM Z application developers do not need to handle JSON or HTTP requests associated with RESTful APIs.

Through the CICS interface, you can initiate online scoring for SparkML, PMML, and ONNX models (see Figure 5-5 on page 76). In a CICS region, a Liberty server operates under the name `ALNSRVSC`. To perform online scoring for SparkML, PMML, and ONNX models within your CICS COBOL application, you use the CICS LINK command. By using this command, you start the `ALNSRVSC` interface, enabling seamless online scoring. This setup provides a simplified programming approach for CICS COBOL applications and makes it easier to integrate model inferencing. It also delivers higher performance compared to the RESTful interface.

```

95 * Assign scoring service name
96 MOVE 'java:global/scoring-wola/WOLASrvHandler!com.ibm.ml.scoring.online.service.WOLASrvHandler'
97 TO service-name.
98 * Register into Daemon Group using BBOA1REG API
99 INSPECT daemongroup CONVERTING ' ' to LOW-VALUES.
100 DISPLAY "Attempting to register WOLA with: "
101 CALL 'BBOA1REG' USING
102 daemongroup,
103 node-name,
104 server-name,
105 register-name,
106 minconn,
107 maxconn,
108 regopts,
109 rc,
110 rsn.
111
112 * Invoke using BBOA1INV for scoring
113 CALL 'BBOA1INV' USING
114 register-name,
115 rqst-type,
116 service-name,
117 service-name-length,
118 rqst-area-addr,
119 rqst-area-length,
120 resp-area-addr,
121 resp-area-length,
122 wait-time,
123 rc,
124

```

Figure 5-4 Example of using the WOLA interface for service registration and invocation

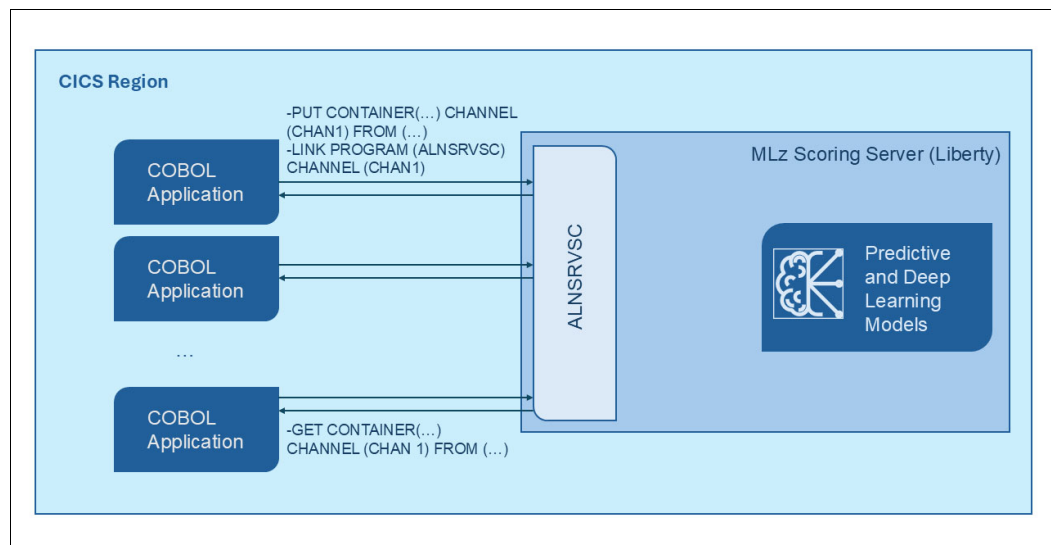


Figure 5-5 CICS regionSample use case for MLz model deployment

## 5.5 Sample use case for MLz model deployment

MLz supports a wide range of use cases and provides flexibility for diverse requirements. With its robust capabilities, MLz helps you address multiple scenarios effectively. Whether you are developing and deploying machine learning models, performing advanced analytics, or using AI technologies, MLz provides the tools and features to support a broad range of needs. Its adaptability and comprehensive functionality make it a strong solution for many use cases and enable you to apply machine learning and AI effectively in your domain.

A specific use case involves predicting z/OS batch job elapsed time.

A single host typically runs 10,000 to 60,000 jobs per day and operates 24 hours a day. These batch jobs often have interdependencies, and even a slight oversight can lead to repeated work and reduced efficiency. Efficient batch processing is essential to meeting the high productivity demands of enterprises.

For financial institutions such as banks, tasks such as salary payments, mortgage repayments, interest accruals, and other operations usually run as nightly batch jobs after business hours. Ensuring that these jobs complete within their required time windows is critical for maintaining normal daytime operations and supporting ongoing banking transactions and services.

Organizations often lack effective methods for monitoring and predicting the progress of batch jobs. You can perform post-analysis, rerun failed jobs, or estimate job duration based on experience, but these approaches can be inconsistent.

This use case describes how you can use MLz to predict the elapsed time of a batch job on the z/OS platform. It explains how MLz integrates with the mainframe environment and highlights its capabilities for optimizing and enhancing performance for batch job processing on z/OS.

### 5.5.1 Batch job elapsed-time prediction

Batch job elapsed-time prediction enables you to estimate how long a batch job takes to complete. This predictive capability uses historical data, job complexity, and other factors. These insights help you improve planning, resource allocation, and decision-making across your workflows. By predicting batch job duration, you can optimize operations and enhance overall efficiency.

### 5.5.2 Business questions and personas

Business questions and personas are essential components of a successful use case because they guide solution design and ensure that the outcome addresses real business needs. By focusing on these aspects, you can develop data-driven strategies and make informed decisions that support growth and operational effectiveness.

#### **Business questions for batch monitor**

In today's fast-paced business environment, time is critical. For z/OS organizations that rely on batch applications for essential data processing, estimating how long these applications take to complete is vital for efficient planning and decision-making. This use case uses advanced analytics and predictive modeling techniques to help answer two crucial business questions.

1. How long will it take for my z/OS batch applications to complete today?

Batch applications play a vital role in processing large volumes of data on a scheduled basis. Accurate run-time estimates help you manage resources effectively, optimize workflows, and meet operational deadlines. By using predictive modeling and historical data, you gain insight into expected batch-job run times and can make informed decisions about resource allocation.

In a mainframe system, approximately 10,000 to 60,000 jobs run throughout the day. These jobs are often interconnected, and even a small oversight can lead to repeated work and reduced efficiency. Efficient batch processing is essential to maintaining high productivity.

For organizations in industries such as banking and finance, tasks such as salary payments, mortgage repayments, interest calculations, and other operations typically run as overnight batch jobs after business hours. Ensuring that these jobs complete within their assigned time window is critical for smooth daytime transaction processing and overall operational continuity.

When you invest in accurate completion-time prediction models for your batch applications, you gain a competitive advantage in today's fast-paced environment. By improving workflow efficiency and resource allocation, you enhance operational performance, strengthen customer satisfaction, and support better overall business results.

## 2. How long will a z/OS batch application take to complete after a specific job finishes?

In complex data-processing environments, batch applications are often interconnected, and some jobs act as prerequisites for others. Knowing when downstream jobs or applications complete is essential for maintaining smooth operations and meeting critical business requirements. Accurate estimates help you manage dependencies, schedule jobs effectively, and optimize resource allocation. This approach reduces downtime and improves workflow efficiency.

Estimating completion time after a specific job finishes requires analyzing historical data, understanding job dependencies, and applying predictive modeling techniques.

Figure 5-6 shows a z/OS batch application that includes three key milestone jobs. These milestone jobs represent major stages in the overall batch-processing flow. By using the available data, you can estimate the total execution time for the batch application.

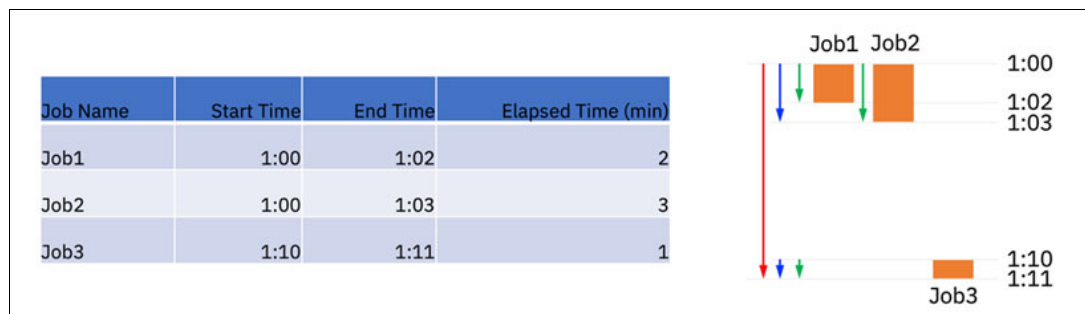


Figure 5-6 Batch application with sample jobs running example

Example 5-1 illustrates how to calculate the batch execution elapsed time for a batch run with a total duration of 11 minutes.

### Example 5-1 Elapsed Time - Batch Execution Time

Elapsed Time with Batch Execution Time=  
 $\max(\text{End Time}) - \min(\text{Start Time}) = 1:11 - 1:00 = 11\text{min}$

After you assign start and end times to each job, each job in the batch application runs for a specific duration. By continuously monitoring the z/OS batch application's progress, you can predict the expected end time for each job, including job1, job2, and job3. If job2's predicted end time deviates from the expected time frame, you can identify potential issues with job2. When you detect such deviations, you receive immediate alerts. These alerts prompt you to investigate and address underlying problems without waiting for subsequent jobs to complete.

The objective extends beyond knowing the overall completion time for today's batch applications. You also determine the anticipated completion time for batch applications after a specific job finishes.

## **Personas for batch job elapsed time prediction**

When you predict z/OS batch execution end times, several key factors influence the process. You can view these factors as Personas that represent different responsibilities or influences on prediction accuracy. Each Persona represents a distinct role that contributes to the batch execution end-time prediction.

### **► IT operations personnel**

With AI assistance, IT operations personnel can monitor the health of batch processing jobs and accurately predict their completion time. This AI-powered solution streamlines the monitoring and management of batch processing applications, ensures efficient CPU usage, and helps you meet defined batch processing time windows.

When you monitor and manage the execution status of batch processing applications with AI, you gain several benefits. By continuously tracking the execution process, the solution quickly identifies anomalies or deviations from expected behavior. This near real-time insight allows you to detect and address issues promptly during batch processing execution, which improves efficiency and reduces downtime.

The AI-powered solution also provides insights into the overall health and performance of batch processing jobs. It analyzes metrics such as resource consumption, execution times, and historical patterns to identify potential optimization opportunities. With this information, you can make informed decisions that improve the efficiency of batch processing operations.

### **► System administrator**

The system administrator uses performance data from existing batch processing jobs to gain valuable insights. Through thorough analysis and evaluation, you can identify areas for improvement within architecture management. This strategic optimization enhances the overall efficiency and effectiveness of the system.

By examining performance metrics and analyzing job behavior, you identify patterns, bottlenecks, and opportunities for optimization. These insights enable you to make informed decisions and implement architectural changes that align with your organization's goals.

With advanced analytics and AI-driven techniques, you identify improvement areas and receive recommendations for architectural enhancements. These recommendations consider factors such as resource usage, job dependencies, and system constraints. When you incorporate these suggestions, you optimize the architecture management process and unlock the system's full potential. This proactive approach allows you to address performance issues or inefficiencies promptly. When you strategically optimize the architecture based on analysis results and recommendations, you ensure that batch processing jobs run smoothly, meet deadlines, and maximize resource utilization.

This data-driven and strategic approach empowers you to make informed decisions, drive continuous improvement, and enhance the overall performance and efficiency of the batch processing system.

► Architect

As the z/OS architect, you use the performance of existing batch processing jobs to optimize architecture management based on analysis results and recommendations. In addition to this analysis, you require a comprehensive visual reporting tool that provides a detailed overview of the daily batch processing executions for various applications.

The visual report presents trends in the daily execution of batch processing jobs. It includes key metrics such as execution times, resource consumption, job dependencies, and other relevant factors. By visualizing this information, you gain insight into performance patterns and identify areas for improvement.

With the visual report, you can easily identify anomalies, bottlenecks, or inefficiencies in batch processing workflows. Its visual representation enables a quick and intuitive understanding of overall performance trends and the impact of specific applications on the system.

With this comprehensive data and visual insight, you make informed decisions and evaluate optimizations that enhance the architecture management process. These actions might include adjusting resource allocations, optimizing job scheduling, or refining application dependencies. The visual report serves as a powerful tool to support these strategic adjustments and optimizations.

### 5.5.3 Batch job elapsed time modeling building

By using dynamic prediction of batch processing run times, you can anticipate the duration of each job accurately. This capability helps you plan workflows effectively, allocate resources optimally, and streamline the data processing pipeline. As a result, you can address potential bottlenecks and resource constraints proactively, which minimizes idle time and maximizes resource usage.

To develop a model that communicates effectively with z/OS batch data in System Management Facility 30 (SMF30) records, consider how the model uses the inherent features of z/OS. Building a successful model requires a thorough understanding of the characteristics and data structures of z/OS, including how z/OS manages and processes batch data and the specific information captured in SMF30 records.

#### Define the prediction target

Predicting batch completion time before the batch begins is a crucial step in efficient batch processing. By using historical data, analyzing job dependencies, and considering resource availability, you can estimate the expected time that it will take for the batch to finish.

During this predictive phase, you can identify potential bottlenecks or areas of concern that might affect the performance of the batch job. By using this early insight, you can proactively allocate resources and optimize job scheduling, ensuring smooth execution and meeting strict time constraints.

Additionally, predicting batch completion time in advance enables better planning and coordination across the organization. It facilitates effective resource allocation, ensuring that sufficient CPU power, memory, and storage are available for the smooth execution of the batch job. The accuracy of this prediction depends on the quality and relevance of historical data used for analysis. Continuous improvement of the prediction model through regular updates with fresh data enhances the precision of the estimates.

By being proactive in predicting batch completion time before it begins, you can maximize resource usage, improve workflow efficiency, and boost overall productivity in the enterprise.

Another crucial step in efficient batch processing is to dynamically update the prediction during batch execution. Traditional batch processing systems often rely on static predictions that are made before the batch starts. However, scenarios can be dynamic, with unpredictable factors that can impact the execution time of individual jobs. For example, varying data volumes, system loads, or external events can affect the actual completion time of jobs within the batch.

To address the limitations of static predictions, modern batch processing systems increasingly adopt dynamic prediction mechanisms. Dynamic updates continuously monitor the progress of batch jobs during execution and adjust predictions in real time based on observed data:

- ▶ Near real-time monitoring<sup>1</sup>

Dynamic updates require near real-time monitoring of job progress and resource utilization. You achieve this by using monitoring tools and data analytics platforms that track key performance metrics such as run time, resource consumption, and job dependencies.

- ▶ Data-driven insight

The data collected during the batch job provides insight into the actual performance of batch jobs. By analyzing this data, you identify patterns, trends, and potential bottlenecks that the initial static prediction did not account for.

- ▶ Adaptive algorithms

Dynamic updates rely on adaptive algorithms that continuously learn from incoming data. These algorithms adjust their predictions based on new information, ensuring that you receive the most accurate estimates throughout batch execution.

## Hypothesis for model building

Hypotheses for model building are crucial when you predict batch application elapsed time. A hypothesis is a testable assumption about the relationship between variables. It guides the model-building process and ensures that you develop the model with a clear objective.

Business-related information and metrics can affect the duration of batch processing jobs, such as daily transaction volume and the number of accounts. Batch processing plays a central role in managing and processing large volumes of data. However, job duration does not depend solely on technical aspects of the system. Business metrics can significantly influence execution time.

In batch processing, job performance and execution time directly relate to core business operations. Several business-related metrics influence how efficiently batch jobs run. Two critical factors that significantly affect batch processing job duration are:

- ▶ Daily transaction volume

Daily transaction volume is a key business metric that significantly affects batch processing job duration. As transaction volume increases, the amount of data that batch jobs must process also rises. This increase can lead to longer execution times as the system handles larger data loads.

Transaction volume can also vary across days of the week or during peak periods, which creates fluctuations in batch processing job times. Understanding these patterns helps you optimize scheduling and resource allocation for batch jobs.

---

<sup>1</sup> We use the term near real-time instead of real-time because true real-time processing implies zero or extremely low latency and continuous, instantaneous analysis of every event as it occurs, which can introduce significant processing overhead and generate excessive event volume for operators to manage. Near real-time processing provides the same operational value for most use cases but avoids the system strain and information overload that strict real-time monitoring can create.

- Number of accounts

The number of accounts in your database directly influences batch processing job execution time. The more accounts you process, the longer the jobs take to complete. This factor is especially relevant in industries such as banking and finance, where large customer bases generate a high number of accounts to process.

In conclusion, when you develop a predictive model for batch jobs, prioritizing jobs based on their business significance is crucial for ensuring timely execution. You must give high-priority jobs, such as critical financial transactions, precedence to safeguard essential business operations from delays. Understanding the impact of each batch job on business outcomes allows you to prioritize strategically, optimize resource allocation, and improve overall system performance.

Successfully implementing a predictive model for running batch jobs requires a careful assessment of the business impact of each job. By categorizing jobs based on their importance to core business operations, you manage job scheduling effectively and allocate resources as needed.

You can also form hypotheses based on when batch jobs run. The runtime of preceding jobs can influence the execution of subsequent jobs, which can create bottlenecks and delays. This section discusses the impact of job dependencies on job runtime and examines strategies to optimize performance and enhance overall system efficiency.

Batch processing involves running a series of tasks in a predefined sequence, where one job's output becomes the next job's input. These interrelated jobs form a complex web of dependencies, and their execution order significantly affects overall efficiency.

Job dependencies generally fall into two categories:

- Sequential dependencies

These dependencies occur when a job starts only after the preceding job finishes and produces the required data or results.

- Parallel dependencies

In this scenario, multiple jobs run concurrently and operate independently. A subsequent job might depend on the combined output of these parallel jobs.

When you consider job dependencies, the runtime of preceding jobs becomes a critical factor in the overall batch duration. If a preceding job takes longer than expected, it delays subsequent jobs and creates a cascading effect across the batch processing workflow.

Poor job sequencing can also result in resource contention when multiple jobs compete for limited resources simultaneously. This contention increases wait times and reduces overall resource efficiency.

Job dependencies create a sequence of tasks where one job's output becomes the next job's input. The order in which you run jobs significantly impacts the total elapsed time of the batch workflow. Therefore, your predictive model must account for these dependencies to estimate each job's completion time accurately.

## **Data engineering for batch jobs**

In a z/OS-specific batch job elapsed-time modeling process, collecting and using historical training data are critical. Understanding the relevant data factors is essential for building an effective predictive model. The following key data elements play an important role in this process.



## ***Business data***

Business data includes the information and records your organization generates during daily operations. This data supports decision-making, improves operational efficiency, and helps maintain a competitive advantage. When you analyze and model batch job elapsed time, consider the following business-related factors.

- **Daily online transaction volume**

Daily online transaction volume represents the total number of transactions processed in an online system during a single day. These transactions include purchases, payments, transfers, and similar user interactions.

Transaction volume significantly affects batch job elapsed time. When earlier jobs process a larger data load, downstream jobs might experience delays, creating cascading effects throughout the batch workflow. Transaction volume can also vary across days of the week or during peak hours. Understanding these fluctuations helps you optimize scheduling and resource allocation.

- **Number of accounts**

The number of accounts in your database directly affects the volume of data that batch jobs must process. Each account includes information such as customer details, transaction history, and preferences. As the number of accounts increases, batch jobs handle more data, which can lengthen elapsed times.

A larger account base also increases demand for CPU, memory, and storage resources. Batch jobs might encounter resource contention as they compete for limited system capacity, resulting in longer processing times. Effective workload distribution can help mitigate these impacts and improve overall performance.

- **Calendar-related information**

Calendar-related constraints influence when batch jobs can run. Limited batch windows can lead to delays, longer wait times, and extended elapsed times if jobs cannot begin immediately. Factors such as business hours, time zones, holidays, month-end cycles, and seasonal workloads can all affect job-run times.

Calendar-related information also affects personnel availability for scheduling, monitoring, and troubleshooting tasks. Aligning job scheduling with staffing coverage helps ensure timely support during critical batch-processing periods.

By incorporating calendar-related considerations into your analysis, you improve batch-processing efficiency, support on-time completion of critical activities, and ensure alignment with business requirements.

## ***Batch feature data***

Batch feature data includes the specific data elements and attributes that you use to analyze, model, and predict the performance of batch processing jobs. These features play a critical role in building machine learning models that help you understand batch-application behavior and performance.

When you work with batch feature data, consider the following important aspects:

- **Job list (names of all jobs belonging to the batch)**

The job list provides essential information about the order in which jobs are scheduled to run. Batch jobs might have dependencies in which one job's output becomes the input for another. By understanding the job list, you enable the predictive model to account for these dependencies and estimate the elapsed time for each job and for the overall batch workflow accurately.

The job list also indicates the batch windows during which jobs can run. These time constraints can influence scheduling and resource-allocation decisions, which in turn affect elapsed-time prediction.

- ▶ **Start job and end job of the batch**

The start job and end job of the batch have a significant impact on z/OS batch job elapsed time prediction. These two points define the boundaries of when the batch job runs and provide essential information for accurate time prediction.

The start job marks the beginning of the batch workflow, and the end job marks its completion. When you identify these points, the predictive model focuses on the relevant time range for making predictions. When you consider only the jobs between the start and end points, the model analyzes the required data efficiently and optimizes predictions for that specific batch execution.

When you identify the start and end jobs, you can understand the batch window constraints in which the jobs must run. This information influences scheduling decisions, helps ensure that the jobs run within the required time frame, and reduces potential delays.

The order in which the jobs run in the batch workflow is essential for accurate prediction. The start job and end job provide sequencing context, enabling the model to understand dependencies and relationships among jobs. Accurate sequencing contributes to precise elapsed time predictions.

In a z/OS batch job, you can collect and analyze several performance metrics to gain insight into its behavior. The following metrics are commonly used:

- ▶ **Elapsed time:** You can measure the total time that the batch job runs.
- ▶ **CPU time:** You can track how much processor time the job consumes.
- ▶ **I/O numbers:** You can review the number of input/output operations the job performs.
- ▶ **Start and end time:** You can record when the job begins and completes.
- ▶ **Historical data:** You can use one year of historical data for analysis and modeling.
- ▶ **SMF data:** You can collect SMF 30 performance data on z/OS (subtype 5).

## **Significance of SMF30 records in batch job elapsed time prediction modeling**

System Management Facility 30 (SMF 30) records are critical components of z/OS that capture essential performance data for batch job accounting and monitoring. These records provide insight into batch job execution, including job identification details, resource usage metrics, start and end times, elapsed time, job step information, and execution statistics. For batch job elapsed time prediction modeling, the inclusion of SMF 30 records is crucial for several reasons.

SMF 30 records offer a comprehensive set of performance metrics related to batch job execution. These metrics include CPU time, I/O operations, memory usage, and other indicators that directly affect batch job elapsed time. When you integrate these metrics into the modeling process, the predictive model gains a complete view of job performance.

When you use SMF 30 records, the prediction model becomes data-driven because it relies on actual performance metrics rather than assumptions or generalizations. Data-driven decision-making allows you to make informed choices, optimize resource usage, and improve the efficiency of batch job processing.

An example of an SMF30 record data is shown in Example 5-2 on page 85. In this example, you work with SMF 30 data that is already loaded and processed into the format required for analysis. The time range for the analysis spans from November 1, 2019, through October 30,

2020. Within this period, the job list range includes jobs with identifiers from APP001 through APP050 for each day. This approach provides a comprehensive view of batch job performance and enables you to examine job run time patterns and trends throughout the specified time frame.

*Example 5-2 Elapsed time batch execution time*

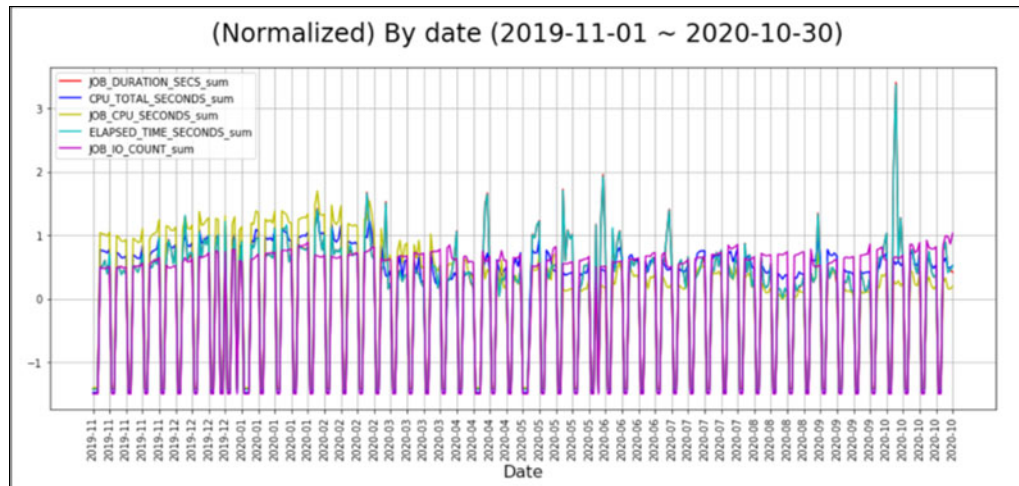
```
smf30_file_path=r'data/Sample_SMF.csv'
job_list_file_path=r'data/all_joblist.csv'

APP_NAME = "AP"
start_jobs = ['APP001']
cutoff_jobs = ['APP050']
```

**Normalized data for SMF30 metrics**

Figure 5-7 shows a chart of the daily sums for JOB\_DURATION\_SECS, CPU\_TOTAL\_SECONDS, JOB\_CPU\_SECONDS, ELAPSED\_TIME\_SECONDS, and JOB\_IO\_COUNT during the period from November 1, 2019, through October 30, 2020. Among these metrics, ELAPSED\_TIME\_SECONDS is especially important for the modeling process. This metric shows the duration of batch job execution for each day in the specified time frame.

When you pay close attention to ELAPSED\_TIME\_SECONDS, you can gain insight into how long the batch jobs run each day during the observed period. This information helps you build predictive models and optimize batch job performance. Related metrics, such as CPU time and I/O count, also provide useful insight into batch job behavior and overall performance.



*Figure 5-7 Normalized SMF30 data for batch application*

Figure 5-8 on page 86 shows a detailed chart of the elapsed time metrics, including ELAPSED\_TIME\_SECONDS\_EXT, ELAPSED\_TIME\_SECONDS\_INT, and ELAPSED\_TIME\_SECONDS\_SUM. These metrics provide comprehensive insight into different aspects of batch job execution time and help you understand batch job performance in greater detail.

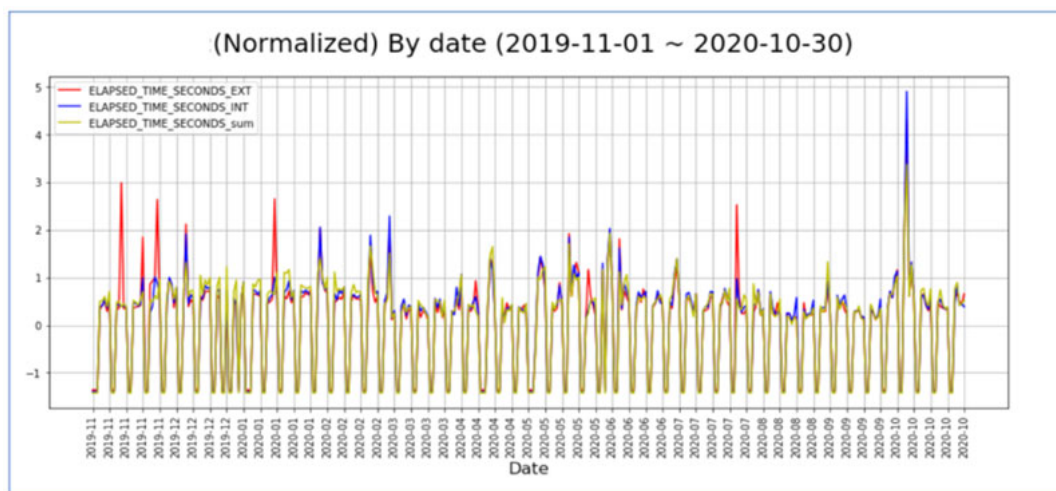


Figure 5-8 Normalized SMF30 Elapsed time for batch application

Figure 5-9 helps you understand the metrics ELAPSED\_TIME\_EXT, ELAPSED\_TIME\_INT, and ELAPSED\_TIME\_SUM.

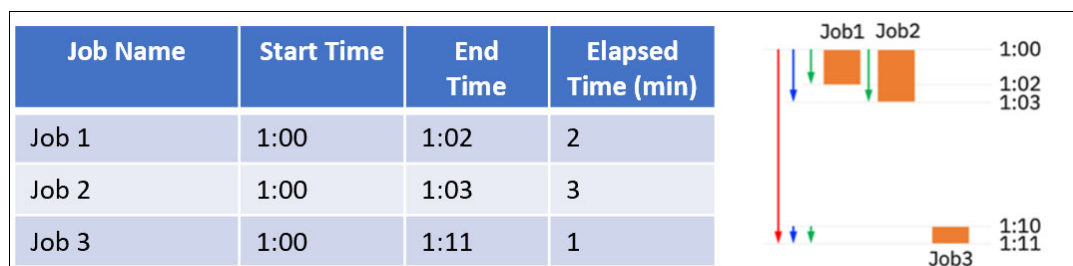


Figure 5-9 Batch application with sample jobs elapsed time

You can use the formulas in the elapsed time metrics to calculate the three previously mentioned values.

#### Example 5-3 Elapsed time metrics

**ELAPSED\_TIME\_EXT:** 11 minutes. ## the model prediction target.

Elapsed Time with External Application Waiting Time=  
 $\max(\text{End Time}) - \min(\text{Start Time}) = 1:11 - 1:00 = 11 \text{ minutes.}$

\*\*\*\*\*

**ELAPSED\_TIME\_INT:** 4 minutes.

Elapsed Time within Internal Application=  
 $\text{sum}(\text{continuous elapsed time}) = (1:03 - 1:00) + (1:11 - 1:10)$   
 $= 3 + 1 = 4 \text{ minutes.}$

\*\*\*\*\*

**ELAPSED\_TIME\_SUM:** 6 minutes

Elapsed Time of all individual jobs=  
 $\text{sum}(\text{every job's elapsed time}) = 2 + 3 + 1 = 6 \text{ minutes.}$

Figure 5-10 on page 87 includes several red peaks that show where the batch transaction's elapsed time significantly exceeds the expected internal application execution time. These red peaks are important indicators of potential performance issues or inefficiencies in batch processing and can require further investigation and optimization.

When you have concerns about the elapsed time for a particular month, you can refer to the monthly elapsed-time report for batch jobs, shown in Figure 5-10 on page 95. This report provides insight into the time range of batch job execution. In this example, the report highlights the elapsed time for September 2020.

You can compare the values `ELAPSED_MINUTES_EXT` and `ELAPSED_MINUTES_INT` to evaluate performance. A substantial difference between these metrics indicates a significant gap between the externally observed elapsed time and the internally expected elapsed time. Such discrepancies can signal performance anomalies that require further investigation and optimization.

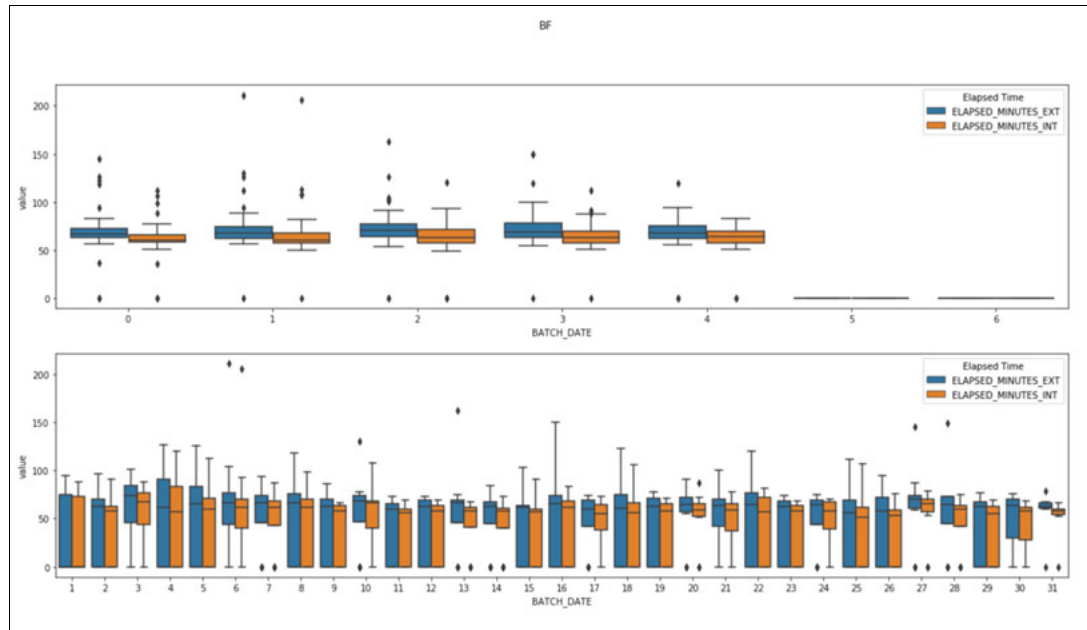


Figure 5-10 Monthly elapsed time for September 2020

Figure 5-11 on page 88 provides a comprehensive view of all batch jobs that run on a specific day. The figure shows the duration of each job by using progress bars that represent the job run time. This visual approach allows you to quickly review daily job performance and identify jobs with longer execution times or potential inefficiencies.

When you use this graphical representation, you can gain insight into job runtime patterns, optimize resource allocation, and streamline batch processing to improve overall efficiency. This approach simplifies the interpretation of complex data and helps you make informed decisions to enhance job processing and system performance.

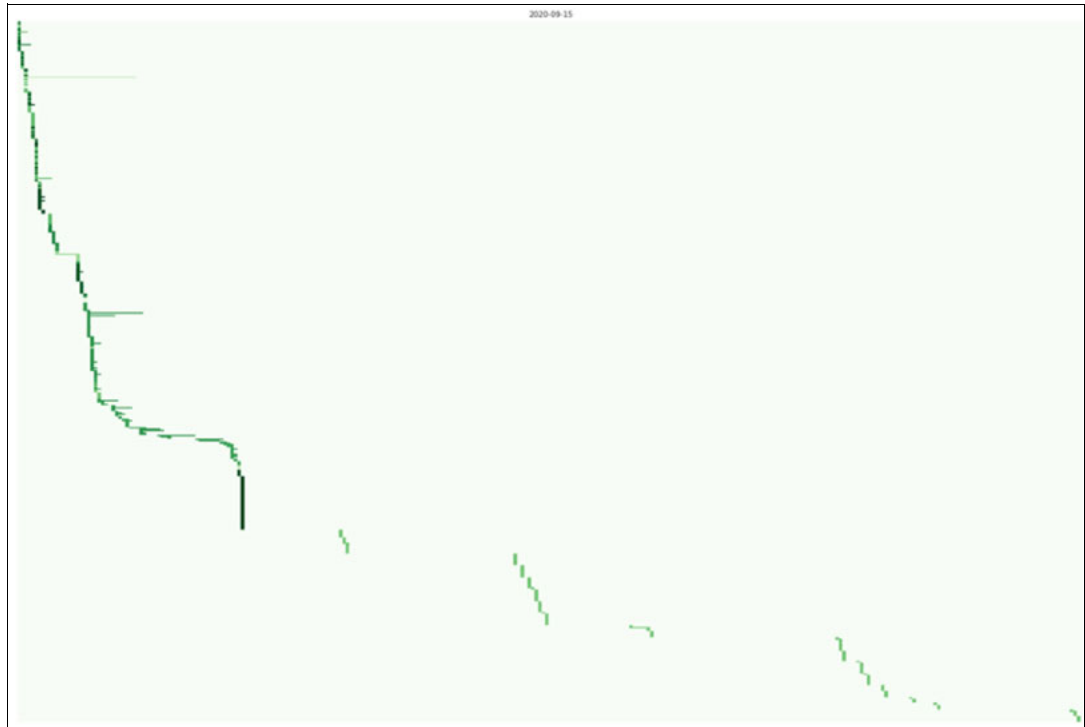


Figure 5-11 Elapsed time for 2020-09-15

## Build predictive models in a stepwise manner

Before you examine model building, it is important to understand two key concepts: influential jobs and milestone jobs.

Influential jobs are batch jobs that have a significant impact on the target variable in the predictive model. These jobs play a critical role in shaping the model's output. When you identify influential jobs, you can focus on the factors that most affect batch processing performance. Analyzing the correlations between these jobs and the target variable provides insight into their importance and overall influence on batch job performance.

An influential job has the following characteristics:

- ▶ It appears consistently in each daily batch application.
- ▶ It shows a strong statistical correlation with the batch run time and with the target variable.
- ▶ You can identify it by examining the correlations between predictors and the target.
- ▶ It includes jobs considered important from a business-logic perspective based on expert experience.

Conduct a correlation analysis between each job and the target variable, such as batch execution time. You calculate the correlation coefficient to measure the strength of the relationship between each job and the target. For this discussion, assume that the data scientist reviewed all jobs in the dataset and identified the influential jobs based on statistical analysis and expert experience.

Milestone jobs refer to a sequence of influential jobs in the batch application path. These jobs act as critical checkpoints or key events within the batch processing system. They mark significant progress or the completion of essential steps when you run a batch job. When you understand the timing and performance of milestone jobs, you can track batch processing progress and evaluate each milestone job's impact on the overall elapsed time. This insight

helps you optimize batch scheduling and resource allocation to improve system efficiency. In Figure 5-12 the milestone jobs appear as a sequence of influential jobs in the batch application.

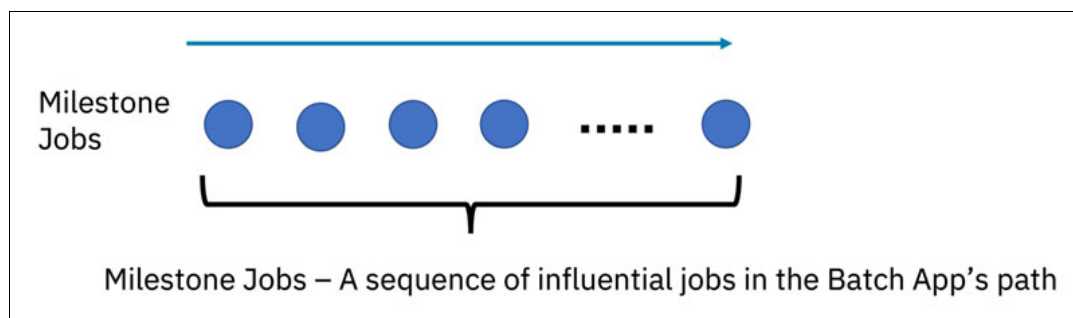


Figure 5-12 Milestone jobs - a sequence of influential jobs

In this solution, a critical task for model building is to identify the correct milestone jobs. Influential jobs within these milestones act as significant checkpoints when you run the batch application, and they generally follow a specific order of execution. Identifying the appropriate milestone jobs is essential because they provide reference points for monitoring progress and understanding dependencies within the batch application.

When you understand the sequential order of influential jobs within these milestones, you can optimize the flow of batch job execution. This understanding helps you allocate resources efficiently and streamline overall performance during data engineering and model building.

## Constructing the influential jobs chain and identifying the milestone jobs

PrefixSpan is a projection-based algorithm that recursively searches for frequent sequential patterns and extends prefix patterns in a depth-first manner. It uses a depth-first search strategy to efficiently explore the search space of potential sequential patterns. The algorithm applies a projection technique to avoid redundant computations, making it highly scalable for large datasets.

Spark PrefixSpan implements the PrefixSpan algorithm and uses Apache Spark's distributed computing capabilities. You can use Spark PrefixSpan to streamline identifying influential and milestone jobs, which helps optimize job scheduling, resource allocation, and overall system efficiency.

The following list describes how you handle the influential jobs sequence with PrefixSpan:

1. Split the entire time frame into several time windows.

You divide the dataset representing the batch application run time into smaller intervals called time windows. These windows help organize the data and make it easier to analyze influential jobs within specific periods.

2. Process the data from the last time window.

You process the data from the last time window first, then move to the previous window, which typically includes more critical z/OS batch transactions. From the last window to the previous one, the data often contains batch transactions that hold greater significance.

3. Manually define several key time points in the last window.

After you finish the last time window, repeat the same processing for the previous window. Use this iterative approach to analyze the influential job sequence and milestone jobs for each time window step by step. This method effectively captures the dynamic behavior of

the z/OS batch application over time. See Figure 5-13 for an example of key jobs in a sequence of influential jobs within a time window.

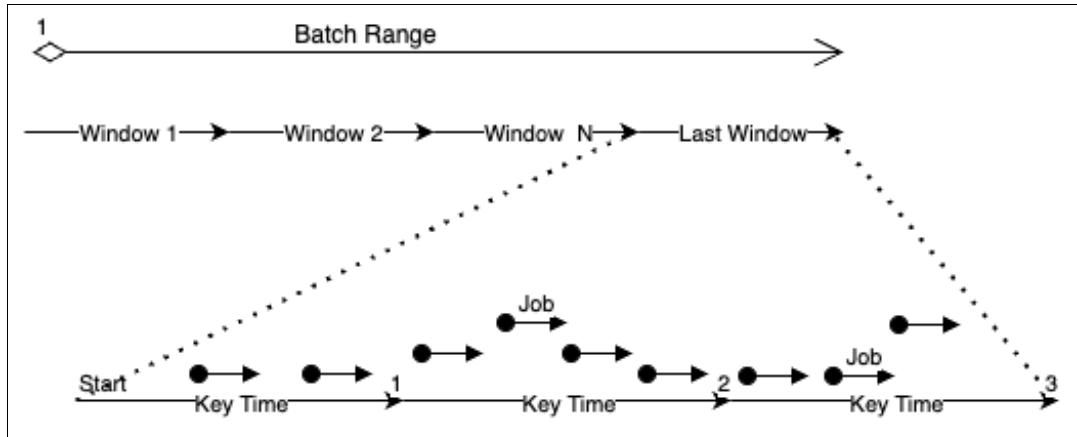


Figure 5-13 PrefixSpan for influential jobs sequence

4. Compute the score for each job at every key time by using PrefixSpan.

You use the PrefixSpan algorithm to calculate a score based on the time gap between the job's completion time and the key time. This score shows how each job aligns with a key time, indicating whether it was completed before or after the critical moment. The scoring method is shown in Example 5-4

*Example 5-4 PrefixSpan algorithm to calculate individual score for each job*

---


$$\text{Score} = \log(\text{execution\_time}) * \frac{\text{abs}(\text{completion\_time} - \text{key\_time})}{\text{time window}}$$


---

5. Compute the average score for each job at every key time.

After you calculate individual scores for each job at the key times, compute the average score for each job across all key times. To calculate the average score for a specific job, add its scores at each key time and divide the sum by the total number of key times. This process gives you an overall measure of the job's alignment with critical events throughout the batch application's execution.

6. Select the job with the minimum score across all jobs.

From the computed average scores, you identify the job with the minimum score. This job is the most critical contributor to the timely completion of the batch application because it consistently aligns well with the key times.

7. Add all key jobs in the previous time window as dependencies.

After you identify the job with the minimum score, include all key jobs from the previous time window as dependencies. These key jobs serve as reference points for tracking progress and dependencies within the batch application. These jobs are also called influential jobs.

You then establish a meaningful chain of influential jobs with their dependencies, which represents a significant sequence of batch job execution. At this stage, you identify milestone jobs. Identifying milestone jobs is a crucial step toward improving batch processing outcomes and meeting business objectives efficiently.



## Model building in a stepwise manner

A stepwise manner in machine learning offers numerous benefits, including improved model performance, resource efficiency, interpretability, and flexibility. By breaking down the model-building process into manageable steps, it helps ensure a more effective and streamlined development process, leading to accurate and reliable machine learning models. IBM z/OS batch applications can be highly complex, involving numerous interconnected jobs with dependencies. By using a stepwise approach, you can break down this complexity into smaller, more manageable tasks, making it easier to understand, develop, and optimize the application.

Figure 5-14 shows multiple milestone phases, each representing a distinct segment of the z/OS batch application's execution. Within each milestone phase is a set of influential jobs that can significantly impact the overall performance of the batch application. To capture these effects accurately, train a separate predictive model for each milestone phase.

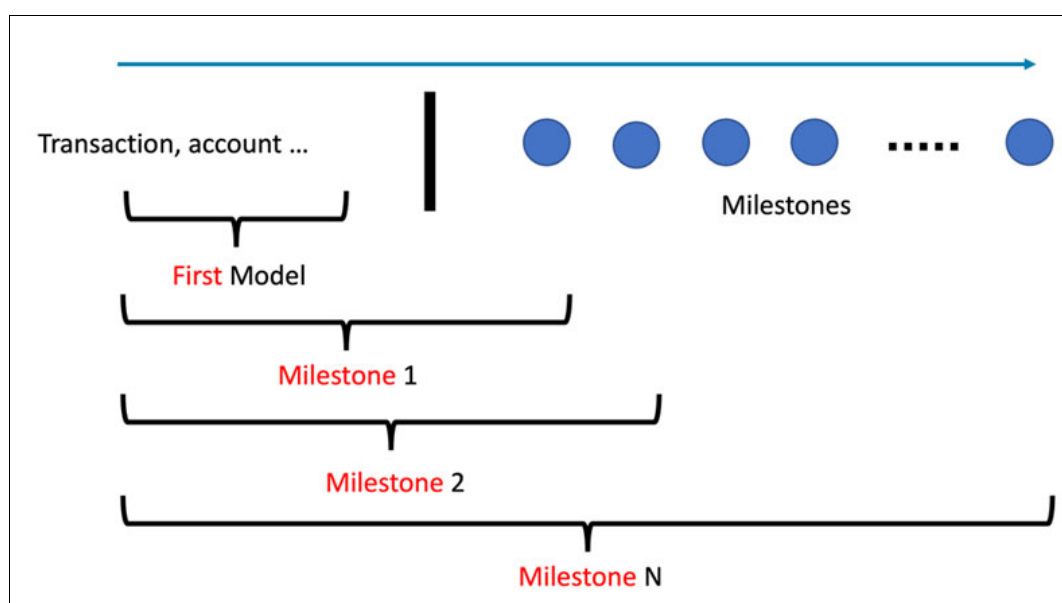


Figure 5-14 Milestones for a batch application

By training individual models for each milestone phase, you can obtain a more granular understanding of the influential jobs' execution patterns within that specific time frame. These models reflect the dynamic nature of the influential jobs and their impact on the batch application's elapsed time during each milestone phase.

This stepwise approach ensures that the predictive models are tailored to capture the unique characteristics and dependencies of influential jobs within each milestone phase. You can optimize the models for specific time frames, which can lead to more accurate predictions and better performance throughout the batch application's execution.

Additionally, the milestone modeling process can be seamlessly integrated with users' job logs and daily z/OS batch transaction data. As shown in Figure 5-15 on page 92, you can use the job log data to train predictive models for each milestone. These models are designed to capture the elapsed time patterns of influential jobs within the milestone phases.

After training milestone models, you can use the near real-time job log data to score the elapsed time for the ongoing batch application execution. By using milestone models to predict elapsed time, you can obtain accurate and up-to-date estimations of job completion times during the execution process. Integrating the job log data and z/OS batch transaction

data with the milestone modeling enhances the precision and reliability of the predictive models. Job logs provide detailed information about job execution, errors, and resource usage, so you can fine-tune the models based on actual performance data.

By scoring elapsed time in near real-time with the milestone models, you can monitor the progress of the batch application and identify any deviations from expected job completion times. You can take prompt actions in case of delays or issues, ensuring that the batch application remains on track and meets its performance goals.

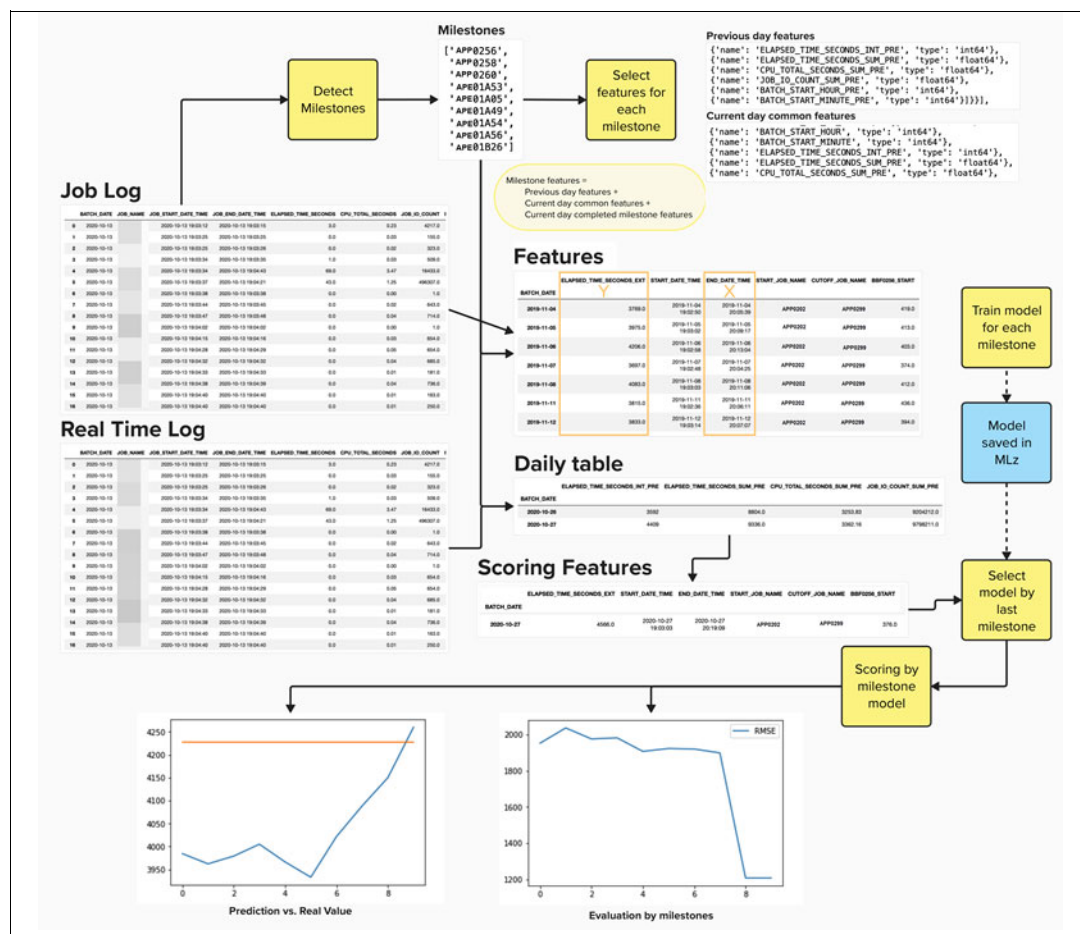


Figure 5-15 Batch jobs elapsed time prediction process

## 5.5.4 Applying batch job elapsed time model with MLz for application integration

After you successfully train and save predictive models, the next step is to deploy and implement them in real-world scenarios.

You use MLz model deployment and scoring capabilities to integrate predictive models into existing z/OS batch application workflows. With this integration, you apply predictive insights from the models in real time, improving the efficiency and performance of batch processing.

As the z/OS batch application evolves, milestone models undergo iterative updates and versioning, which MLz conveniently manages in its model metadata repository. You use the versioning feature to track iterations and improvements to the models over time. This feature ensures that you can access previous versions for comparison and reference when needed.

When milestone models are ready for near real-time log prediction, you deploy and score them using the MLz scoring engine. By integrating scoring into the workflow, you apply the models' predictive capabilities during the actual execution of z/OS batch applications. By scoring near real-time logs with these milestone models, you continuously receive updated predictions of influential job elapsed times. You use these data-driven insights to make adaptive decisions and optimize batch processing performance.

## Batch job model with MLz model management

Figure 5-16 describes the generation of several milestone models, each capturing a specific phase of z/OS batch application execution. These milestone models play a critical role in predicting the elapsed time of influential jobs within their respective phases. To ensure seamless application integration, these models must be persisted for future prediction.

By saving the trained models into the MLz metadata repository, you gain several advantages. One key advantage is model versioning. The versioning feature ensures that you can always access previous versions of the model, if needed. It provides a complete history of model development and maintains a record of how the model has evolved over different stages of refinement.

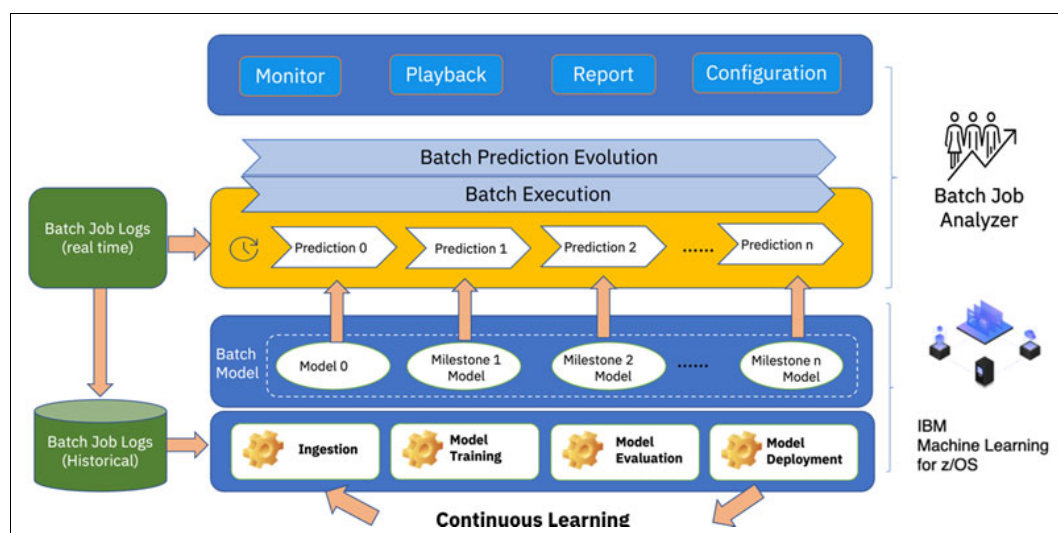


Figure 5-16 Batch jobs elapsed time use case integration with MLz

When you save models in the MLz metadata repository, you make it easier to reload them and access them for scoring. The MLz scoring engine can seamlessly retrieve stored models, allowing you to perform near real-time scoring for new batch application executions. This capability ensures that you use the latest model insights for accurate and up-to-date predictions, improving the efficiency and effectiveness of z/OS batch applications.

After you complete model training, you use the MLz interface to specify the model, training data, and target. After you finish this step, you find the model listed in the MLz model repository. Example 5-5 shows how to save a model by using the MLz model management API.

### Example 5-5 Batch model with MLz model management API example

```
def save_model(self, model, model_name, train_x, train_y):
 ml_client = MLRepositoryClient(self.core_service_url)
 ml_client.authorize_with_token(self.access_token)
 props1 = MetaProps({
 MetaNames.AUTHOR_NAME: self.MLz_info.user_name,
```

```

 MetaNames.AUTHOR_EMAIL: "batchprediction@ibm.com"}})
input_artifact = MLRepositoryArtifact(model, name=model_name,
 meta_props=props1,
 training_data=train_x.head(),
 training_target=train_y)
saved_artifact = ml_client.models.save(artifact=input_artifact)
dict_meta = saved_artifact.meta.get()
model_version_url = dict_meta['modelVersionUrl']
print("model_version_url = " + model_version_url)

```

---

## Batch job model prediction with MLz scoring engine

The batch job prediction process with the MLz scoring engine uses predictive models developed for elapsed time estimation. By deploying and scoring these models during z/OS batch execution, you gain near real-time insights that support adaptive decisions and performance optimization.

Figure 5-17 on page 95 shows multiple scoring requests based on milestones. You can dynamically review predicted elapsed times for specific milestone jobs. This near real-time prediction and review process plays a crucial role in improving the efficiency and performance of z/OS batch applications.

As the application progresses through milestone phases, you use milestone models to continuously predict elapsed times for influential jobs within each phase. The system generates these predictions as the batch runs, providing timely insights into the expected performance of critical jobs.

You send scoring requests to the MLz scoring engine to access the latest predictive capabilities and obtain near real-time updates for each milestone job. By comparing these predictions with actual performance during run time, you identify discrepancies and potential issues early.

Example 5-6 illustrates how you package a scoring request and invoke the MLz scoring engine to infer batch job elapsed time during run time.

### *Example 5-6 Batch model with MLz model scoring API example*

---

```

test_data =
joblog_df[(joblog_df[batch_config.batch_date_col].isin(['2020-10-26','2020-10-27'])
)]
score_url =
'https://MLz_scoring_ip:MLz_scoring_port/iml/v2/scoring/online/model_deployment_id
'MLzClient = MLz_Scoring_Client(scoring_endpoint=score_url)
res = MLzClient.scoring(t_w_df)

```

---

Figure 5-17 on page 95 shows prediction results for a specific time point around 20:17. The progress bar in the first line indicates a green status, which suggests that the predicted finish time for the entire batch application is expected to fall within the normal range based on the current milestone job APE01A05.

Although job APE01A41 is marked red and appears to exceed its expected endpoint, the overall batch application is not significantly impacted. The green progress bar indicates that execution is progressing smoothly and is expected to complete within the preferred timeframe.

You use the predictive capabilities of milestone models to anticipate elapsed times for influential jobs at different points in the batch execution. By comparing near real-time performance of specific jobs, such as APE01A41 in this case, with predictions, you identify deviations and potential performance issues and respond proactively.

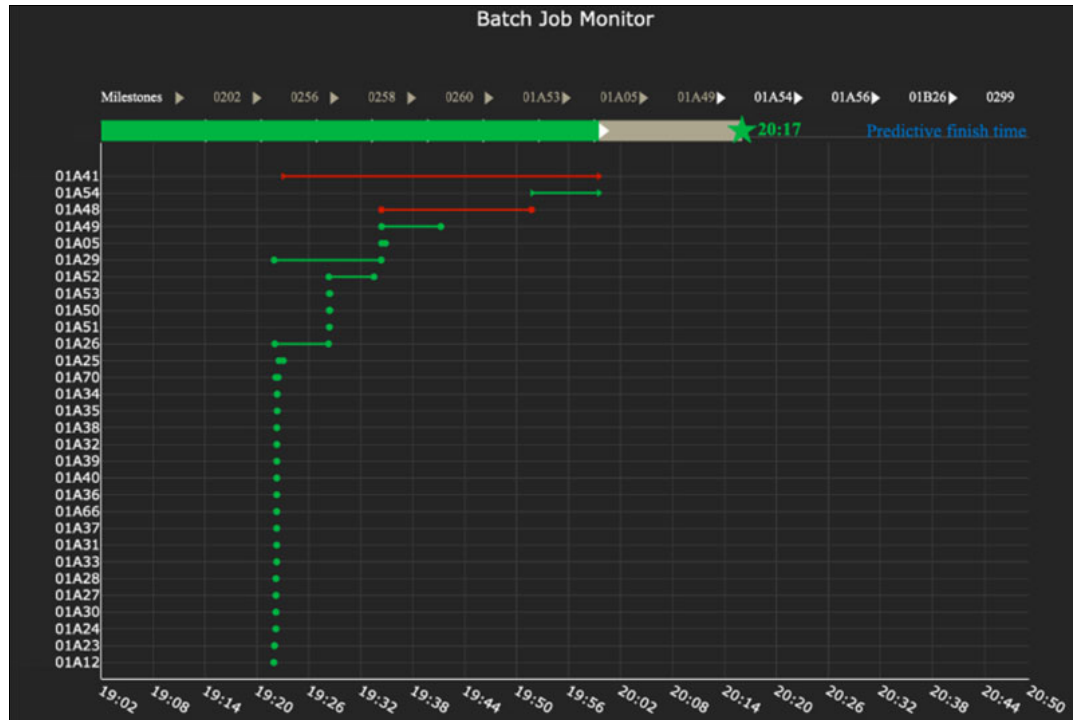


Figure 5-17 Batch job elapsed time prediction for normal case

Figure 5-18 on page 96 shows batch job prediction results for a specific time point around 21:33. The progress bar in the first line is red, which indicates that the predicted finish time for the entire batch application is likely outside the normal range based on the current milestone job APP0260.

The red status suggests that the batch execution might not meet the desired completion time. Milestone models predict a longer elapsed time for influential jobs within the current phase, which indicates potential performance issues affecting the overall batch application.

Job APE01A41 is also marked red, confirming that this job is experiencing a performance bottleneck or delay that impacts overall batch processing. The deviation from the predicted elapsed time for APE01A41 shows that you need to investigate and address the root cause of this issue promptly.

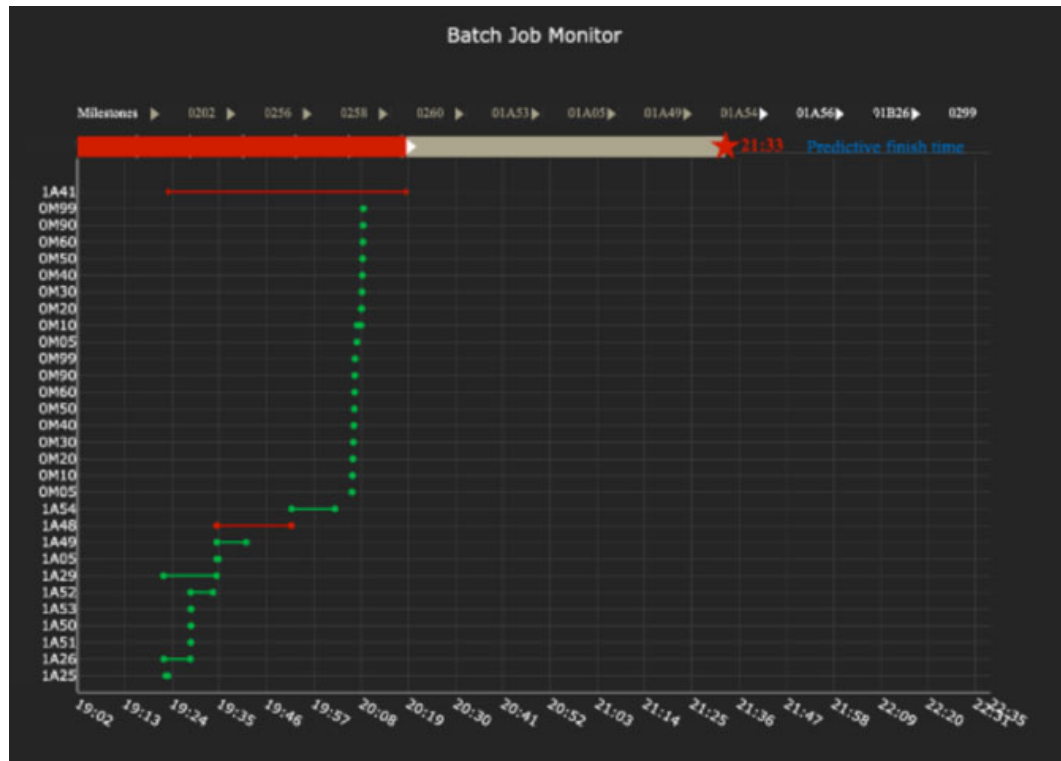


Figure 5-18 Batch job elapsed time prediction for error case

In conclusion, you use real-time reviews of batch job predictions from milestone models to gain valuable insights into the performance of individual jobs and the overall batch application. By identifying and addressing issues promptly, you take proactive steps to optimize batch processing and meet critical business deadlines.

The MLz Repository Service and Scoring Service are essential for integrating a batch prediction solution into your z/OS environment. These services enable you to manage and use predictive models efficiently, ensuring optimal performance and timely completion of critical batch jobs.

You use the MLz Repository Service and Scoring Service to fully integrate predictive analytics into your z/OS batch processing environment. These services allow you to deploy, manage, and score predictive models effectively, providing insights into batch job elapsed times and empowering you to optimize the performance of critical batch applications.

## 5.6 Best practice on model deployment and inference

The deployment and inference phase is where your machine learning models move from theory to real-world application. This phase is critical because it directly affects the performance, scalability, and reliability of your applications.

To ensure success, you follow best practices for deploying and scoring predictive models in your z/OS environment. These practices include using robust deployment pipelines, monitoring inference performance, and leveraging services such as the MLz Repository Service and Scoring Service for seamless integration. By applying these best practices, you maintain model accuracy, optimize resource usage, and deliver consistent results for critical batch processing workloads.

## 5.6.1 Optimizing ML model integration and deployment for business decisions

When you integrate and deploy machine learning models, one major challenge is bridging the gap between the training environment and the production setting, especially when AI is used to optimize business decisions. Model selection is critical because you need a balance between accuracy and inference speed.

You have access to various machine learning and deep learning frameworks, such as TensorFlow, PyTorch, and scikit-learn, for model development. Production environments often include diverse programming languages and platforms. In traditional sectors such as banking and health care, rebuilding the entire infrastructure is impractical. Instead, you convert framework-specific models into exchangeable formats to ensure interoperability between training and production environments.

Two widely adopted formats are Predictive Model Markup Language (PMML) and Open Neural Network Exchange (ONNX). PMML, an XML-based format, supports frameworks such as scikit-learn and SAS. ONNX is a versatile standard for deep learning models that enables portability across frameworks and platforms. Popular frameworks like TensorFlow and PyTorch support direct exporting to ONNX, which simplifies deployment and decouples model development from production.

To improve inference speed, you apply techniques such as model quantization, pruning, and compression to reduce model size. ONNX further streamlines inference in a unified environment, allowing data scientists to use their skills across both development and deployment processes.

Model accuracy depends not only on training iterations but also on the quality and volume of training data. Poor data can lead to misinterpreted trends, bias, and unreliable predictions. Your training dataset must be large enough and carefully curated—accurate, consistent, and rich in meaningful information.

## 5.6.2 Choosing the right inferencing interface for your use case

The MLz scoring server offers a range of programming interfaces designed for developers working on the z/OS platform. These interfaces include options like REST, Java, CICS, and WOLA. When selecting the most suitable inferencing interface for your specific needs, consider certain factors. The decision should be based on your main priorities and concerns. For instance, if you value quick response times and high throughputs, opting for the CICS or WOLA transactional inferencing interface would be beneficial.

Additionally, your choice depends on the nature of your application. If your application is built by using Python or Java, the REST APIs are a convenient option. However, if your application is developed by using an IBM Z native language, such as COBOL, using REST APIs might not be the best fit. Instead, choosing a CICS or WOLA interface is a better option.

Security requirements also play a role in your decision-making. If your IBM Z system has specific security needs, using REST APIs might introduce network-related risks that you need to be cautious about.

Making the right choice among these inferencing interfaces involves a careful evaluation of your priorities, application type, performance requirements, and security concerns, ensuring that you select an interface that aligns seamlessly with your overall objectives and technical landscape.







# Streamlining AI from models to applications by using machine learning operations

This chapter describes the art and science of seamlessly moving from machine learning models to real-world applications. Machine learning operations (MLOps) is the catalyst that combines data science, engineering, and deployment. By using MLOps, efficiency, scalability, and ethical considerations converge to not only enhance the capabilities of your models but to redefine how AI makes a tangible impact on the world. This chapter is your guide to navigating the intricate path from models to applications, where MLOps is the compass that can guide you toward operational excellence in the realm of artificial intelligence.

- ▶ 6.1, “Understanding MLOps” on page 100
- ▶ 6.2, “Model development - model training and tuning” on page 104
- ▶ 6.3, “Model review and deployment” on page 106
- ▶ 6.4, “Model monitoring” on page 112
- ▶ 6.5, “AI Governance and security” on page 120
- ▶ 6.6, “How generative AI is evolving MLOps” on page 121

## 6.1 Understanding MLOps

Machine learning operations (MLOps) represent streamlined and automated machine learning, data engineering, and DevOps practices throughout the entire lifecycle of machine learning (ML) models.

### 6.1.1 Colocation of applications and data

One of the key value propositions of AI on IBM Z is having your data and core applications colocated. Previous sections described how the software development process evolved to keep up with the increasing changes in hardware and compute capacity, starting with waterfall and agile-DevOps.

Machine learning is not a stand-alone piece of code or block that can exist independently. It requires training data, an ongoing flow of new data, an application with which the ML model is colocated and is inferencing. With new data, training and inferencing continue to evolve and change. These changes affect the quality and accuracy of the model. ML teams actively monitor the model for indicators that confirm model degradation. Those models must be updated, possibly with new training. The updated models also require a deployment of the new version of the model.

In a workflow where the development process is agile, the process can break easily without a robust and scalable way to integrate the changes in the model with the rest of the components. By using development operations or DevOps, IT teams can apply a systematic, recorded, and repeatable process to streamline and continuously monitor the integration and deployment of their application changes with the overall infrastructure. At an enterprise level, without a robust and scalable process, significant business and technical challenges affect the relevant teams and the organization as a whole.

MLOps combines machine learning with software development and overall DevOps to deliver machine learning models into production at scale. With MLOps, it is imperative to bring the different domains and personas together such as the line of business (Business analysts), data (data engineers, data science), and DevOps (ML Engineer, Application Developer, System programmers, Infrastructure engineers).

### 6.1.2 Optimizing MLOps with IBM Machine Learning for z/OS

IBM Z allows building a modern, scalable infrastructure designed for transactional workloads. IBM Machine Learning for z/OS (MLz) enables deploying AI near transactional workloads and their originating data for quicker business insights through faster implementation of AI models.

You can integrate a modern and robust MLOps process with MLz and the overall IBM Z infrastructure. MLOps is a key requirement for clients to implement AI on IBM Z. Figure 6-1 on page 101 illustrates an end-to-end MLOps pipeline with IBM Machine Learning on IBM Z.

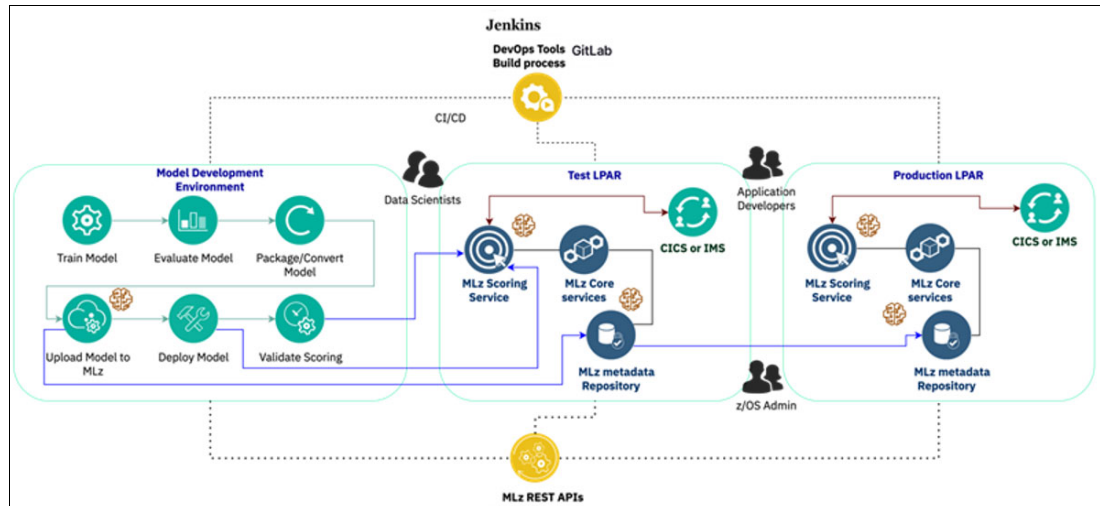


Figure 6-1 End-to-end MLOps pipeline for Machine Learning for IBM z/OS

MLz supports the MLOps process through RESTful interfaces with the comprehensive set of APIs provided to support ML implementation and the end-to-end process. The end-to-end process starts with data pre-processing, model training, model management, and model deployment and includes model scoring and continuous model monitoring.

MLz v3.1 includes a further optimization of the APIs with API Refine, user role and privilege control, and updates to API documentation and examples. For clients, this means that if they use MLz APIs, they can integrate MLz into their existing Continuous Integration and Continuous Delivery (CI/CD) pipelines. The details of the MLz, CI/CD pipeline integration process are described in this chapter.

Usually, the machine learning lifecycle architecture consists of the following three stages:

1. Development stage
2. Pre-production stage
3. Production stage

The lifecycle stages include receiving code updates, training, deploying, and monitoring models.

## Development stage

As illustrated in Figure 6-2 on page 102, an orchestrated development pipeline typically includes the following steps:

1. Data connection and validation
2. Data preparation
3. Model training and evaluation
4. Model deployment
5. Model validation (optional)

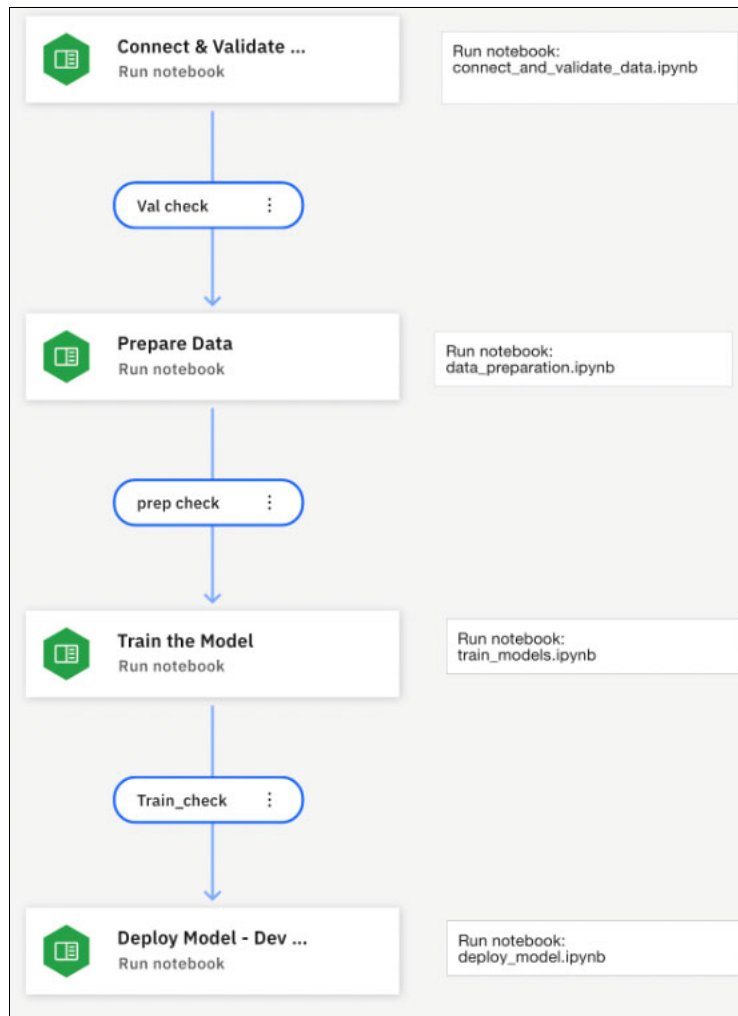


Figure 6-2 Sample development pipeline pulled from IBM Cloud Pak for Data

The output scripts tend to be Python scripts in Jupyter Notebooks. They are version controlled by using Git and serve as one of the components in the pre-production pipeline.

### Pre-production stage

When there is a change committed to the Jupyter Notebooks and a pull request is made, Jenkins starts the integration tests. As the integration tests pass, administrators merge the changes and Jenkins triggers the pre-production pipeline (see Figure 6-3 on page 103) that generally includes the following steps:

1. Data extraction and data validation
2. Data preparation
3. Model training and model evaluation
4. Model deployment (pre-production space)
5. Model monitoring and model validation

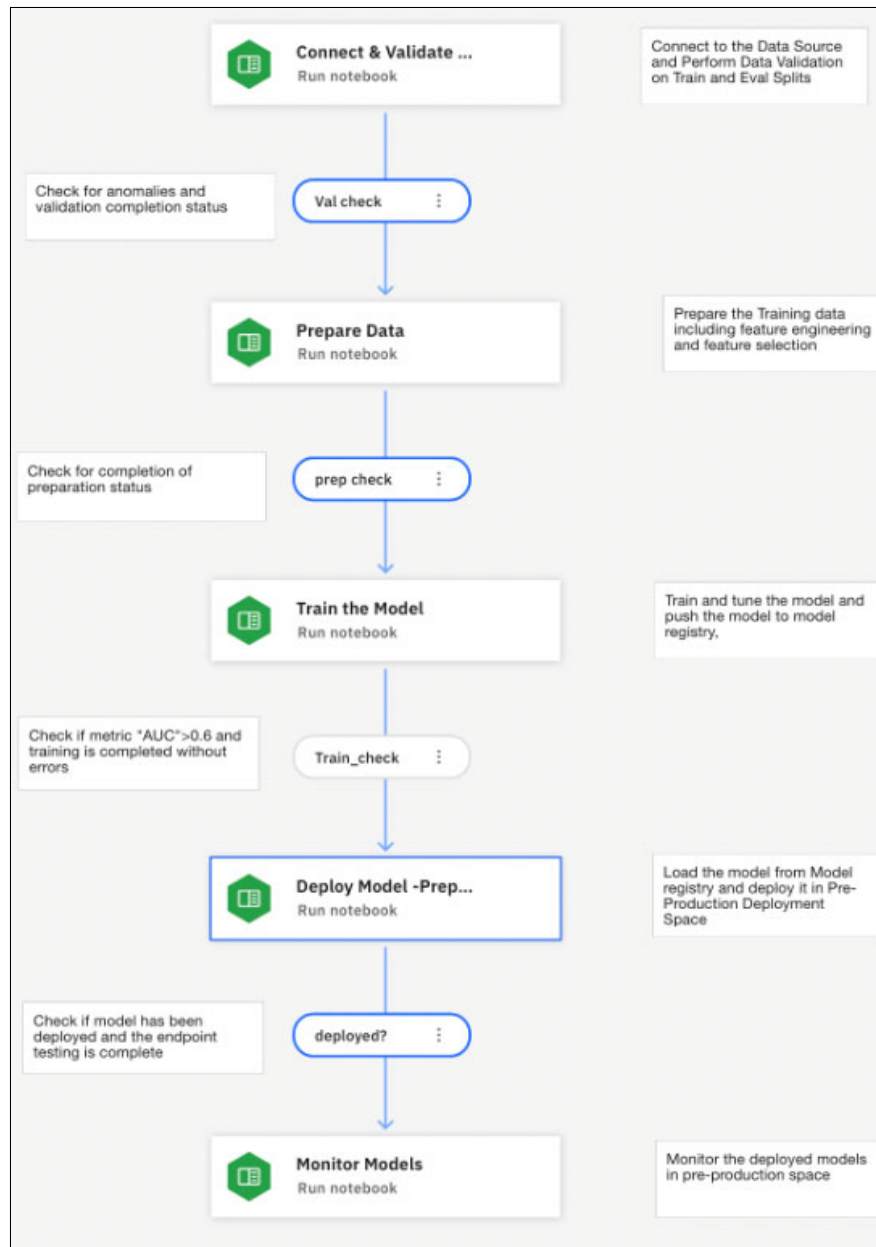


Figure 6-3 Sample pre-production pipeline pulled from IBM Cloud Pak for Data

## Production stage

An orchestrated production pipeline, as illustrated in Figure 6-4 on page 104, typically contains the following steps:

1. Deployment checks
2. Model deployment (production space)
3. Model monitoring
4. Model retraining

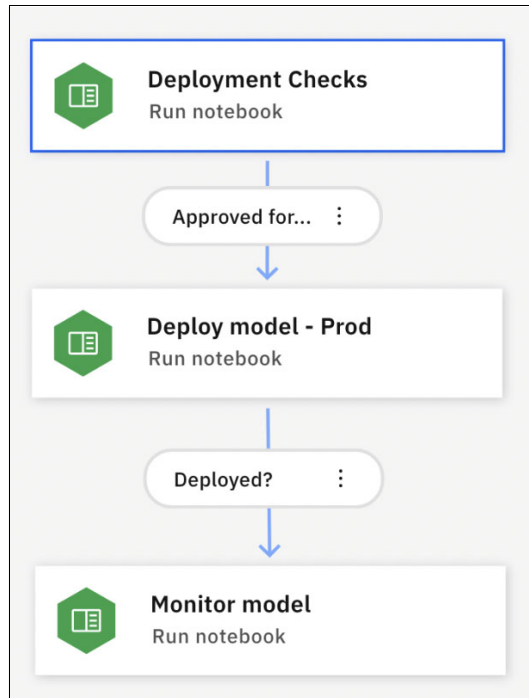


Figure 6-4 Sample production pipeline pulled from IBM Cloud Pak for Data

Notice that most of the steps throughout the three stages of development, pre-production, and production are the same, despite falling under three distinct pipelines. The steps are further discussed in the following sections.

## 6.2 Model development - model training and tuning

Model training and tuning constitute the foundational phase of the machine learning lifecycle, where data and algorithms combine to create predictive models that can make informed decisions on new, unseen data. This phase involves a series of iterative steps that encompass data preprocessing, algorithm selection, training, validation, and fine-tuning. A high-level flow of this phase is depicted in Figure 6-5 on page 105.

### 6.2.1 Data preprocessing

Data preprocessing is the process of data mining where data is treated as raw products. Various processes need to be applied to raw data to convert it to a more refined data. These various processes include data cleansing, transforming, and data conversion, to name a few. Chapter 3, “The art of data engineering” on page 21 discusses data preprocessing in detail, what those steps are, and how they come together.

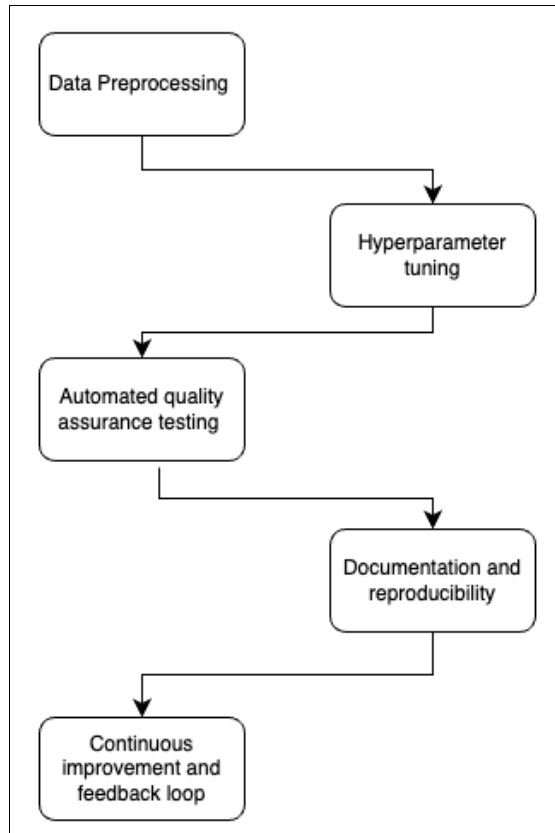


Figure 6-5 Flow for model training and tuning

## 6.2.2 Hyper parameter tuning

The model training process used multiple datasets for the model. Each dataset and model needs a different set of hyper parameters to find the correct parameter to be used by the model. Numerous iterations of model tests are done with a variety of parameters. Every parameter needs to run through the model end-to-end. This entire process is called hyper parameter tuning and runs simultaneously with the model. This hyper parameter tuning process is a combination of manual and automated process.

Hyper parameters are the combination of external parameters, an external variable for the model. An example of a hyper parameter is the number of nodes and layers in the neural network and decision branches.

Hyper parameter selection is an important and essential process and is critical for model performance. Hyper parameter selection is a trial-and-error process because each model is different, and only one set of rules can be applied at a time, so multiple attempts are required.

## 6.2.3 Automated quality assurance testing

Model training and tuning repetitive process. The model must be tuned frequently, and retraining is in case source data changes. The model might be seeing data that the model has not seen before. New data might improve the model, so the model must be retrained and tuned repeatedly. It is quite a repetitive process, but most steps can be automated. Automation can produce reliable results without introducing human error.

The automated test process is part of MLOps, where data and model output are monitored and based on the model output. Automated retraining and tuning processes can be triggered based on the model's accuracy. When the model is retrained and tuned, the automated testing process can be streamlined, and the entire model is validated with reality and without any human intervention.

#### **6.2.4 Documentation and reproducibility**

The trustworthy ML model has four major pillars: Explainability, Regulation, Integrity, and Reproducibility. Reproducibility is very important for the AL/ML model. Reproducibility is the process of producing ML model output by following the same workflow and giving the same output. Every run should produce a similar result every time. Reproducibility also has some challenges, such as changes in the dataset, no proper logging, changes in hyper parameters, and changes in the model environment. Document the entire model workflow with evidence for explainability. Ensure that the model output explains how it reached the model output.

#### **6.2.5 Continuous improvement and feedback loop**

After the model is deployed to the production environment, it must be monitored continuously. Various factors, such as changes in input data, runtime environment changes, and stale data, can impact model output. Model feedback should trigger the model retraining and tuning process.

The next section talks in detail about MLOps under the model review and deployment section. ML ops can monitor the model regularly with feedback methods. MLOps can monitor the model output and ensure that it produces reliable output.

### **6.3 Model review and deployment**

The step of model assessment and deployment emerges as the pivot that bridges the gap between model development and the tangible impact of MLOps as illustrated in Figure 6-6 on page 107. This critical stage marks the transition of complex algorithms from conceptual wonders to practical realities, ensuring that they not only perform well, but also seamlessly align with organizational goals.

The model review and deployment steps are a synchronization of examination and validation. It involves an in-depth review of the model's architecture, code integrity, and generalization abilities. This comprehensive review ensures that the model's findings are not limited to the laboratory and can be applied in relevant, real-world applications.

This section outlines approaches for deploying accurate, ethical, and safe models that align with business needs during the MLOps journey. It emphasizes the importance of automation for deployment consistency and describes a range of deployment strategies from microservices to containers and the factors that influence their selection.



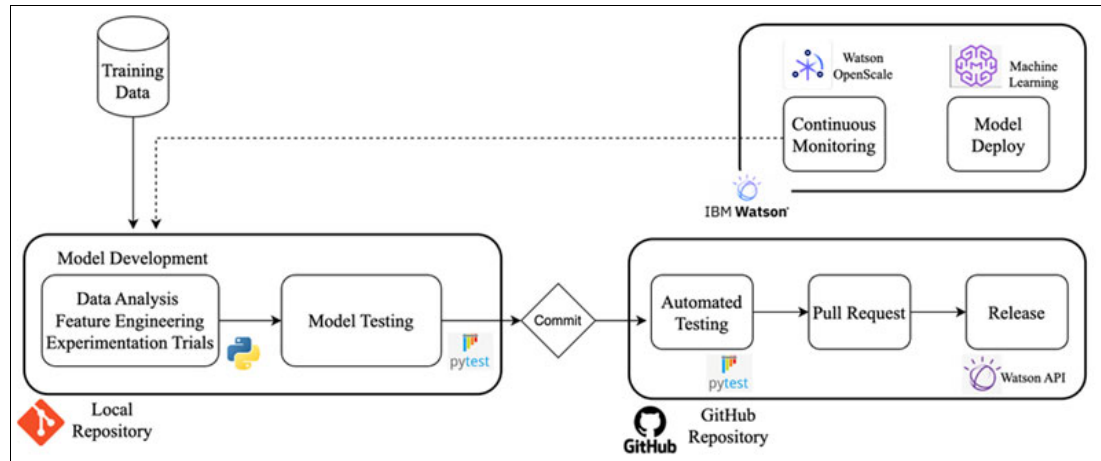


Figure 6-6 Model development lifecycle

### 6.3.1 Model review and validation

In the journey from model development to real-world application, the phase of model review and validation plays a crucial role in ensuring that the models not only work but also work well. You can think of this phase as the thorough inspection before launching a rocket ship. The inspection is focused on thoroughly checking every part of the aircraft to make sure that it is ready.

To adequately prepare a model from development to real-world application, the model review and validation phase iterates through the following processes:

- ▶ **Code review and validation.** The model's code is comparable to the blueprint of a complex structure. During code review, teams collaborate to carefully examine the code for any mistakes or problems. It is like going through a checklist to ensure that everything is in the right place. This step helps detect errors early and can help verify that the model's foundations are solid.
- ▶ **Unit testing.** Unit testing involves running small tests on different parts of the model's code to ensure that they work as expected. If any piece does not work as expected, it is fixed before it becomes a part of the whole. This ensures that each component functions properly.
- ▶ **Model validation.** Validating a model is like performing a trial run. The model is tested on new data to determine whether it can make accurate predictions. This process ensures that the model can handle new situations and is not just memorizing the previous inputs. Validating also helps identify if there is any bias or unfairness in the model's decisions, ensuring it treats everything fairly.
- ▶ **Addressing bias.** Models can sometimes unintentionally make unfair decisions based on historical data. Addressing bias means actively working to make sure the model's predictions are fair. This is a vital step in ensuring that the model does not perpetuate or amplify existing inequalities.

In summary, the model review and validation phase act as the quality control checkpoint, helping to guarantee that the model is robust, unbiased, and ready for deployment. A critical step to ensuring the model's performance aligns with real-world expectations and ethical standards before it enters the operational stage.

## 6.3.2 Model versioning

Model versioning is a critical practice within the realm of MLOps that involves tracking different iterations, or versions, of a machine learning model. Software developers use version control systems to manage code changes. Model versioning allows data scientists and MLOps engineers to systematically track and manage changes that are made to a model's architecture, code, and configuration over time.

The following terms and their descriptions are provided from the context of model versioning:

### Tracking progress

Each model version represents a snapshot of the model's state at a specific point in time. This helps teams track the evolution of the model from initial development to refined iterations by capturing improvements, bug fixes, and enhancements.

### Reproducibility

Model versioning ensures that a specific model version can be accurately reproduced whenever needed. This is crucial for maintaining consistency in research, development, and production environments.

### Experimentation

Data scientists often experiment with various algorithms, hyper parameters, and data pre-processing techniques. Model versioning allows them to keep a record of these experiments and the corresponding model performance.

### Collaboration

In collaborative environments, model versioning enables team members to collaborate more effectively. Team members can easily share, review, and work on different versions of the model, enhancing productivity and knowledge sharing.

### Comparative analysis

With multiple model versions available, teams can perform comparative analysis to understand how changes impact performance. This aids in making informed decisions about which version to deploy.

### Deployment and rollback

When deploying models into production, having well-managed versions simplifies the deployment process. If a new version presents unexpected issues, rolling back to a previous version is straightforward.

You can use the model management APIs to manage the models that are stored in MLz. You can use the following API to get the metadata of all the versions of a model:

```
GET /v3/ml_assets/models/{model_id}/versions
```

For more information, see [Model management APIs](#).

To implement effective model versioning, teams can use version control tools, such as Git, and combined it with specific practices tailored to machine learning. This might involve tagging model versions, documenting changes in a version control system, and integrating versioning within the MLOps pipeline.

## 6.3.3 Integrating with CI/CD pipelines

Much like DevOps, CI/CD is a set of practices and steps (Figure 6-7 on page 109) that software and application development teams use to deliver code changes with speed and reliability by using automation.

In machine learning, these stages are different than in software development. A model depends not only on the code but also on the data and the hyper parameters. Additionally, deploying a model to production is more complex than deploying software to production. For a rapid and reliable update of pipelines in production, you need a robust automated CI/CD system. With automated CI/CD pipelines, data scientists can rapidly explore new ideas around feature engineering, model architecture, and hyper parameters.

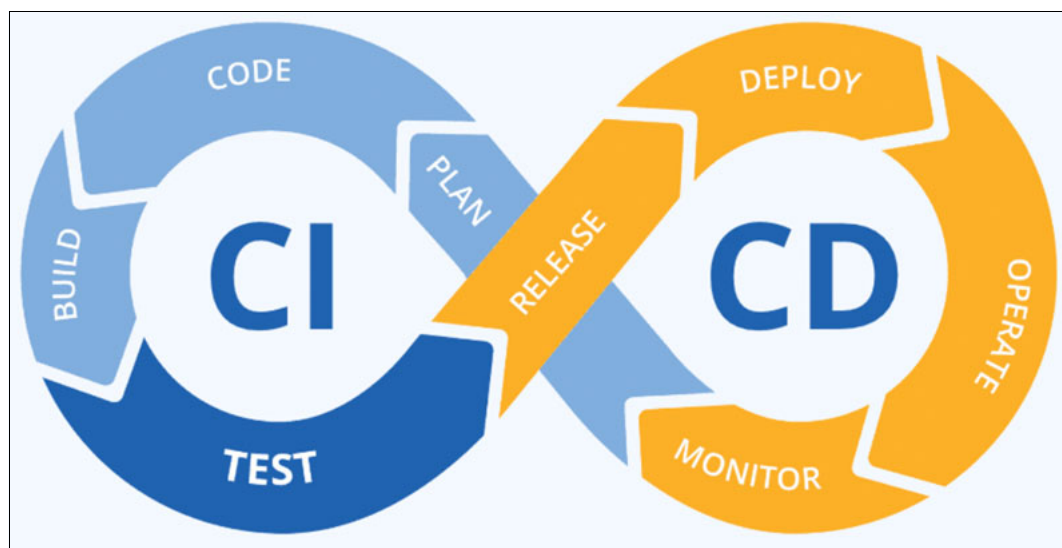


Figure 6-7 Stages of a CI/CD pipeline

### Continuous Integration (CI)

Continuous integration is a software development process where developers integrate the new code more frequently throughout the development cycle. Automated testing is done against each iteration of the build to identify integration issues earlier, when they are often easier to fix, which also helps avoid problems at the final merge for the release. Overall, continuous integration helps streamline the build process, resulting in higher-quality software and more predictable delivery schedules.

Continuous integration within the realm of machine learning implies that whenever code or data is updated, the machine learning pipeline is restarted. This process is orchestrated in a manner where versioning and reproducibility are maintained, allowing seamless codebase sharing across various projects and teams. Each pipeline rerun might encompass activities like training, testing, or generating updated reports to simplify the task of comparing against other versions in production.

The following instances illustrate the CI workflow:

- ▶ Running and versioning training and assessment for each repository commit
- ▶ Running and contrasting experimental runs for every Pull Request toward a specific branch
- ▶ Initiating new runs at regular intervals

### Continuous Deployment (CD)

Continuous deployment stands as an automated approach for rolling out new releases into production or other environments like staging. This approach facilitates swift and ongoing changes, enabling prompt user feedback and accommodating new data for model retraining or novel models.

The following instances exemplify the CD workflow:

- ▶ Ensuring that the infrastructure environment meets requirements before deployment
- ▶ Validating model outcomes by using established inputs
- ▶ Conducting load tests and evaluating model latency

## **CI/CD tools and configuration**

When you select CI/CD tools, focus on how to optimize and automate the software development process. An effective CI/CD pipeline uses open source tools for integration, testing, and deployment. Correct configuration of your CI/CD process also impacts the success of the software development pipeline.

A common open source CI/CD tool is Jenkins. Jenkins is an automated CI server that is written in Java and used for automating CI/CD steps and reporting. Other open source tools for integration include Travis CI and CircleCI.

Integrated development environments (IDE), such as GitHub or AWS CodeCommit, help developers create, maintain, and track software packages. Platforms like GitLab seek to provide the IDE within a comprehensive platform that includes other tools.

When operating in a cloud environment, teams use containers like Docker for packaging and shipping applications, and they use Kubernetes for orchestration. Although Kubernetes is not strictly for the CI/CD pipeline, it is used in many CI/CD workflows.

## **CI/CD pipeline phases**

From source code to production, the following phases make up the development lifecycle and workflow of the CI/CD pipeline:

- ▶ **Build.** This phase is part of the continuous integration process and involves the creation and compiling of code. Teams build off source code collaboratively and integrate new code while quickly determining any issues or conflicts.
- ▶ **Test.** At this stage, teams test the code. Automated tests happen in both continuous delivery and deployment. These tests might include integration tests, unit tests, and regression tests.
- ▶ **Deliver.** An approved codebase is sent to a production environment. This stage is automated in continuous deployment and is only automated in continuous delivery after developer approval.
- ▶ **Deploy.** The final product with any changes moves into production. In continuous delivery, products or code are sent to repositories and then moved into production or deployment by human approval. In continuous deployment, this step is automated.

## **6.3.4 Model deployment**

Deploying a machine learning model into a production environment requires careful consideration of various factors, including scalability, resource usage, response time, and ease of maintenance. Different deployment strategies offer unique approaches to making your model accessible and operational. Some common deployment strategies are batch processing and online deployment.

### **Batch processing**

Sometimes, it is more efficient to perform model inference on a batch of data rather than real-time processing. Batch processing involves scheduling model inference on a batch of data at specific intervals. This strategy is suitable for scenarios where low latency is not a priority, such as generating reports or analyzing historical data. Examples include insurance

premium and claims analytics, sales and inventory analysis, log analysis and anomaly detection, and credit scoring and loan approval.

An example of batch processing is discussed in section 5.3.2, “Model deployment” on page 70.

### Online deployment

An online deployment strategy, also known as real-time deployment, involves making machine learning models available for inference and predictions in real-time as new data arrives. This strategy is useful for applications where quick and immediate responses are required based on incoming data. Examples include fraud-detection, chatbots, recommendation systems, image recognition in video streams, and IoT applications.

You can use the following API to create online deployment for a model:

```
POST /v3/published_models/${modelId}/deployments
```

You can also use following API to get the detailed information of a deployment:

```
GET /v3/published_models/${modelId}/deployments/${deploymentId}
```

For more information, see [Deployment APIs](#).

When you choose a deployment strategy, consider factors such as the nature of your application, expected traffic patterns, resource requirements, and deployment complexity. Each strategy has its advantages and challenges, so selecting the one that aligns best with your project goals and constraints is crucial. Additionally, the availability of tools and platforms that support your chosen strategy can simplify the deployment process and enhance its scalability and reliability.

## 6.3.5 Model inference and scoring

Model inferencing and scoring represent the bridge between machine learning models and tangible, real-world impact. In the dynamic landscape of MLOps, this phase demands meticulous attention to responsiveness, interpretability, scalability, and security. By seamlessly integrating these considerations, organizations can harness the full potential of their models, making AI-driven decisions that are not only accurate but also ethical, secure, and aligned with business objectives.

After you create an online deployment for a model, you can access its predictions by sending requests to its scoring endpoint.

For more information, see [Scoring APIs](#).

To automate the inferencing process, you can use the sample Python functions, provided in Example 6-1, to interact with MLz through REST APIs.

*Example 6-1 Using Python functions to automate the inferencing process*

---

```
Get MLz user token
def gen_wmlz_user_token(user_name, password):
 gentoken_url = '{}:{}'.format(
 wmlz_service_host, wmlz_service_port)
 gentoken_request_header = {
 'Content-Type': 'application/json'
 }
```

```

 gentoken_request_data = json.dumps({'username': user_name, 'password':
password})
 gentoken_response = session.post(gentoken_url, gentoken_request_data,
headers=gentoken_request_header, verify=False)
 gentoken_response_dict = json.loads(gentoken_response.content)
 return gentoken_response_dict['token']

Retrieve model ID and model version ID of the uploaded model
def get_model_info(model_name):

 get_model_info_url = '{}:/{}/v3/ml_assets/models?modelName={}'.format(
 wmlz_service_host, wmlz_service_port, model_name)
 get_request_header = {
 'Authorization': service_token
 }
 get_model_info_response = session.get(get_model_info_url,
headers=get_request_header, verify=False)
 print("get model info result:" + str(get_model_info_response.content))
 get_model_info_response_dict = json.loads(get_model_info_response.content)
 model_id =
get_model_info_response_dict.get('resources')[0].get('metadata').get('guid')
 model_version_id =
get_model_info_response_dict.get('resources')[0].get('entity').get('model_version'
).get('guid')
 return model_id, model_version_id

Perform online scoring
def do_score(scoring_url, scoring_input):

 score_request_header = {"Authorization": service_token, "Content-Type":
"application/json"}
 score_response = session.post(scoring_url, headers=score_request_header,
data=scoring_input, verify=False)
 score_response_dict = json.loads(score_response.content)
 return score_response_dict

```

---

## 6.4 Model monitoring

Model monitoring is a crucial phase within the machine learning operations (MLOps) framework that focuses on the ongoing observation, analysis, and management of deployed models in real-world environments. As models move from controlled development settings to dynamic production configurations, monitoring helps guard against performance degradation, concept drift, anomalies, and other operational challenges:

- ▶ **Model performance.** The capability to assess a model's performance through a set of metrics and recording its decisions or outcomes can offer valuable directional insights. These insights can be contrasted with historical data, aiding in comparisons among different models to determine the most effective one.
- ▶ **Data and security.** Modern models rely on intricate feature pipelines and automated workflows, which involve dynamic data that is subjected to diverse transformations. Given the complexity, data inconsistencies and errors can unintentionally erode model performance over time. Additionally, models can be vulnerable to attacks through data injection and other means.

- **Explainability.** The opaque nature of models poses challenges in understanding and debugging, particularly within a production setting. The ability to understand a model's decision is crucial not only for enhancement but also for accountability, especially in sectors like finance.
- **Addressing bias.** As machine learning models glean relationships from training data, they can inadvertently magnify existing biases or introduce new ones. Detecting and mitigating bias during the development phase is intricate yet imperative.
- **Monitoring for drift.** The statistical characteristics of the target variable, which the model seeks to predict, can evolve unexpectedly over time. This phenomenon, which is known as concept drift, leads to decreasing prediction accuracy as time progresses and poses challenges that require monitoring and mitigation.

## 6.4.1 Evaluating and reevaluating a model under MLz

As your data changes over time, the quality of the predictions from a particular model might degrade. With the WML for z/OS, you can evaluate a SparkML or PMML model and schedule it for periodic reevaluations to ensure its accuracy. Evaluating models by using MLz consists of following steps:

1. Create evaluation
2. Deploy models
3. Schedule evaluation

Figure 6-8 is an example of a successful evaluation, where the Area under curve (AUC) was more than 0.75. Because the AUC is greater than random chance, the model is considered to be accurate.

| Schemas        |         |          |             |                |             |
|----------------|---------|----------|-------------|----------------|-------------|
|                |         |          | Table       | JSON           | JSON schema |
|                |         |          | Copybook    |                |             |
| Input          |         |          | Output      |                |             |
| Name           | Type    | Nullable | Name        | Type           | Nullable    |
| AGE            | integer | false    | prediction  | basic(double)  | false       |
| COUNTRY        | string  | false    | probability | tensor(double) | false       |
| CUSTOMER_ID    | integer | false    |             |                |             |
| GENDER         | string  | false    |             |                |             |
| MARITAL_STATUS | string  | false    |             |                |             |
| NATIONAL_ID    | string  | false    |             |                |             |

| Evaluation results                                                                                            |                      |           |               |                |               |
|---------------------------------------------------------------------------------------------------------------|----------------------|-----------|---------------|----------------|---------------|
| <div> <span>Next evaluation starts at Aug 30, 2023 3:45 PM</span> <span>Reschedule evaluation +</span> </div> |                      |           |               |                |               |
| Start time                                                                                                    | End time             | Status    | Records count | Area under ROC | Area under PR |
| Aug 29, 2023 3:45 PM                                                                                          | Aug 29, 2023 3:46 PM | COMPLETED | 1320          | 0.780          | —             |
| Aug 28, 2023 3:45 PM                                                                                          | Aug 28, 2023 3:46 PM | COMPLETED | 1320          | 0.780          | —             |

Figure 6-8 Evaluating a model by using MLz

For more information, see [Evaluating and reevaluating a model](#).

## 6.4.2 Monitoring deployed models

IBM Watson OpenScale offers a high-caliber setting that is designed for AI applications. It is possible to set up MLz to collaborate seamlessly with the Watson OpenScale service within an IBM Cloud Pak for Data environment. You can use it to understand how your AI models make decisions, detect and mitigate bias and drift, increase the quality and accuracy of your predictions, explain transactions, and perform what-if analysis. The capabilities of Watson

OpenScale can be used to validate MLz models employed in real-time scoring by using the REST API.

### Using IBM Watson OpenScale to monitor your deployed AI models

When the model is ready, then the data scientist pushes the model to the production environment, which is also known as the business line environment. Then, the data scientist can use Watson OpenScale to check whether the z/OS AI model is experiencing model drift or not. The people in the business, such as a loan officer or a loan customer, can use Watson OpenScale to understand the reason for the model results.

Perform the following steps to get a z/OS machine-learning model-scoring endpoint from MLz:

1. Log in to Machine Learning for IBM z/OS.
2. Click the **Deployments** tab.

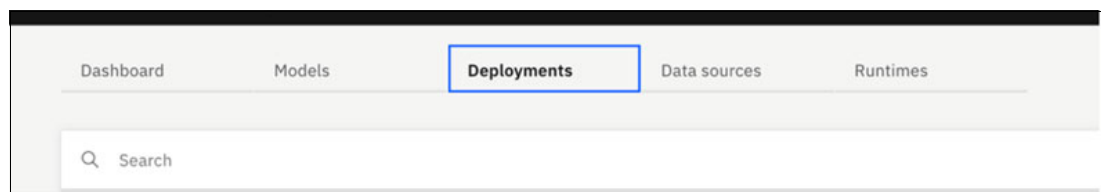


Figure 6-9 MLz model Deployments list

3. Get the scoring endpoint.

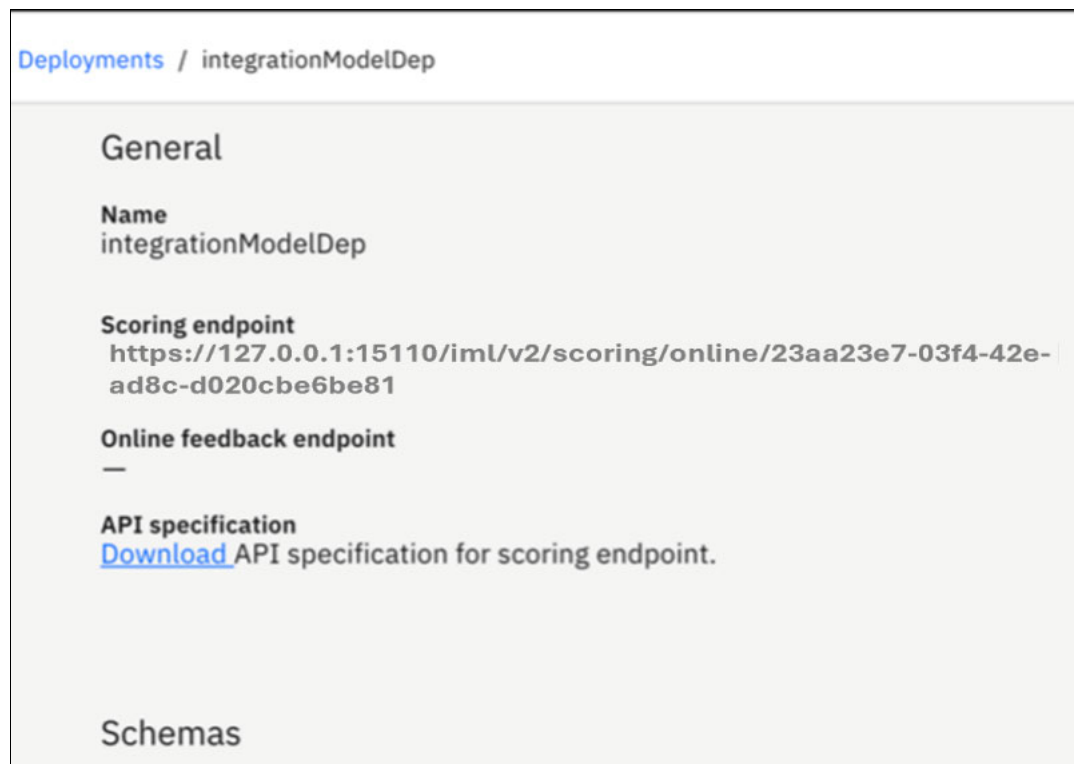


Figure 6-10 Deployed model in MLz



Perform the following steps to create a production machine learning provider in Watson OpenScale on the IBM Cloud Pak for Data platform:

1. In the Watson OpenScale System setup page, click **Add machine learning provider**. Bind Machine Learning for IBM z/OS as the machine learning provider, populating the Connection details tag with the following information:
  - Service provider: **Custom Environment**
  - Authentication type: **Basic**
  - Username: *<MLz username>*
  - Password: *<MLz user password>*
  - Environment type: **Production**
2. Click the **Save** button.

Figure 6-11 Connection details in Watson OpenScale

3. On the IBM Watson OpenScale Insight dashboard, create a subscription base for the machine learning service provider that you created, then bind the Watson OpenScale subscription to the MLz deployment:
  - a. From the Machine learning provider menu, select the correct Machine learning Provider and environment from step 2.1.
  - b. In the Endpoint field, add the MLz scoring endpoint from step 1.3.

**Note:** The endpoint should use the MLz UI port number. For example, if the scoring endpoint in the deployment is `https://*.*.*.*:15110/iml/v2/scoring/online/23aa23e7-03f4-420e-ad8c-d020cbe6be81` and base URL is `https://*.*.*.*:15101/`, then the endpoint for the Watson OpenScale provider is `https://*.*.*.*:15101/iml/v2/scoring/online/23aa23e7-03f4-420e-ad8c-d020cbe6be81`.

- c. Enter the Deployment Name.
- d. Click **Configure**.

Select a model deployment

Choose a machine learning provider and provide deployment details.

Machine learning Provider

pro provider (Production) ▼

Endpoint

https://127.0.0.1:15101/iml/v2/scoring/online/23aa23e7-03f4-

Deployment Name

ProDep

Description

Cancel Configure

Figure 6-12 Watson OpenScale model deployment details

4. Configure the Watson OpenScale monitors - Fairness, Quality, Drift, and Explain.
5. Get the OpenScale Datamart ID and Subscription ID from the Endpoints page.

Model info

Model details

Endpoints

Evaluations

Fairness

Quality

Drift

Explainability

Go to model summary

Endpoints

Description

Watson OpenScale evaluates models for fairness and drift using logged scoring requests (payloads) received by the model. Scoring requests are logged using the payload logging endpoint.

Watson OpenScale evaluates models for quality using labeled test data. Labeled test data is provided using the feedback endpoint or by file upload.

Upload feedback data

Model information

Endpoints

Deployment details

|                          |                                      |
|--------------------------|--------------------------------------|
| Name                     | wmlzProDep                           |
| Description              |                                      |
| Deployment ID            | 35b92b12-d580-4f6e-84d9-9da30b7296f7 |
| Data type                | Numeric/categorical                  |
| Algorithm type           | Binary classification                |
| Machine learning service | pro provider                         |

Integration details

|                       |                                          |
|-----------------------|------------------------------------------|
| OpenScale Datamart ID | 00000000-0000-0000-0000-1650002923916292 |
| Subscription ID       | b0396a8e-6b6f-48d4-a5ce-4a4e01528fa3     |
| Service binding ID    | a897840-0a7e-4125-8580-ea2319542e78      |

Asset details

Figure 6-13 Model information details in Watson OpenScale

Use the following steps to enable and configure Watson OpenScale on MLz

1. Enable Watson OpenScale in the MLz deployment:
  - a. Right-click the **Action icon** under IBM Machine for z/OS deployment.
  - b. Select **Configure for OpenScale**.

2. Configure Watson OpenScale with the following details:
  - a. Select **Enable**
  - b. Instance: *<CP4D UI URL>*
  - c. User: *<OpenScale user>*
  - d. Password: *<OpenScale user password>*
  - e. DataMartID: *<OpenScale Datamart ID>* (from Step 2)
  - f. Target ID: *<Subscription ID>* (from Step 2)
3. Then click **OK**.

Config Opensclae

☒ Enable ☐ Disable

Instance

https://cpd-zen.apps.openscales390x-0027.cp.fyre.ibm.com

User

wmlz01

Password

\*\*\*\*\*

DataMart ID (Required)

00000000-0000-0000-0000-1649990273177897

Target ID

f9089f90-91ee-48b7-8e37-e36307d7c3ac

Cancel OK

Figure 6-14 Configuration settings in Watson OpenScale

4. Evaluate the MLz model for Fairness, Quality, Drift, and Explain with Watson OpenScale.

After Watson OpenScale has been configured and enabled, you can load the Insights Dashboard, which contains tiles for any models being monitored (Figure 6-15 on page 118).

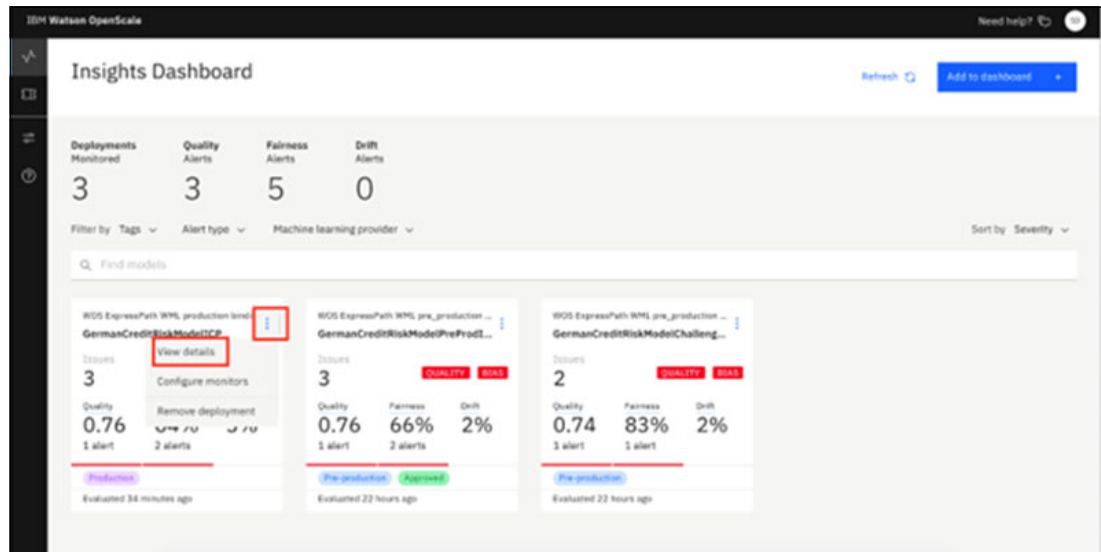


Figure 6-15 Watson OpenScale Insights Dashboard

You can then evaluate the models on the various set monitors, such as fairness, quality, and drift (see Figure 6-16).

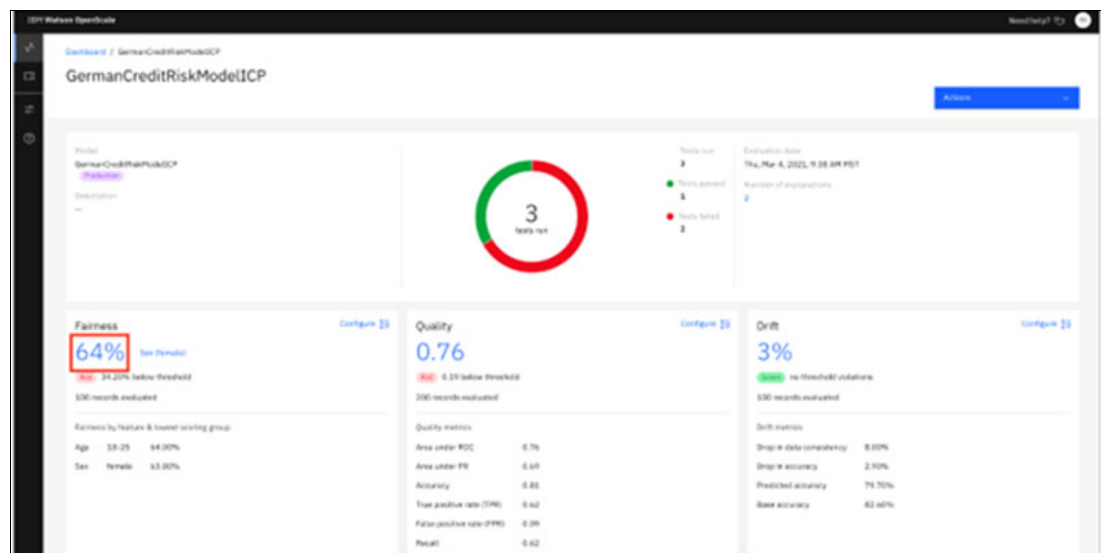


Figure 6-16 Evaluation monitors with Watson OpenScale

To understand your model's evaluation results better, you can dive deeper into each evaluation monitor (see Figure 6-17 on page 119).

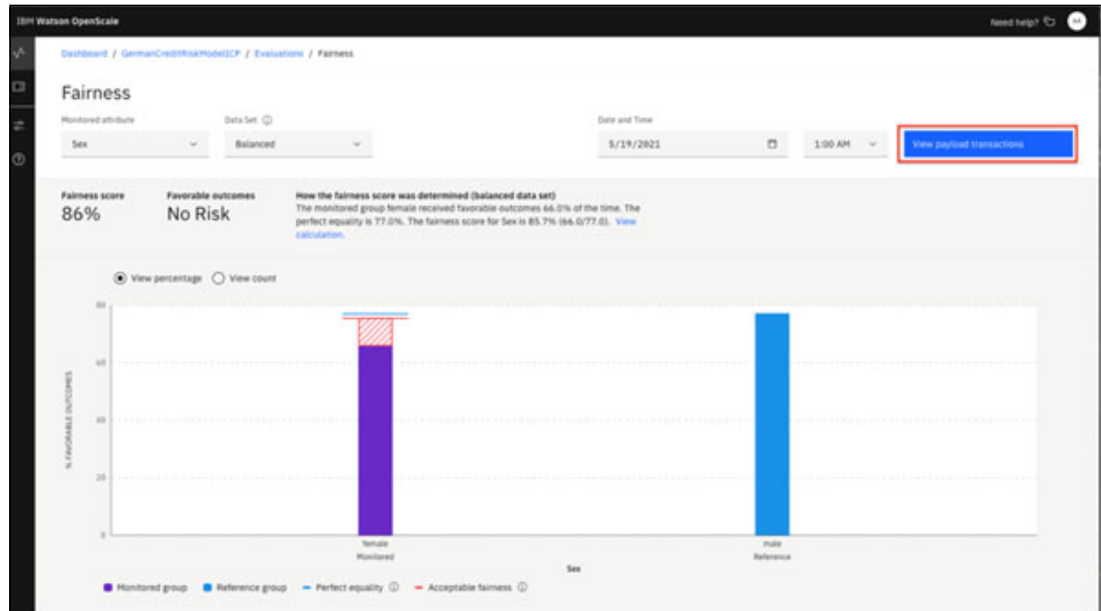


Figure 6-17 Watson OpenScale Evaluation monitor - Fairness

You can also explain the predictions of the model, where in you can see the relative weights of the most important features for the prediction (see Figure 6-18).



Figure 6-18 Watson OpenScale Explainability feature

Model monitoring requires the integration of various tools, technologies, and processes into the deployment pipeline. Automated monitoring systems, when they are combined with human oversight, ensure that models continue to deliver accurate, fair, and reliable predictions even as real-world conditions evolve.

In summary, model monitoring ensures that deployed models remain effective and aligned with operational requirements. It bridges the gap between model development and ongoing production, transforming models from static artifacts into dynamic, adaptable, and valuable assets within the MLOps ecosystem.

## 6.5 AI Governance and security

AI Governance and security are paramount considerations when machine learning models are deployed into production environments. As organizations increasingly rely on AI for critical decision-making, it becomes imperative to ensure ethical, legal, and secure practices are in place throughout the model's lifecycle. AI Governance and security are important in an MLOps lifecycle for the following reasons:

- ▶ **Ethical AI Governance.** Ethical AI Governance helps guard against the unintentional perpetuation of biases or discriminatory behaviors within AI models. It involves:
  - **Bias Detection and Mitigation**  
Mechanisms to detect and rectify biases in training data, ensuring that models make fair and equitable predictions.
  - **Fairness Assessments**  
Continuous assessments to gauge model fairness, preventing decisions that favor one group at the expense of others.
  - **Transparency and Explainability**  
Employing techniques like model agnostic algorithms to provide transparent and interpretable explanations for model predictions, fostering trust and accountability.
  - **Regulatory Compliance**  
The regulations that involve AI are evolving rapidly. AI Governance practices involve:
    - **Data Privacy Compliance**  
Implementing stringent data protection measures to comply with regulations like GDPR or HIPAA.
    - **Model Behavior Compliance**  
Ensuring that deployed models adhere to legal requirements, such as those governing financial decisions or health care treatments.
- ▶ **Data Security.**  
Data security is a linchpin in AI Governance and security within MLOps:
  - **Encryption and Access Controls.**  
Employing robust encryption techniques and access controls to safeguard sensitive data that is used in model training and inferencing.
  - **Secure Data Storage**  
Implementing secure storage protocols to protect data from unauthorized access or breaches.
- ▶ **Incident Response and Accountability.**  
Preparing for AI-related incidents is integral:
  - **Incident Response Plans.**  
Developing clear plans and protocols to address model failures, security breaches, or ethical lapses.
  - **Audit Trails.**
  - **Maintaining detailed audit trails** that record model development, validation, and deployment processes for accountability and compliance purposes.
  - **Access Control and Authentication**

- Strict access controls and authentication mechanisms ensure that only authorized personnel can interact with deployed models, preventing unauthorized usage or tampering.
- Monitoring and Anomaly Detection  
Automating continuous monitoring of deployed models to detect anomalies, security breaches, or performance degradation, enabling proactive responses.
- Documentation and Compliance Audits  
Comprehensive documentation of AI Governance and security practices aids in compliance audits and regulatory checks, ensuring that all processes are aligned with legal requirements.

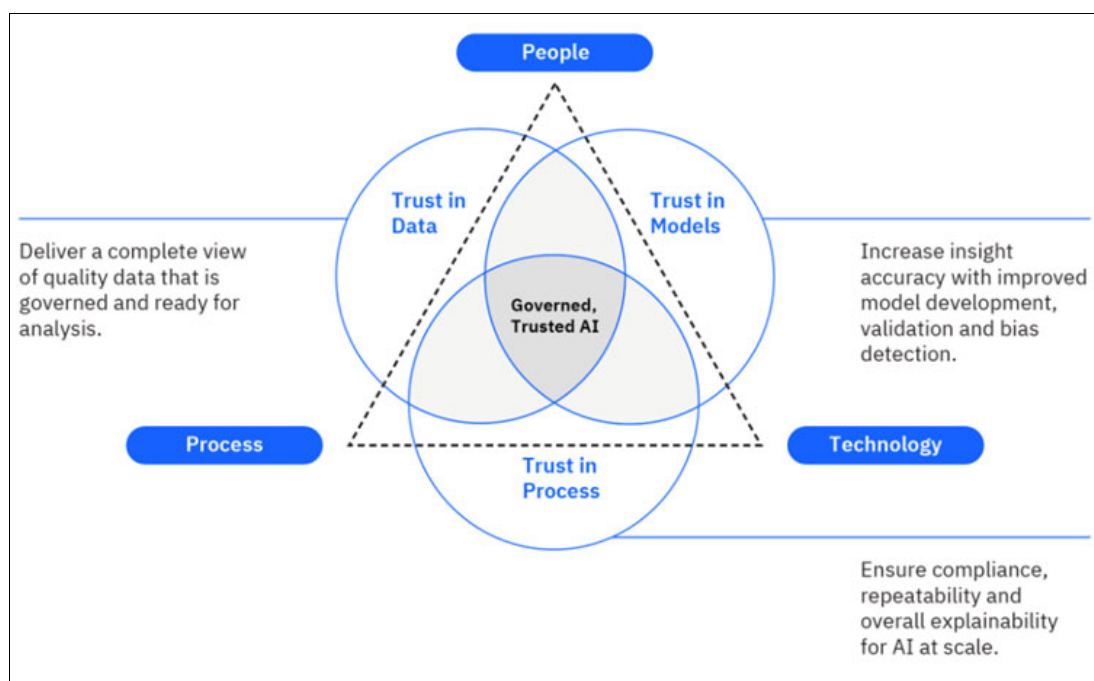


Figure 6-19 AI Governance on IBM Z for Trustworthy AI

AI Governance and security are not only static considerations. They are dynamic and integral components of the MLOps journey. By integrating ethical practices, robust security measures, and regulatory compliance into every stage of MLOps, organizations can harness the power of AI and maintain trust, transparency, and the highest standards of security and ethics.

## 6.6 How generative AI is evolving MLOps

The release of OpenAI's ChatGPT and IBM's watsonx sparked interests in AI capabilities across industries and disciplines and produce a variety of data types and can further develop the MLOps process. This technology, which is known as generative AI, can write software code, create images, produce a variety of data types, and further develop the MLOps process.

Generative AI is a type of deep-learning model that takes raw data, processes it, and “learns” to generate probable outputs. In other words, the AI model uses a simplified representation of the training data to create a new work that is similar, but not identical, to the original data. For example, by analyzing the language that is used by Shakespeare, a user can prompt a

generative AI model to create a Shakespeare-like sonnet on a specified topic to create an entirely new work.

Generative AI relies on foundation models to create a scalable process. As AI has evolved, data scientists have acknowledged that building AI models can take data, energy, and time. Building AI models includes compiling, labeling, and processing datasets that the models use to “learn” and includes energy to process the data and iteratively train the models. Foundation models try to solve this problem. A foundation model takes a massive quantity of data and by using self-supervised learning and transfer learning can take that data to create models for a wide range of tasks.

This advancement in AI means that datasets are not task-specific. The model can apply information that is used in one situation and apply it to another. Engineers use foundation models to create the training models for MLOps processes faster. They take the foundation model and fine-tune it using their own data, instead of taking their data and building a new model.

When you use MLOps, it is not just about the models. It is about the impact that those models have on the world. It is about streamlining the path from data to decisions, from algorithms to applications, and from insights to impact. Welcome to the world of MLOps, where the future is defined by the intelligent application of data-driven insights.



# Abbreviations and acronyms

|              |                                                     |               |                                                              |
|--------------|-----------------------------------------------------|---------------|--------------------------------------------------------------|
| <b>AI</b>    | Artificial intelligence                             | <b>PIPEDA</b> | Personal Information Protection and Electronic Documents Act |
| <b>AML</b>   | Anti-Money Laundering                               | <b>PMML</b>   | Predictive Model Markup Language                             |
| <b>AUC</b>   | Area under curve                                    | <b>RPOs</b>   | Replication with recovery point objectives                   |
| <b>CD</b>    | Continuous Deployment                               | <b>RTOs</b>   | Recovery time objectives                                     |
| <b>CI</b>    | Continuous Integration                              | <b>SAF</b>    | System Authorization Facility                                |
| <b>CP4D</b>  | Cloud Pak for Data                                  | <b>SCC</b>    | Security and Compliance Center                               |
| <b>DL</b>    | Deep learning                                       | <b>SHAP</b>   | Shapley Additive Explanations                                |
| <b>DLC</b>   | Deep Learning Compiler                              | <b>SLAs</b>   | service level agreements                                     |
| <b>DPU</b>   | Data processing unit                                | <b>SMF30</b>  | System Management Facility 30                                |
| <b>ELT</b>   | Extract, load, and transform                        | <b>SOR</b>    | System of Record                                             |
| <b>ETL</b>   | Extract, transform, and load                        | <b>SSL</b>    | Secure Sockets Layer                                         |
| <b>GANs</b>  | Generative adversarial networks                     | <b>TEE</b>    | Trusted Execution Environment                                |
| <b>GDPR</b>  | General Data Protection Regulation                  | <b>TLS</b>    | Transport Layer Security                                     |
| <b>GIGO</b>  | Garbage In, Garbage Out                             | <b>UI</b>     | User-friendly web interface                                  |
| <b>GLBA</b>  | Gramm-Leach-Bliley Act                              | <b>VAEs</b>   | Variational autoencoders                                     |
| <b>HIPAA</b> | Health Insurance Portability and Accountability Act | <b>WAS</b>    | WebSphere Application Server                                 |
| <b>HPCS</b>  | Hyper Protect Crypto Services                       | <b>WOLA</b>   | WebSphere Optimized Local Adapter                            |
| <b>HSMs</b>  | Hardware Security Modules                           | <b>zDLC</b>   | Z Deep Learning Compiler                                     |
| <b>IBM</b>   | International Business Machines Corporation         | <b>zIIP</b>   | Z Integrated Information Processor                           |
| <b>IDE</b>   | Integrated development environments                 | <b>zSCC</b>   | Z Security and Compliance Center                             |
| <b>ISDL</b>  | India Systems Development Lab                       |               |                                                              |
| <b>JDBC</b>  | Java Database Connectivity                          |               |                                                              |
| <b>JWT</b>   | JSON Web Token                                      |               |                                                              |
| <b>KPIs</b>  | Key performance indicators                          |               |                                                              |
| <b>LDA</b>   | Linear discriminant analysis                        |               |                                                              |
| <b>LIME</b>  | Local Interpretable Model-Agnostic Explanations     |               |                                                              |
| <b>LLM</b>   | Large language model                                |               |                                                              |
| <b>ML</b>    | Machine learning                                    |               |                                                              |
| <b>MLOps</b> | Machine learning operations                         |               |                                                              |
| <b>MLz</b>   | Machine Learning for z/OS                           |               |                                                              |
| <b>MVPs</b>  | Minimal viable products                             |               |                                                              |
| <b>NLP</b>   | Natural language processing                         |               |                                                              |
| <b>ODM</b>   | Operational Decision Manager                        |               |                                                              |
| <b>ODS</b>   | Operational data store                              |               |                                                              |
| <b>ONNX</b>  | Open Neural Network Exchange                        |               |                                                              |
| <b>PCA</b>   | Principal component analysis                        |               |                                                              |
| <b>PII</b>   | Personally Identifiable Information                 |               |                                                              |



# Related publications

The publications listed in this section are considered particularly suitable for a more detailed discussion of the topics covered in this book.

## IBM Redbooks

The following IBM Redbooks publications provide additional information about the topic in this document. Note that some publications referenced in this list might be available in softcopy only.

- ▶ *Solving challenges of instant payments by using AI on IBM zSystems*, REDP-5698
- ▶ *Accelerate Mainframe Application Modernization with Hybrid Cloud*, REDP-5705
- ▶ *Securely Leverage Open-Source Software with Python AI Toolkit for IBM z/OS*, REDP-5709
- ▶ *Mainframe Application Modernization Patterns for Hybrid Cloud*, SG24-8532

You can search for, view, download or order these documents and other Redbooks, Redpapers, Web Docs, draft and additional materials, at the following website:

[ibm.com/redbooks](https://ibm.com/redbooks)

## Help from IBM

IBM Support and downloads

[ibm.com/support](https://ibm.com/support)

Services from IBM Consulting

[ibm.com/services](https://ibm.com/services)

IBM Training

[ibm.com/training](https://ibm.com/training)











SG24-8552-01

ISBN 073846256X

Printed in U.S.A.

Get connected

